

ArenaX

[Recreational Activities Booking System]

Mini Project Report

Submitted by

Elizabeth Mary Abraham

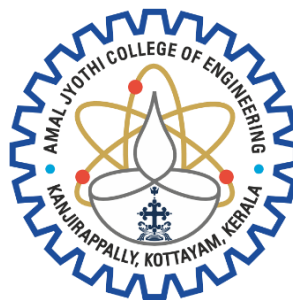
Reg. No.: AJC22MCA-I034

In Partial fulfillment for the Award of the Degree of

INTEGRATED MASTER OF COMPUTER APPLICATIONS

(INMCA)

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY



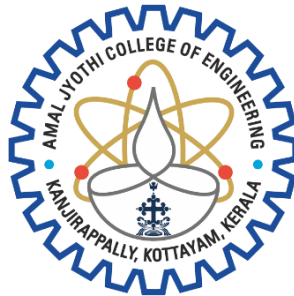
AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

[Affiliated to APJ Abdul Kalam Technological University, Kerala. Approved by AICTE, Accredited by NAAC. Koovappally, Kanjirappally, Kottayam, Kerala – 686518]

2024-2025

DEPARTMENT OF COMPUTER APPLICATIONS
AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS
KANJIRAPPALLY



CERTIFICATE

This is to certify that the Project report, “**ArenaX**” is the bona fide work of **ELIZABETH MARY ABRAHAM (Regno: AJC22MCA-I034)** carried out in partial fulfillment of the requirements for the award of the **Degree of Integrated Master of Computer Applications** at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, affiliated to **APJ Abdul Kalam Technological University**. The project was undertaken during the period from **January 01, 2025** to **April 23, 2025**.

Dr. Paulin Paul

Ms. Meera Rose Mathew

Internal Guide

Coordinator

Rev. Fr. Dr. Rubin Thottupurathu Jose

Head of the Department

DECLARATION

I hereby declare that the project report “**ARENAX**” is a bona fide work done at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, affiliated to **APJ Abdul Kalam Technological University**, towards the partial fulfillment of the requirements for the award of the **Integrated Master of Computer Applications (INMCA)** during the period from **January 01, 2025 to April 23, 2025**.

Date: 10-04-2025

ELIZABETH MARY ABRAHAM

KANJIRAPPALLY

Reg: AJC22MCA-I034

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. It has been said that gratitude is the memory of the heart. I wish to express my sincere gratitude to our Director (Administration) **Rev. Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department **Rev. Fr. Dr. Rubin Thottupurathu Jose** for helping us. I extend my whole hearted thanks to the project coordinator **Ms. Meera Rose Mathew** for her valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide **Dr. Paulin Paul** for her inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

ELIZABETH MARY ABRAHAM

ABSTRACT

The proposed system titled “ArenaX” is an advanced easy-to-use platform designed for booking and managing different indoor and outdoor games, fitness facilities, catering to sports enthusiasts and various recreational activities for users. The system aims to develop a user-friendly platform that facilitate a range of physical, leisure and fitness activities. A registered users can quickly browse through available events or activities and reserve time slots/resources that suits their schedules and interests. The proposed system has three user-based modules such as: Admin, Manager and User module. The admin module manages the entire system, managing each activities details, registrations, membership, scheduling events and updations. Based on the kind of membership the user is opting such as: Normal, Standard, Premium the user’s module has user’s login, profile updations, reservations for games/events/resources, view transaction history. In addition to this user also hence providing to post suggestion, mark feedback, and file complaints. In conclusion, “ArenaX” simplifies the booking and management of sports and recreational activities, delivering a smooth experience for users and administrators alike.

CONTENT

SL. NO	TOPIC	PAGE NO
1	INTRODUCTION	1
1.1	PROJECT OVERVIEW	2
1.2	PROJECT SPECIFICATION	2
2	SYSTEM STUDY	3
2.1	INTRODUCTION	4
2.2	EXISTING SYSTEM	4
2.2.1	NATURAL SYSTEM STUDIED	5
2.2.2	DESIGNED SYSTEM STUDIED	5
2.3	DRAWBACKS OF EXISTING SYSTEM	5
2.4	PROPOSED SYSTEM	6
2.5	ADVANTAGES OF PROPOSED SYSTEM	7
3	REQUIREMENT ANALYSIS	8
3.1	FEASIBILITY STUDY	9
3.1.1	ECONOMICAL FEASIBILITY	9
3.1.2	TECHNICAL FEASIBILITY	10
3.1.3	BEHAVIORAL FEASIBILITY	10
3.1.4	FEASIBILITY STUDY QUESTIONNAIRE	11
3.2	SYSTEM SPECIFICATION	16
3.2.1	HARDWARE SPECIFICATION	16
3.2.2	SOFTWARE SPECIFICATION	16
3.3	SOFTWARE DESCRIPTION	16
3.3.1	HTML	16
3.3.2	CSS	16
3.3.3	JQUERY	17
3.3.4	JAVASCRIPT	17
3.3.5	PHP	17
3.3.6	MYSQL	17
4	SYSTEM DESIGN	18
4.1	INTRODUCTION	19
4.2	UML DIAGRAM	19
4.2.1	USE CASE DIAGRAM	20

4.2.2	SEQUENCE DIAGRAM	22
4.2.3	STATE CHART DIAGRAM	24
4.2.4	ACTIVITY DIAGRAM	25
4.2.5	CLASS DIAGRAM	26
4.2.6	OBJECT DIAGRAM	27
4.2.7	COMPONENT DIAGRAM	27
4.2.8	DEPLOYMENT DIAGRAM	29
4.3	USER INTERFACE DESIGN USING FIGMA	30
4.4	DATABASE DESIGN	31
4.4.1	RELATIONAL DATABASE MANAGEMENT SYSTEM	31
4.4.2	NORMALIZATION	32
4.4.3	SANITIZATION	36
4.4.4	INDEXING	36
4.5	TABLE DESIGN	37
5	SYSTEM TESTING	47
5.1	INTRODUCTION	48
5.2	TEST PLAN	49
5.2.1	UNIT TESTING	49
5.2.2	INTEGRATION TESTING	50
5.2.3	VALIDATION TESTING	50
5.2.4	USER ACCEPTANCE TESTING	50
5.2.5	AUTOMATION TESTING	51
5.2.6	SELENIUM TESTING	51
6	IMPLEMENTATION	63
6.1	INTRODUCTION	64
6.2	IMPLEMENTATION PROCEDURE	64
6.2.1	USER TRAINING	65
6.2.2	TRAINING ON APPLICATION SOFTWARE	65
6.2.3	SYSTEM MAINTENANCE	65
6.2.4	HOSTING	66
7	CONCLUSION & FUTURE SCOPE	68
7.1	CONCLUSION	69
7.2	FUTURE SCOPE	69
8	BIBLIOGRAPHY	70

9	APPENDIX	72
9.1	SAMPLE CODE	73
9.2	SCREEN SHOTS	134

List of Abbreviation

PHP	-	PHP: Hyper Text Preprocessor
HTML	-	Hyper Text Markup Language
CSS	-	Cascading Style Sheet
SQL	-	Structured Query Language
UML	-	Unified Modeling Language

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

ArenaX is a smart booking system designed to streamline reservations for recreational activities. Users can easily book activities, join events, and manage memberships with ease. The system offers three membership types, each providing different levels of access to activities and events.

With dedicated modules for users, managers, and administrators, ArenaX ensures a seamless experience. The User Module allows users to register, browse activities, make bookings, and upgrade memberships. The Manager Module handles activity management, event coordination, and user assistance, ensuring smooth operations. The Admin Module oversees the entire system, managing users, memberships, and revenue tracking.

By automating the booking process and optimizing membership management, ArenaX enhances efficiency, minimizes manual efforts, and offers a hassle-free way to engage in recreational activities.

1.2 PROJECT SPECIFICATION

ArenaX is an advanced platform developed to simplify the booking process for recreational activities. This solution is built considering system requirements, user expectations, and efficient activity management. It enables users to schedule activities, join events, and handle their memberships seamlessly. The system offers three membership types, each with distinct privileges. Normal members can make single bookings with payment required for each. To access recurring bookings, events, or additional activities, they must upgrade. Standard members have single bookings included in their membership, allowing them to book without additional payment. They can access recurring bookings and events but must pay separately. To use the calendar feature or access premium-only activities, an upgrade to Premium is required. Premium members enjoy full access to all activities, events, and the calendar without additional payments, ensuring a seamless booking experience. By automating the booking process and optimizing membership management, ArenaX enhances operational efficiency, reduces manual effort, and provides users with a convenient platform for engaging in recreational activities.

CHAPTER 2

SYSTEM STUDY

2.1 INTRODUCTION

Previously, recreational activity bookings were entirely manual, requiring significant time and effort. Users had to visit booking centers in person, fill out paperwork, and wait for confirmations. Managing schedules, memberships, and payments manually led to frequent errors and inefficiencies, causing delays and confusion. Cancellations and rescheduling were also difficult to manage, often requiring additional paperwork and manual adjustments. As demand for recreational activities grew, the manual system became increasingly overwhelmed, leading to frustration for both users and staff.

Staff had to meticulously track reservations, verify user eligibility, and update records manually. Any mistakes in the process could result in double bookings or lost reservations. Additionally, generating reports and maintaining records was tedious, making overall management cumbersome and time-consuming. The lack of a centralized system meant that staff had to cross-check multiple physical records, further slowing down operations. Limited accessibility to booking information also made it difficult for users to plan their schedules effectively.

2.2 EXISTING SYSTEM

In our existing system, all booking transactions for recreational activities are managed manually, making the process slow and inefficient. Users must visit booking centers or contact staff to check activity availability, submit requests, and complete payments. This leads to delays and the risk of double bookings or scheduling conflicts. Another major drawback is the time-consuming process of tracking memberships, payments, and activity participation. Administrators must manually verify bookings, update records, and generate reports, which can take an entire day to complete. Rescheduling or cancelling activities also requires additional paperwork, further adding to the workload. Due to these inefficiencies, managing recreational activities has become increasingly difficult as demand grows. After conducting a feasibility study, we decided to transition from the manual booking system to a computerized one, ensuring a more efficient, accurate, and user-friendly experience.

2.2.1 NATURAL SYSTEM STUDIED

The existing recreational activity booking system lacks a centralized platform, making it difficult to check availability, track reservations, and manage payments. Users manually verify membership privileges, causing confusion and delays. Administrators struggle with booking management, leading to inefficiencies. ArenaX streamlines this process, allowing users to book activities, join events, and manage memberships with ease. Normal Membership offers limited activities with single paid bookings, while recurring bookings and premium activities require an upgrade. Standard Membership includes free single bookings, but recurring bookings, events, and calendar access need payment or an upgrade. Premium Membership provides full access to all activities, events, and features without extra charges. By automating tracking and centralizing operations, ArenaX eliminates errors, enhances user experience, and simplifies administrative tasks, making recreational activity management more efficient.

2.2.2 DESIGNED SYSTEM STUDIED

The ArenaX system displays available recreational activities, categorized into indoor, outdoor, and fitness. The website does not offer free online membership upgrades; users must manually opt for higher plans. All users can browse activities, but booking privileges vary based on membership type. The platform includes an event registration system and provides premium members with access to a personal booking calendar.

2.3 DRAWBACKS OF EXISTING SYSTEM

- Lack of automated membership upgrades, requiring manual intervention.
- Difficulty in tracking available activities based on membership level.
- No centralized notification system for upcoming events and bookings.
- Limited customization options for users in managing their schedules and preferences.
- No integrated payment gateway for seamless transactions.
- Manual verification of user eligibility for activities and events.
- Limited reporting and analytics for administrators to track user engagement.
- No mobile app support, restricting accessibility on the go.

2.4 PROPOSED SYSTEM

The proposed ArenaX booking system is developed after analyzing the issues and limitations of the existing system. ArenaX is a digital platform designed to manage and streamline recreational activity bookings and event participation efficiently. The system helps users browse activities, book sessions, and track their membership benefits without hassle. The primary objective of this project is to create an easy-to-use and user-friendly application that simplifies the booking process for both users and administrators. Different modules have been incorporated to manage various functions within the system. The platform enables users to register, upgrade their membership, and book activities according to their membership privileges. The system also provides automated booking management, event registration, and a secure payment gateway to ensure seamless transactions. Premium members benefit from additional features, including access to all activities and an integrated calendar for efficient schedule management. The main feature of this system is that it provides a clear and structured booking process, eliminating the confusion of manual reservations. Users can easily access all available activities based on their membership type without unnecessary complications. Additionally, the platform effectively maintains member details, booking history, and event participation records, ensuring a smooth and hassle-free experience.

The major modules included in the system are:

- **Admin**
- **Users**
- **Event Management**
- **Membership & Booking System**
- **Payment Gateway**
- **Calendar & Scheduling System**

2.5 ADVANTAGES OF PROPOSED SYSTEM

- Automated booking and membership upgrades reduce manual effort.
- Secure payment gateway ensures hassle-free transactions.
- Centralized system for easy tracking of activities, events, and bookings.
- Personalized dashboard for users to manage their schedules and bookings.
- Real-time notifications and reminders for events and activity schedules.
- Seamless access to activities based on membership type without confusion.
- Efficient management of event registrations and participation records.
- Mobile-friendly interface for convenient access from any device.
- Improved user experience with a structured and user-friendly interface.
- Enhanced administrative control through analytics and reporting tools.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

The proposed ArenaX booking system is a digital platform designed to streamline recreational activity bookings and event participation. It enables users to browse activities, book sessions, and manage memberships efficiently, providing a user-friendly experience for both users and administrators. The system includes modules for membership management, booking automation, event registration, and secure payments. Premium members enjoy additional benefits like access to all activities and an integrated calendar for scheduling. A feasibility study ensures the project's viability by evaluating technical, economic, and operational aspects. It assesses system development feasibility, cost-effectiveness, and the platform's ability to enhance user experience and streamline bookings. This study helps determine the system's success and guides implementation planning. It cannot be simple to determine whether or not a proposed project is worthwhile pursuing without a feasibility study. The document provides the feasibility of the project that is being designed and lists. Various areas that were considered very carefully during the feasibility study of this project such as Technical, Economic and Operational feasibility. The following are its features:

3.1.1 ECONOMICAL FEASIBILITY

The ArenaX website, functions as a cost-efficient, service-oriented platform focused on delivering value and convenience to its users. By utilizing open-source technologies, development and software costs are significantly reduced, allowing us to stay well within a sustainable financial threshold. The platform replaces labor-intensive, manual booking processes with automated systems, thus lowering operational costs and enabling streamlined service delivery. With online bookings and integrated payments, users experience faster service while staff dependency is minimized. Additionally, user-centric features like notifications, calendars (for premium users), and feedback mechanisms help increase customer satisfaction and retention. From a business standpoint, the services offered under different memberships are well-suited to attract a broad user base while maintaining profitability. ArenaX also addresses user expectations through secured transactions, flexible booking options, and utility-rich features, potentially surpassing competitor platforms. Overall, the low-cost, high-utility nature of the system ensures that it can deliver quality services while maintaining economic viability and supporting business scalability.

3.1.2 TECHNICAL FEASIBILITY

The ArenaX booking system is technically feasible as it leverages robust, scalable, and modern web technologies to deliver high performance and reliability. Built on a secure cloud-based infrastructure, the platform supports real-time booking, automated scheduling, and access for users across various devices. The architecture is optimized for scalability, ensuring that the system can handle growing traffic and increasing data volume without performance issues. It integrates effortlessly with third-party services such as secure payment gateways, event registration APIs, and calendar tools. Security protocols including SSL encryption, and role-based access control are employed to safeguard user data and transactions. The backend systems are developed using efficient, well-supported frameworks that ensure long-term maintainability. Additionally, continuous monitoring and error handling mechanisms enhance system reliability and uptime. ArenaX is designed to be compatible with future technological advancements, ensuring that it remains adaptable and technically sound for years to come.

3.1.3 BEHAVIORAL FEASIBILITY

The ArenaX system is designed with user behaviour in mind, focusing on intuitive navigation and ease of access to encourage engagement. Users are more likely to adopt the platform as it eliminates complex booking procedures and introduces a clean, responsive interface. The system enhances interaction by offering personalized membership benefits, timely notifications, and recurring booking options that align with user preferences. Behavioural trends suggest that users value transparency, efficiency, and convenience—all of which are integral to ArenaX. By simplifying the booking journey and supporting flexible membership upgrades, ArenaX increases satisfaction and loyalty. Its user-centric approach encourages repeated visits, while offering consistent support through seamless customer service and feedback integration. As the system adapts to user behaviour, it promotes long-term usage and acceptance among a wide range of audiences. The platform's design fosters trust and user confidence, ensuring its behavioural feasibility across diverse recreational needs.

3.1.4 FEASIBILITY STUDY QUESTIONNAIRE

1. How would the organization cope if this system was not implemented?

If the ArenaX system is not implemented, the organization would continue relying on traditional or manual methods for managing bookings and events. This would lead to slower processes, increased human error, limited scalability, and lower user satisfaction. Growth would be restricted, and maintaining customer engagement would become increasingly challenging over time.

2. What are the problems with current processes and how would a new system help alleviate these problems?

The current manual or semi-digital booking processes are time-consuming, error-prone, and lack real-time updates. Users face delays in bookings and limited access to event information. ArenaX solves these issues by automating the process, enabling instant bookings, providing secure payments, and offering membership-based access for a smoother user experience.

3. What direct contribution will the system make to the business objectives and requirements?

The ArenaX system directly supports business goals by automating bookings, enhancing user engagement, and offering tiered memberships for revenue generation. It improves service efficiency, reduces manual workload, and provides valuable insights through data tracking. These contributions help achieve higher customer satisfaction, operational scalability, and long-term business sustainability.

4. Can information be transferred to and from other organizational systems?

Yes, the ArenaX system is designed to allow information exchange with other organizational systems through APIs and data integration tools. This enables smooth data transfer for user profiles, event details, payment records, and scheduling. Such interoperability ensures efficiency, centralized data management, and improved coordination across related platforms or departments.

5. Does the system require technology that has not previously been used in the organization?

Yes, the ArenaX system introduces several modern technologies not previously used in the organization. These include secure online payment gateways, calendar integration for scheduling,

and real-time booking management. It also uses a cloud-based backend and responsive web design to ensure accessibility across devices, offering a more advanced digital experience.

6.What must be supported by the system and what need not be supported?

The ArenaX system must support user registration, membership upgrades, activity bookings, event participation, and secure payments. It should also support calendar integration for Premium users and admin-level management of users and content. It need not support offline bookings, third-party activity listings, or services outside recreational and event-based functions.

REQUIREMENT GATHERING

1. Project Overview:

The ArenaX Recreational Activity Booking System is a user-friendly web-based platform designed to simplify the process of booking indoor, outdoor, and fitness activities. It supports event participation and offers structured membership plans like Normal, Standard, and Premium, each with its own set of privileges. The goal of ArenaX is to digitize and streamline activity management for users and administrators, providing a seamless booking experience, secure payments, and increased accessibility to recreational services. It enhances user satisfaction, saves time, and promotes active participation in recreational events.

2. System Scope:

The ArenaX system is intended for full-scale implementation to support commercial and institutional use. It enables users to browse, book, and participate in various recreational activities while offering comprehensive tools for administrators to manage events, bookings, and memberships efficiently.

3. Target Audience:

- Individuals seeking to book indoor, outdoor, or fitness activities.
- Users interested in participating in recurring events or specialized fitness programs.
- Admins responsible for overseeing bookings, event management, and membership control.
- Institutions or clubs offering recreational facilities and services.

4. Modules:

User Functions:

Users can sign up and choose from three membership plans (Normal, Standard, Premium). Based on their membership, they can book activities, register for events, make payments, and manage their bookings via calendar tools (Premium only).

Admin Functions:

Admins oversee user registrations, approve and manage events and activities, handle payments, manage system data, and ensure smooth operations across the platform.

Membership Management:

Users can upgrade their membership for additional benefits. Each plan offers different levels of access to bookings, event registrations, calendar tools, and payment privileges.

5. User Roles:

- **Admin:** Full control over the system including managing users, bookings, events, and payments.
- **Normal Member:** Can access only selected activities; payment required for bookings and events.
- **Standard Member:** Booking is included in the membership; access to more activities, events with payment, calendar upgrades require premium.
- **Premium Member:** Full access to all services including activities, events, and calendar without additional payments.

6. System Ownership:

ArenaX is owned and managed by a single administrator or organization responsible for the platform's development, maintenance, membership structure, event handling, and data security.

7. Industry/Domain:

Recreation and Wellness Industry – focused on delivering digital solutions for fitness centers, activity clubs, community event organizers, and recreational institutions.

8.Data Collection Contacts:

Name: Thomas Abraham

Role: Owner of UFS (United Fitness Studio) Gym

Contact: thomaskol23.manu@gmail.com, 9591920423

9. Questionnaire for Data Collection

1. What is the purpose of the ArenaX booking system?

The ArenaX booking system aims to make recreational activity booking simple and efficient. It allows users to search, select, and book indoor, outdoor, and fitness activities online. By managing everything digitally, ArenaX saves users time and helps administrators organize activities and memberships with ease.

2. How does the system generate revenue?

ArenaX earns revenue through its three-tier membership plans: Normal, Standard, and Premium. Each membership unlocks different features. The platform also collects fees for certain bookings and event registrations based on the membership plans. These services help generate continuous income while keeping the platform affordable and accessible.

3. What are the key features of the system?

Key features include membership management, activity and event bookings, recurring booking options, and secure online payments. Premium users get a calendar to manage their schedules. Users can also view their scheduled bookings and also the payment histories. These tools make the user experience smooth and provide everything needed to manage recreational bookings in one place.

4. How does the system ensure cost-effectiveness?

The system uses open-source technology, which reduces software costs. It also automates bookings and payments, cutting down manual work. These features save both time and money for users and administrators, making ArenaX a cost-effective solution for managing recreational activities.

5. What makes the system user-friendly?

ArenaX has a clean and easy-to-use interface. Users can quickly book activities, view available slots, and manage their memberships. Automated reminders and smooth navigation help users enjoy a stress-free booking experience, even if they aren't familiar with technology.

6. What security measures are in place?

The platform uses encryption to protect user data and integrates with secure payment gateways. Role-based access control ensures that only authorized users can access sensitive information. These measures keep user data safe and build trust among users.

7. How does ArenaX handle user feedback and support?

ArenaX includes a direct feedback system where users can share their experience or report issues. This feedback is reviewed regularly by the support team to improve services. Prompt responses to concerns and constant updates based on user input help build trust and ensure long-term user satisfaction.

8. How does the system improve efficiency?

The platform reduces manual work by automating bookings, payments, and membership tracking. Real-time updates and reminders make things run more smoothly. This helps both users and staff save time and stay organized.

9. What technology stack is used for development?

ArenaX is developed using modern web frameworks that are secure, scalable, and easy to maintain. These tools ensure that the system performs well, is reliable, and can adapt to future improvements or changes in technology.

10. What are the benefits for premium members?

Premium members get full access to all activities and events with no extra fees. They can also use a calendar to manage their schedules easily. This membership is ideal for users who want complete convenience and faster, unlimited access to all services.

3.2 SYSTEM SPECIFICATION

3.2.1 Hardware Specification

Processor - Intel core i3 and above

RAM - 4 GB or Higher

Hard disk - Minimum 100 GB Free Space

3.2.2 Software Specification

Front End - HTML5, CSS, JavaScript, AJAX, jQuery

Back End - PHP

Database - MySQL

Client on PC - Windows 7 and above.

Technologies used - JS, HTML5, AJAX, jQuery, PHP, CSS

3.3 SOFTWARE DESCRIPTION

3.3.1 HTML

HTML (Hypertext Markup Language) is the most basic building block of the Web. HTML is a fairly simple language made up of elements, which can be applied to pieces of text to give them different meaning in a document, structure a document into logical sections, and embed content such as images and videos into a page. In the proposed system, HTML 5 is used to carry out the design part of the webpage. HTML validations are also used.

3.3.2 CSS

While HTML is used to define the structure and semantics of your content, CSS is used to style it and lay it out. For example, you can use CSS to alter the font, colour, size, and spacing of your content, split it into multiple columns, or add animations and other decorative features. In the proposed system CSS 3 is used. Animation using CSS is also used to give the website a unique look & feel.

3.3.3 jQuery

jQuery is a lightweight, "write less, do more", JavaScript library. The purpose of jQuery is to make it much easier to use JavaScript on your website. jQuery takes a lot of common tasks that require many lines of JavaScript code to accomplish, and wraps them into methods that you can call with a single line of code. jQuery also simplifies a lot of the complicated things from JavaScript, like AJAX calls and DOM manipulation. The jQuery library contains the following features: HTML/DOM manipulation, CSS manipulation, HTML event methods, Effects and animations, AJAX

3.3.4 JavaScript

JavaScript was initially created to “make web pages alive”. The programs in this language are called scripts. They can be written right in a web page’s HTML and run automatically as the page loads. In the proposed system loading screen, different popup animations are implemented using Java Script.

3.3.5 PHP

PHP is a server scripting language used for making dynamic web pages. That means PHP allows you to use scripts on a web server to produce a response customized for each client’s (user’s) request. The proposed system developed in PHP 7 and is hosted in a local server (Local Host). All database interactions are carried out using PHP.

3.3.6 MySQL

MySQL is a database management system that allows you to manage relational databases. It is open-source software backed by Oracle. All data related with the system is stored and manages using MySQL database.

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

Design is the first step into the development phase for any engineered product system. Design is a creative process. A good design is the key to effective system. The term “design” is defined as “the process of applying various techniques and principles for the purpose of defining a process or a system in sufficient detail to permit its physical realization”. It may be defined as a process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient detail to permit its physical realization. Software design sits at the technical kernel of the software engineering process and is applied regardless of the development paradigm that is used. The system design develops the architectural detail required to build a system or product. As in the case of any systematic approach, this software too has undergone the best possible design phase fine tuning all efficiency, performance and accuracy levels. The design phase is a transition from a user-oriented document to a document to the programmers or database personnel. System design goes through two phases of development: Logical and Physical Design.

4.2 UML DIAGRAM

UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems. UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997. UML stands for Unified Modeling Language. UML is different from the other common programming languages such as C++, Java, COBOL, etc. UML is a pictorial language used to make software blueprints. UML can be described as a general-purpose visual modeling language to visualize, specify, construct, and document software system. Although UML is generally used to model software systems, it is not limited within this boundary. It is also used to model non- software systems as well. For example, the process flow in a manufacturing unit, etc. UML is not a programming language but tools can be used to generate code in various languages using UML diagrams. UML has a direct relation with object-oriented analysis and design. After some standardization, UML has become an OMG standard. All the elements, relationships are used to make a complete UML diagram and the diagram represents a system. The visual effect of the UML diagram is the most important part of the entire process. All the other elements are used to make it complete.

UML includes the following nine diagrams:

- Class diagram
- Object diagram

- Use case diagram
- Sequence diagram
- Activity diagram
- State chart diagram
- Deployment diagram
- Component diagram

4.2.1 USE CASE DIAGRAM

A use case diagram is a visual representation of the interactions between system components. A approach for identifying, outlining, and organizing system requirements is called a use case. The word "system" here refers to a thing that is being created or run, like a website for mail-order product sales and services. UML (Unified Modeling Language), a standard language for the modelling of real-world objects and systems, uses use case diagrams. The planning of general requirements, the validation of a hardware design, the testing and debugging of a software product in development, the creation of an online help reference, or the completion of a job focused on customer support are all examples of system objectives. For instance, use cases in a product sales context can involve customer service, item ordering, catalogue updating, and payment processing. There are four elements in a use case diagram.

- The boundary, which defines the system of interest in relation to the world around it.
- The actors, usually individuals involved with the system defined according to their roles.
- The use cases, which are the specific roles are played by the actors within and around the system.
- The relationships between and among the actors and the use cases.

Use case diagrams are drawn to capture the functional requirements of a system. After identifying the above items, we have to use the following guidelines to draw an efficient use case diagram.

- The name of a use case is very important. The name should be chosen in such a way so that it can identify the functionalities performed.
- Give a suitable name for actors.
- Show relationships and dependencies clearly in the diagram.
- Do not try to include all types of relationships, as the main purpose of the diagram is to identify the requirements.
- Use notes whenever required to clarify some important points.

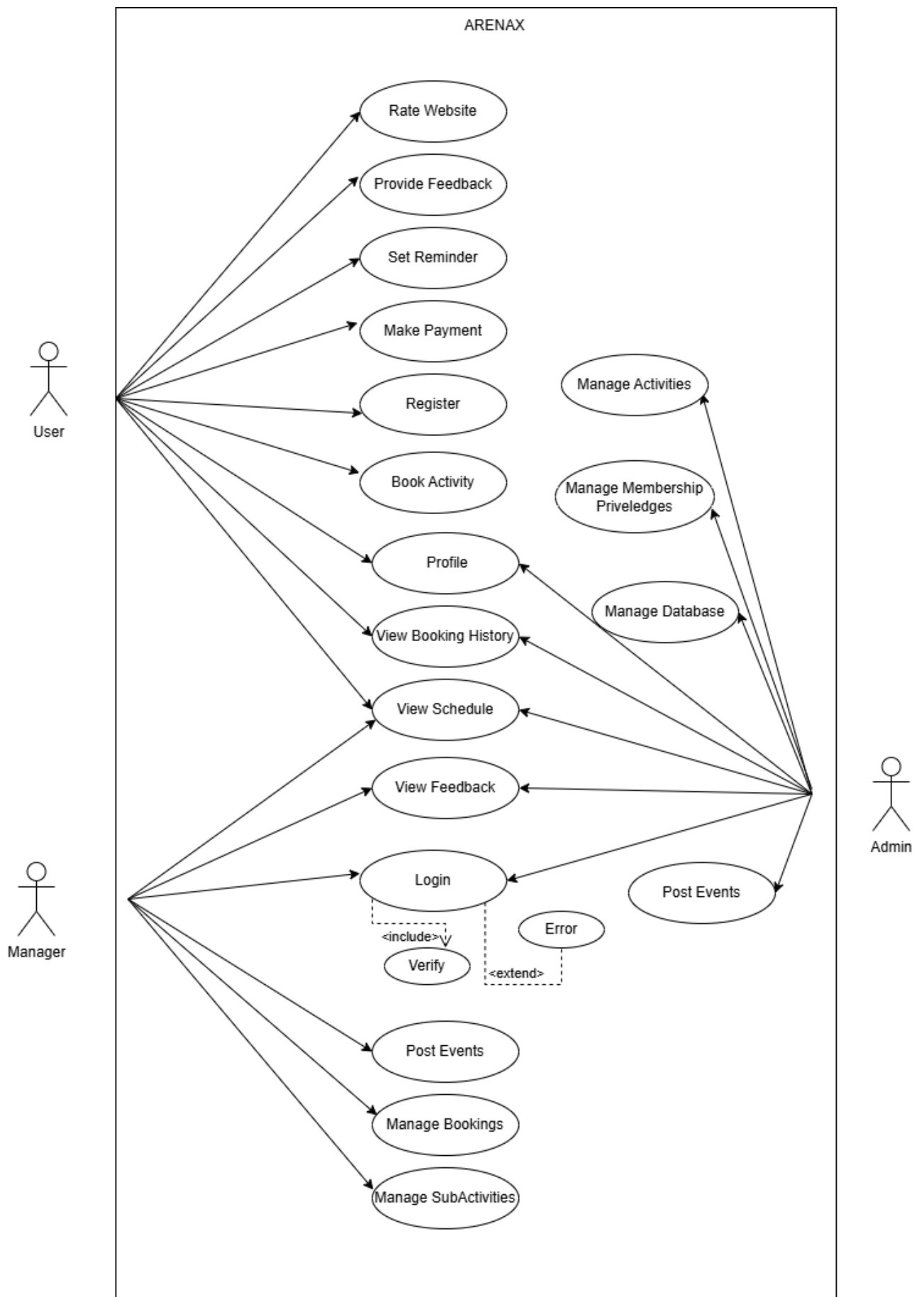


Figure 1: Use Case Diagram

4.2.1 SEQUENCE DIAGRAM

A sequence diagram essentially shows how things interact with one another sequentially, or the order in which these interactions occur. A sequence diagram can also be referred to as event diagrams or event scenarios. Sequence diagrams show the actions taken by the components of a system in chronological order. Businesspeople and software engineers frequently use these diagrams to record and comprehend the requirements for new and current systems.

Sequence Diagram Notations –

- **Actors** – In a UML diagram, an actor represents a particular kind of role that interacts with the system and its objects. An actor is always beyond the purview of the system that we want to use the UML diagram to represent. We employ actors to portray a variety of roles, including those of human users and other outside subjects. In a UML diagram, an actor is represented using a stick person notation. In a sequence diagram, there might be several actors.
- **Lifelines** – A lifeline is a named element in a sequence diagram that represent an individual participant. So, in a sequence diagram, each incident is represented by a lifeline. A sequence diagram's lifeline elements are at the top.
- **Messages** – Messages are used to show how objects communicate with one another. The messages are displayed on the lifeline in chronological sequence. Arrows are how messages are represented. A sequence diagram's main components are lifelines and messages.

Messages can be broadly classified into the following categories:

- Synchronous messages
- Asynchronous Messages
- Create message
- Delete Message
- Self-Message
- Reply Message
- Found Message
- Lost Message
- **Guards** – To model conditions we use guards in UML. They are used when we need to restrict the flow of messages on the pretext of a condition being met. Guards play an important role in letting software developers know the constraints attached to a system or a particular process.

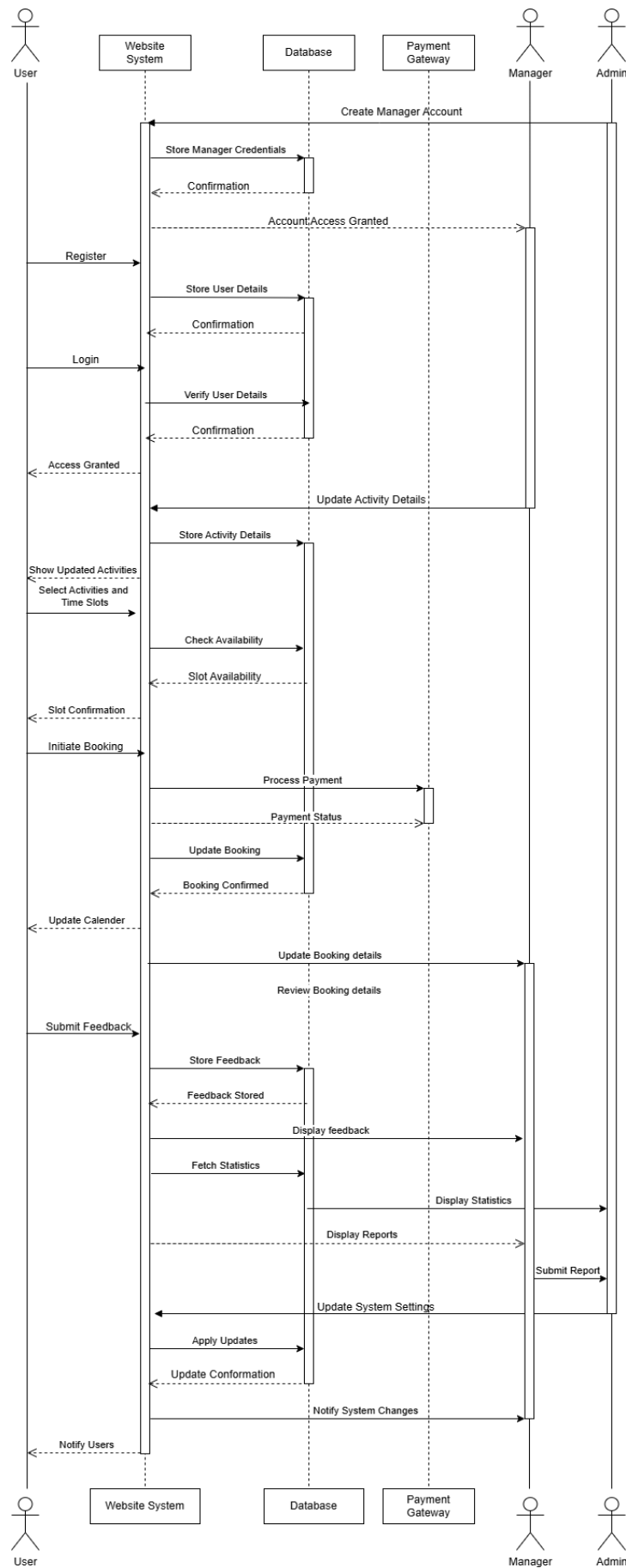


Figure 2: Sequence Diagram

4.2.3 STATE CHART DIAGRAM

A state diagram, also known as a state machine diagram or state chart diagram, is an illustration of the states an object can attain as well as the transitions between those states in the Unified Modeling Language (UML).

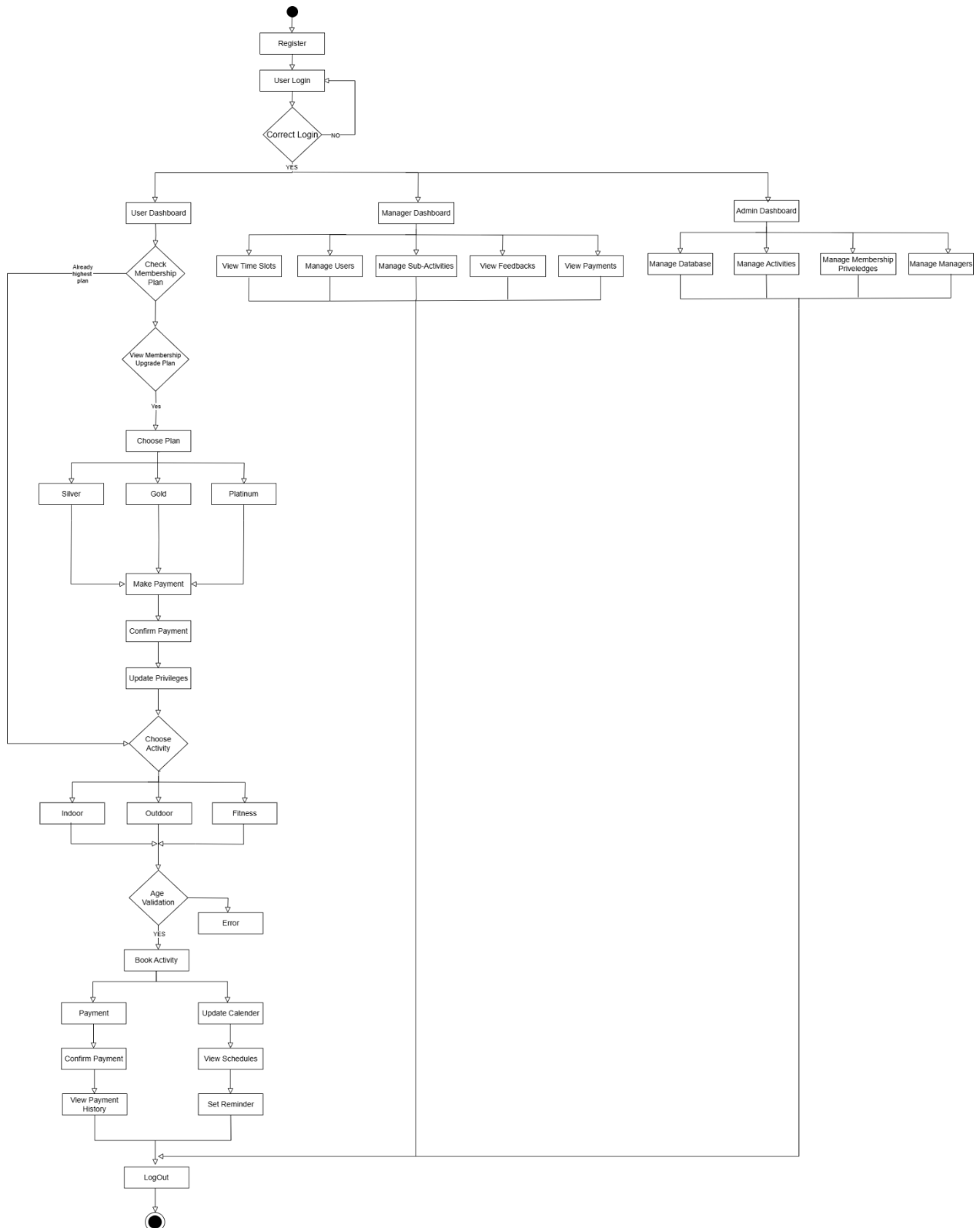


Figure 3: State Chart Diagram

4.2.4 ACTIVITY DIAGRAM

Another crucial UML diagram for describing the system's dynamic elements is the activity diagram. An activity diagram is essentially a flowchart that shows how one activity leads to another. The action might be referred to as a system operation. One operation leads to the next in the control flow. This flow may be parallel, contemporaneous, or branched. Activity diagrams use many features, such as fork, join, etc., to cope with all types of flow control. An activity diagram is a behavioral diagram i.e., it depicts the behavior of a system. An activity diagram portrays the control flow from a start point to a finish point showing the various decision paths that exist while the activity is being executed.

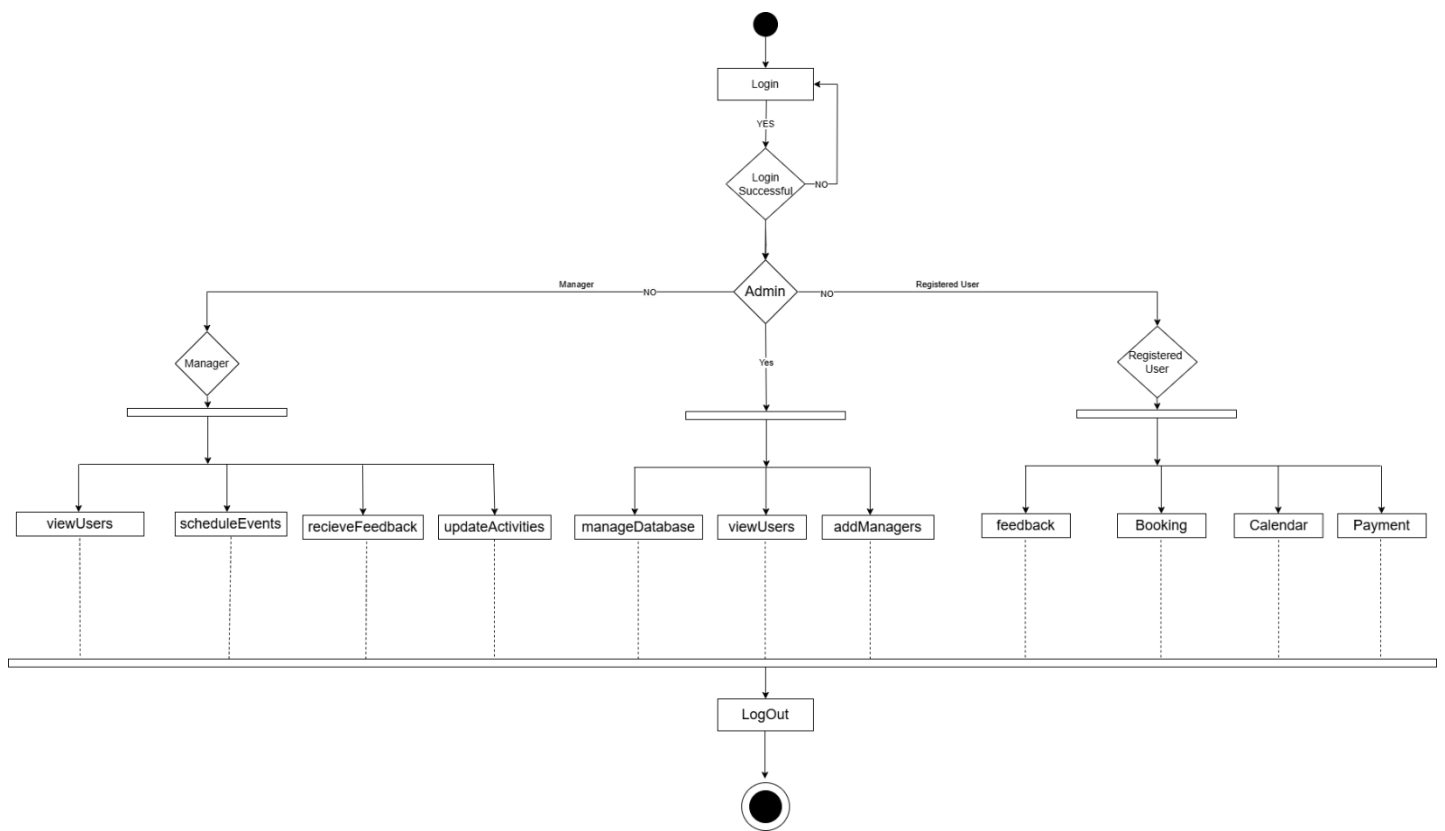


Figure 4: Activity Diagram

4.2.5 CLASS DIAGRAM

Class diagrams is a Static diagram. It represents the application's static view. Class diagrams are used to create executable code for software applications as well as for visualizing, explaining, and documenting various elements of systems. The characteristics and functions of a class are described in a class diagram, along with the restrictions placed on the system. Because they are the only UML diagrams that can be directly transferred to object-oriented languages, class diagrams are frequently employed in the modelling of object-oriented systems. A collection of classes, interfaces, affiliations, collaborations, and constraints are displayed in a class diagram.

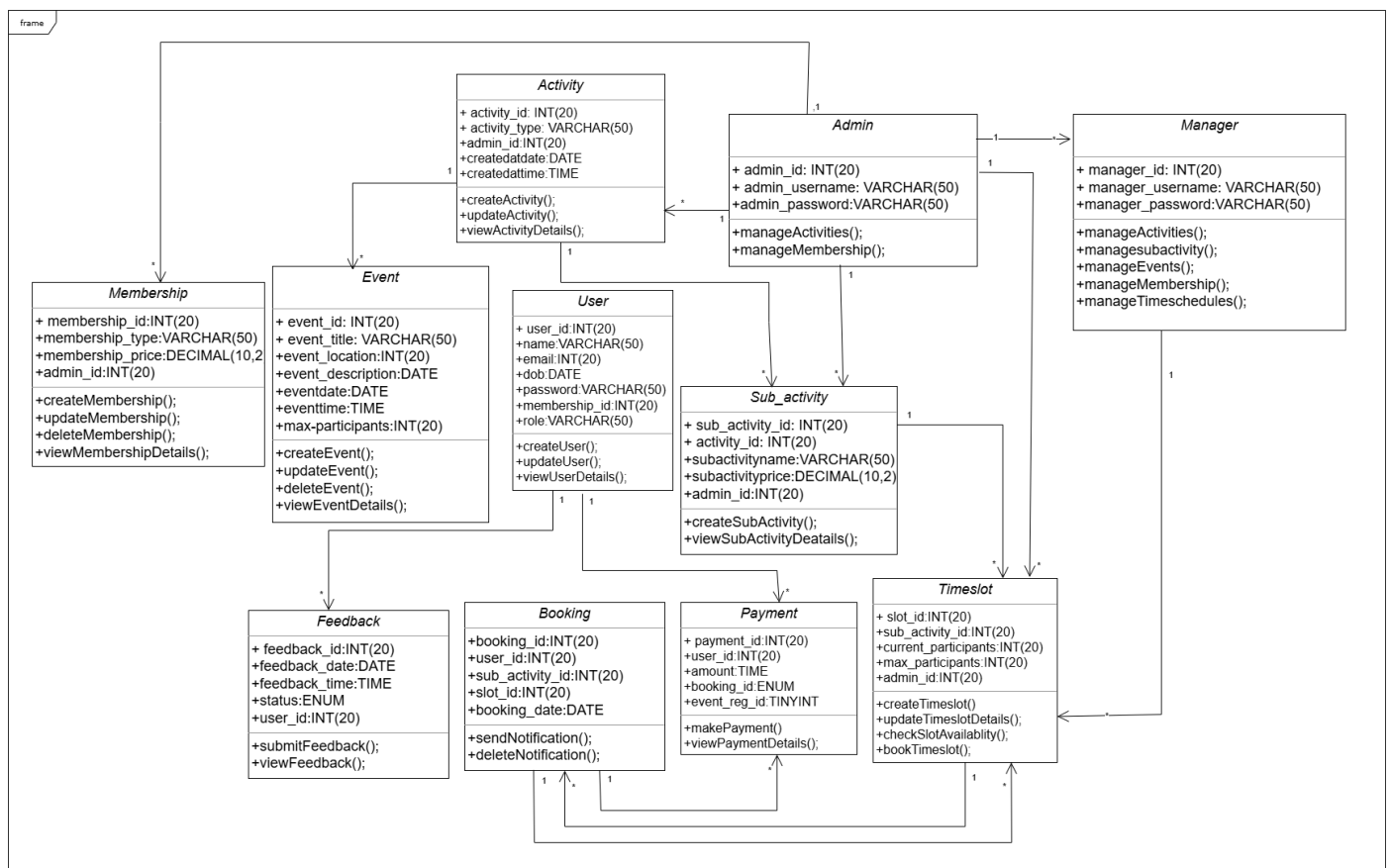


Figure 5: Class Diagram

4.2.2 DIAGRAM OBJECT

Since class diagrams are the source of object diagrams, class diagrams are a prerequisite for object diagrams. An instance of a class diagram is represented by an object diagram. Class and object diagrams both use the same fundamental ideas. The static view of a system is also represented by object diagrams, but this static view represents a momentary snapshot of the system. To represent a group of items and their connections as an instance, object diagrams are employed.

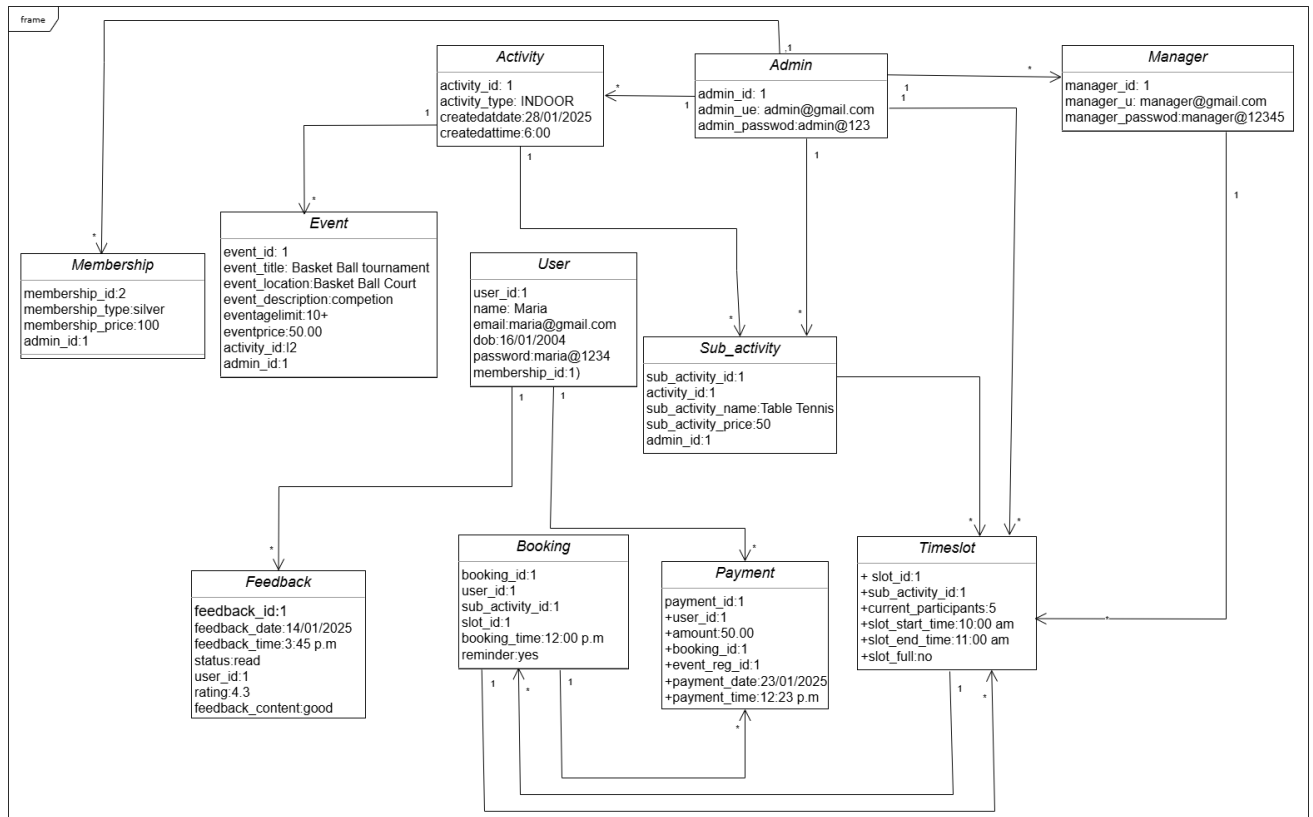


Figure 6: Object Diagram

4.2.3 COMPONENT DIAGRAM

Component diagrams come in a variety of behaviors and personalities. The physical parts of the system are represented using component diagrams. Executables, libraries, files, documents, and other items that are physically present in a node are just a few examples. Component diagrams are used to show how the components of a system are connected and arranged. These diagrams can also be used to construct systems that can be run.

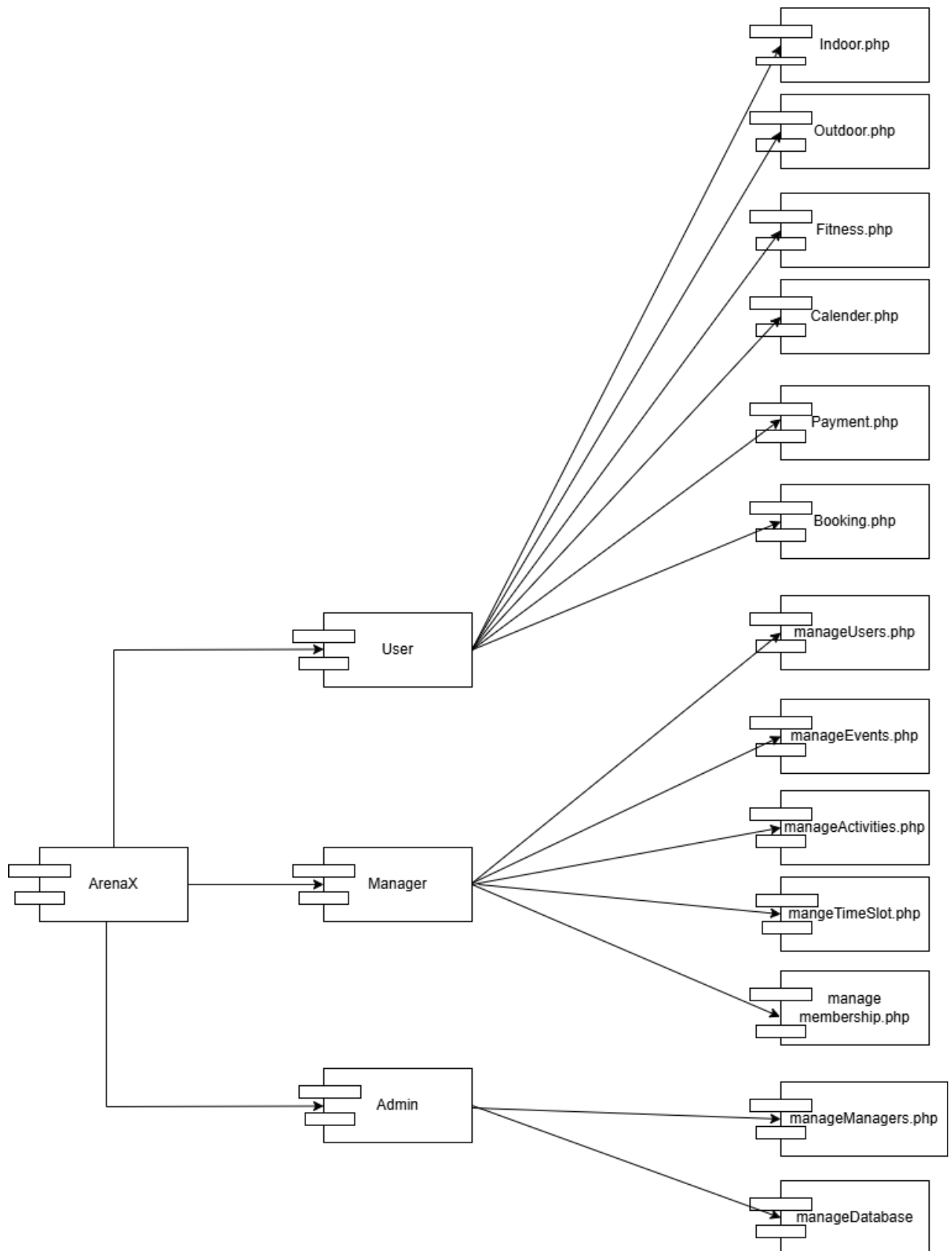


Figure 7: Component Diagram

4.2.8 DEPLOYMENT DIAGRAM

An execution architecture of a system, containing nodes like hardware or software execution environments, and the middleware linking them, is shown in a deployment diagram, a form of UML diagram. Typically, deployment diagrams are used to represent the actual hardware and software of a system. By using it, you can comprehend how the hardware will physically deliver the system. In contrast to other UML diagram types, which primarily depict the logical components of a system, deployment diagrams assist describe the hardware structure of a system.

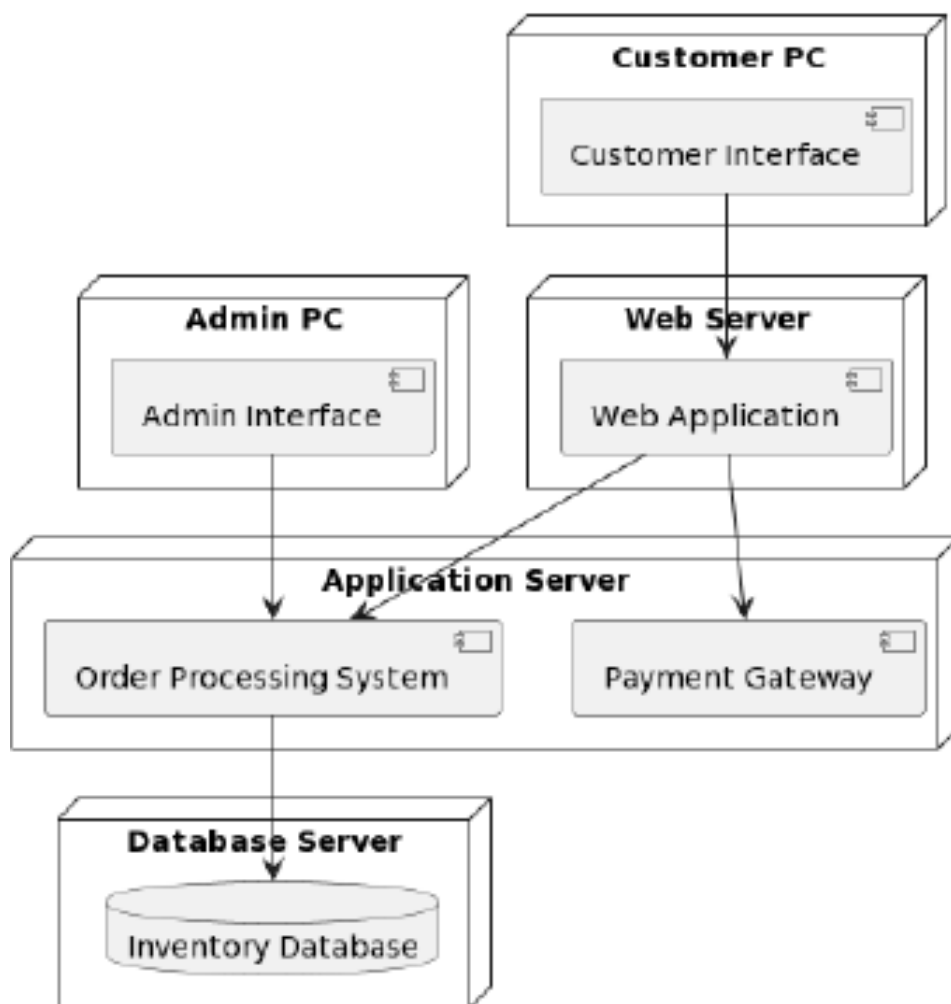
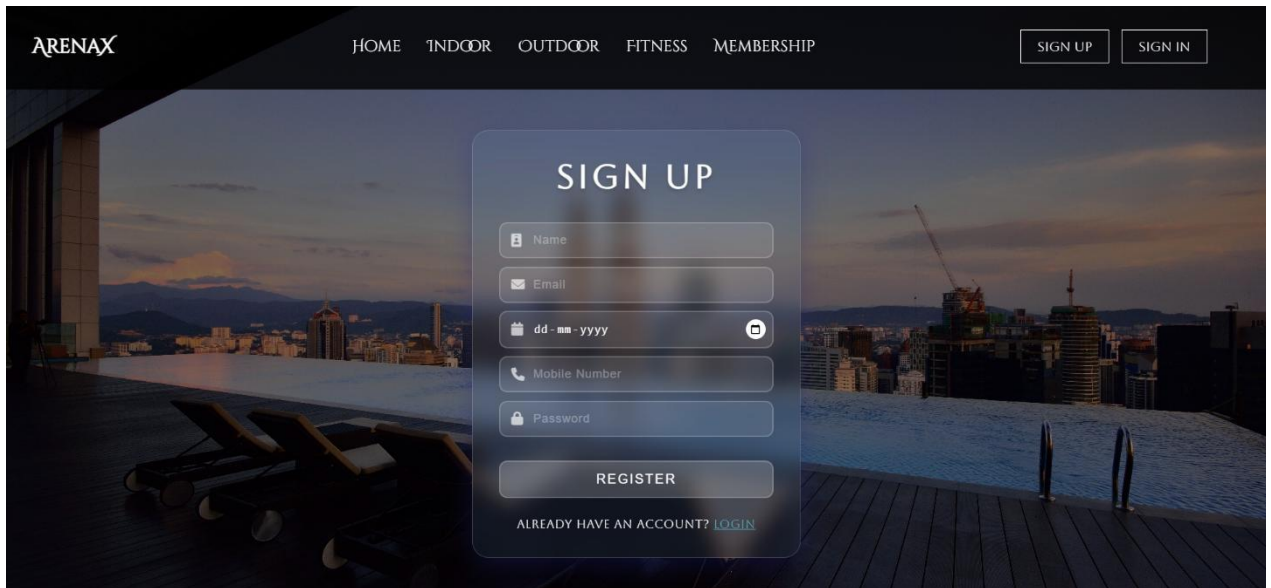


Figure 8: Deployment Diagram

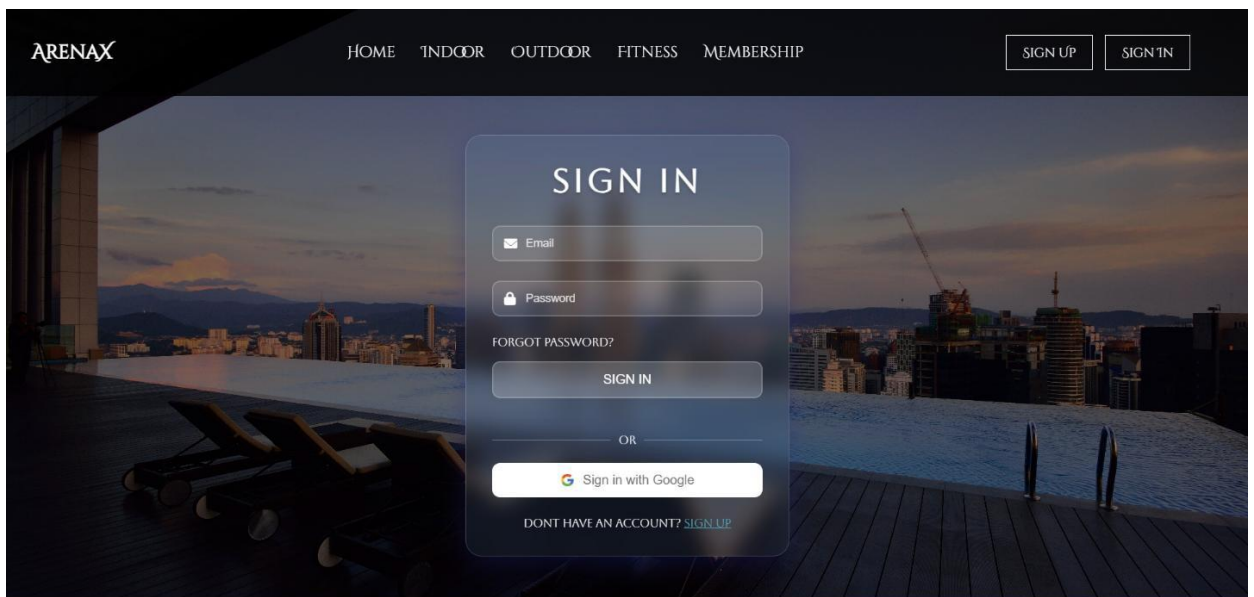
4.3 USER INTERFACE DESIGN USING FIGMA

Form Name: Sign Up Page



The Sign Up page features a dark header with the 'ARENAX' logo on the left and navigation links (HOME, INDOOR, OUTDOOR, FITNESS, MEMBERSHIP) in the center. On the right side of the header are two buttons: 'SIGN UP' and 'SIGN IN'. The main content area has a background image of a rooftop pool at dusk. A central, semi-transparent white card contains the 'SIGN UP' title. Below the title are five input fields: 'Name' (with a person icon), 'Email' (with an envelope icon), a date field labeled 'dd - mm - yyyy' (with a calendar icon), 'Mobile Number' (with a phone icon), and 'Password' (with a lock icon). A 'REGISTER' button is positioned below the fields. At the bottom of the card, there is a link: 'ALREADY HAVE AN ACCOUNT? [LOGIN](#)'.

Form Name: Sign In Page



The Sign In page features a dark header with the 'ARENAX' logo on the left and navigation links (HOME, INDOOR, OUTDOOR, FITNESS, MEMBERSHIP) in the center. On the right side of the header are two buttons: 'SIGN UP' and 'SIGN IN'. The main content area has a background image of a rooftop pool at dusk. A central, semi-transparent white card contains the 'SIGN IN' title. Below the title are two input fields: 'Email' (with an envelope icon) and 'Password' (with a lock icon). Below the fields is a link: 'FORGOT PASSWORD?'. A 'SIGN IN' button is positioned below the link. Below the button is the text 'OR'. Below 'OR' is a 'Sign in with Google' button with the Google logo. At the bottom of the card, there is a link: 'DONT HAVE AN ACCOUNT? [SIGN UP](#)'.

4.4 DATABASE DESIGN

A database is an organized mechanism that has the capability of storing information through which a user can retrieve stored information in an effective and efficient manner. The data is the purpose of any database and must be protected. The database design is a two-level process. In the first step, user requirements are gathered together and a database is designed which will meet these requirements as clearly as possible. This step is called Information Level Design and it is taken independent of any individual DBMS. In the second step, this Information level design is transferred into a design for the specific DBMS that will be used to implement the system in question. This step is called Physical Level Design, concerned with the characteristics of the specific DBMS that will be used. A database design runs parallel with the system design.

4.4.1 Relational Database Management System (RDBMS)

A relational model represents the database as a collection of relations. Each relation resembles a table of values or file of records. In formal relational model terminology, a row is called a tuple, a column header is called an attribute and the table is called a relation. A relational database consists of a collection of tables, each of which is assigned a unique name. A row in a table represents a set of related values.

Relations, Domains & Attributes

A table is a relation. The rows in a table are called tuples. A tuple is an ordered set of n elements. Columns are referred to as attributes. Relationships have been set between every table in the database. This ensures both Referential and Entity Relationship Integrity. A domain D is a set of atomic values. A common method of specifying a domain is to specify a data type from which the data values forming the domain are drawn. It is also useful to specify a name for the domain to help in interpreting its values. Every value in a relation is atomic, that is not decomposable.

Relationships

Table relationships are established using Key. The two main keys of prime importance are Primary Key & Foreign Key. Entity Integrity and Referential Integrity.

Relationships can be established with these keys:

- Entity Integrity enforces that no Primary Key can have null values.
- Referential Integrity enforces that no Primary Key can have null values.

- Referential Integrity for each distinct Foreign Key value, there must exist a matching Primary Key value in the same domain. Other keys are Super Key and Candidate Keys.

4.4.2 NORMALIZATION

Data are grouped together in the simplest way so that later changes can be made with minimum impact on data structures. Normalization is formal process of data structures in manners that eliminates redundancy and promotes integrity. Normalization is a technique of separating redundant fields and breaking up a large table into a smaller one. It is also used to avoid insertion, deletion, and updating anomalies. Normal form in data modelling use two concepts, keys and relationships. A key uniquely identifies a row in a table. There are two types of keys, primary key and foreign key. A primary key is an element or a combination of elements in a table whose purpose is to identify records from the same table. A foreign key is a column in a table that uniquely identifies record from a different table. All the tables have been normalized up to the third normal form. As the name implies, it denotes putting things in the normal form. The application developer via normalization tries to achieve a sensible organization of data into proper tables and columns and where names can be easily correlated to the data by the user. Normalization eliminates repeating groups at data and thereby avoids data redundancy which proves to be a great burden on the computer resources. These include:

- Normalize the data.
- Choose proper names for the tables and columns.
- Choose the proper name for the data.

First Normal Form

The First Normal Form states that the domain of an attribute must include only atomic values and that the value of any attribute in a tuple must be a single value from the domain of that attribute. In other words, 1NF disallows “relations within relations” or “relations as attribute values within tuples”. The only attribute values permitted by 1NF are single atomic or indivisible values. The first step is to put the data into First Normal Form. This can be done by moving data into separate tables where the data is of similar type in each table. Each table is given a Primary Key or Foreign Key as per requirement of the project. In this we form new relations for each non-atomic attribute or nested relation. This eliminated repeating groups of data. A relation is said to be in first normal form if only if it satisfies the constraints that contain the primary key only.

Example (Before 1NF – Unnormalized Table):

Student_ID	Student_Name	Courses	Contact
101	Alice	Math, Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science, History	7654321098

Conversion to 1NF:

Student_ID	Student_Name	Course	Contact
101	Alice	Math	9876543210
101	Alice	Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science	7654321098
103	Charlie	History	7654321098

Second Normal Form

According to Second Normal Form, for relations where primary key contains multiple attributes, no non-key attribute should be functionally dependent on a part of the primary key. In this we decompose and setup a new relation for each partial key with its dependent attributes. Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it. This step helps in taking out data that is only dependent on a part of the key. A relation is said to be in second normal form if and only if it satisfies all the first normal form conditions for the primary key and every non-primary key attributes of the relation is fully dependent on its primary key alone.

Example (Before 2NF - 1NF Table):

Student_ID	Student_Name	Course	Contact
101	Alice	Math	9876543210
101	Alice	Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science	7654321098
103	Charlie	History	7654321098

Conversion to 2NF:

Students Table

Student_ID	Student_Name	Contact
101	Alice	9876543210
102	Bob	8765432109
103	Charlie	7654321098

Enrollment Table

Student_ID	Course
101	Math
101	Science
102	English
103	Science
103	History

Third Normal Form

According to Third Normal Form, Relation should not have a non-key attribute functionally determined by another non-key attribute or by a set of non-key attributes. That is, there should be no transitive dependency on the primary key. In this we decompose and set up relation that includes the non-key attributes that functionally determines other non-key attributes. This step is taken to get rid of anything that doesn't depend entirely on the Primary Key. A relation is said to be in third normal form if only if it is in second normal form and more over the non key attributes of the relation should not be depend on another non-key attribute.

Example (Before 3NF - 2NF Tables): Students Table (Before 3NF)

Student_ID	Student_Name	Contact	Address
101	Alice	9876543210	New York
102	Bob	8765432109	Los Angeles
103	Charlie	7654321098	Chicago

Enrollment Table

Student_ID	Course
101	Math
101	Science
102	English
103	Science
103	History

Conversion to 3NF:

Students Table

Student_ID	Student_Name
101	Alice
102	Bob
103	Charlie

Contact Table

Contact_ID	Student_ID	Phone	Address
1	101	9876543210	New York
2	102	8765432109	Los Angeles
3	103	7654321098	Chicago

Enrollment Table

Student_ID	Course
101	Math
101	Science
102	English
103	Science
103	History

4.4.3 SANITIZATION

Data validation is an integral part of web development, especially while working with forms where the user first enters their personal data and then sends that to the database. Data sent in an invalid format can cause DBMS security problems. Hackers often use SQL injections to insert malicious SQL commands into the database. SQL injections can even destroy the database once inserted. Therefore, to safeguard the database from hackers, it is necessary to sanitize and filter the user entered data before sending it to the database. Data sanitization involves purposely, permanently deleting, or destroying data from a storage device, to ensure it cannot be recovered. Ordinarily, when data is deleted from storage media, the media is not really erased and can be recovered by an attacker who gains access to the device.

4.4.4 INDEXING

Indexing is used to quickly retrieve particular data from the database. It is a technique that uses data structures to optimize the searching time of a database query. Indexing reduces the number of disks required to access a particular data by internally creating an index table. Search Key contains the copy of the Primary Key or the Candidate Key of the database table. Generally, we store the selected Primary or Candidate keys in a sorted manner so that we can reduce the overall query time or search time. An index takes a search key as input, efficiently returns a collection of matching records. Indexing in Database is defined based on its indexing attributes.

Different types of indexing methods are:

- **Primary Indexing:** Primary Index is an ordered file which is fixed length size with two fields. The first field is the same as a primary key and second, field is pointed to that specific data block. In the primary Index, there is always one to one relationship between the entries in the index table.
- **Secondary Indexing:** The Secondary Index in DBMS can be generated by a field which has a unique value for each record, and it should be a candidate key. It is also known as a non-clustering index. This two-level database indexing technique is used to reduce the mapping size of the first level. For the first level, a large range of numbers is selected because of this; the mapping size always remains small.
- **Multilevel Index:** Multilevel Indexing in Database is created when a primary index does not fit in memory. In this type of indexing method, you can reduce the number of disk accesses to short any record and kept on a disk as a sequential file and create a sparse base on that file.

4.5 TABLE DESIGN

1.tbl_users

Primary key: **user_id**

Foreign key: **membership_id** references table **tbl_memberships**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	user_id	Int	Primary Key	Primary Key
2	name	Varchar(100)	Not Null	Name of the user
3	email	Varchar(100)	Unique	Email of the user
4	dob	Date	Not Null	Date of Birth
5	mobile	Varchar(15)	Not Null	Mobile Number
6	password	Varchar(255)	Not Null	Password of account
7	membership_id	Int	Foreign Key	Foreign Key
8	role	Int	Not Null	Checks if user, manager or admin
9	reset_token	Varchar(100)	Not Null	Reset passwords token
10	token_expiry	Datetime	Not Null	Expiration time of the token
11	profile_image	Varchar(255)	Not Null	Image of the user

2.tbl_activity

Primary key: **activity_id**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	activity_id	Int	Primary Key	Primary Key
2	activity_type	Varchar(50)	Unique	Type of the activity

3. tbl_sub_activity_namePrimary key: **sub_act_id**Foreign key: **activity_id** references table **tbl_activity**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	sub_act_id	Int	Primary Key	Primary Key
2	sub_act_name	Varchar(100)	Not Null	Name of the user
3	activity_id	Int	Foreign Key	Foreign Key
4	membership_type	Enum(normal,standard,premium)	Not Null	Type of membership

4. tbl_sub_activityPrimary key: **sub_activity_id**

Foreign key: **sub_act_id** references table **tbl_sub_activity_name**,
activity_id references table **tbl_activity**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	sub_activity_id	Int	Primary Key	Primary Key
2	activity_id	Int	Foreign Key	Foreign Key
3	sub_activity_price	Decimal(10,2)	Not Null	Email of the user
4	sub_activity_image	Varchar(255)	Not Null	Image of the Sub Activity
5	sub_act_id	Int	Foreign Key	Foreign Key

5. tbl_timeslots

Primary key: **sub_activity_id**

Foreign key: **sub_activity_id** references table **tbl_sub_activity**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	slot_id	Int	Primary Key	Primary Key
2	sub_activity_id	Int	Foreign Key	Foreign Key
3	slot_date	Date	Not Null	Date of the timeslot
4	slot_start_time	Time	Not Null	Starting time of the slot
5	slot_end_time	Time	Not Null	Ending time of that slot
6	max_participants	Int	Not Null	Maximum participants allowed
7	current_participants	Int	Default 0	Current number of participants
8	slot_full	Boolean	Default False	Checking if the slot is fill or not

6. tbl_event_registration

Primary key: **event_reg_id**

Foreign key: **event_id** references table **tbl_events**,

user_id references table **tbl_users**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	event_reg_id	Int	Primary Key	Primary Key
2	event_id	Int	Foreign Key	Foreign Key
3	user_id	Int	Foreign Key	Foreign Key
4	bill	Varchar(255)	Not Null	Bill path

7. tbl_booking

Primary key: **booking_id**

Foreign key: **user_id** references table **tbl_users**,

sub_activity_id references table **tbl_sub_activity**,

slot_id references table **tbl_timeslots**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	booking_id	Int	Primary Key	Primary Key
2	user_id	Int	Foreign Key	Foreign Key
3	sub_activity_id	Int	Foreign Key	Foreign Key
4	slot_id	Int	Foreign Key	Foreign Key
5	booking_date	Date	Not Null	Date of booking
6	booking_time	Time	Not Null	Time of the booking
7	bill	Varchar(255)	Not Null	Bill path
8	reminder	Boolean	Default False	Reminder

8. tbl_memberships

Primary key: **membership_id**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	membership_id	Int	Primary Key	Primary Key
2	membership_type	Varchar(50)	Not Null	Membership type
3	membership_price	Decimal(10,2)	Not Null	Membership price

9. tbl_events

Primary key: **event_id**

Foreign key: **activity_id** references table **tbl_activity**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	event_id	Int	Primary Key	Primary Key
2	activity_id	Int	Foreign Key	Foreign Key
3	event_title	Varchar(100)	Not Null	Title of the event
4	event_description	Text	Not Null	Description of the event
5	event_date	Date	Not Null	Date of event
6	event_time	Time	Not Null	Time of the event
7	event_location	Varchar(255)	Not Null	Location of the event
8	max_participants	Int	Not Null	Maximum participants allowed
9	event_age_limit	Int	Not Null	Age limit of the event
10	event_price	Decimal(10,2)	Not Null	Price of the event
11	event_image	Varchar(255)	Not Null	Image of the event
12	created_at_date	Date	Not Null	Date when the event was created
13	created_at_time	Time	Not Null	Time when the event was created

10. tbl_feedback

Primary key: **feedback_id**

Foreign key: **user_id** references table **tbl_users**,

activity_id references table **tbl_activity**,

sub_act_id references table **tbl_sub_activity_name**,

event_id references table **tbl_events**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	feedback_id	Int	Primary Key	Primary Key
2	user_id	Int	Foreign Key	Foreign Key
3	activity_id	Int	Foreign Key	Foreign Key
4	activity_type	Varchar(50)	Not Null	Type of activity
5	sub_act_id	Int	Foreign Key	Foreign Key
6	activity_name	Varchar(100)	Not Null	Activity name
7	feedback_content	Text	Not Null	Feedback content
8	rating	Tinyint	Not Null	Rating from users
9	feedback_date	Date	Not Null	Date of feedback
10	feedback_time	Time	Not Null	Time of feedback
11	status	Enum(Pending,Reviewed)	Not Null	Status of feedback
12	event_id	Int	Foreign Key	Foreign Key

11. tbl_recurring_bookingsPrimary key: **recurring_id**Foreign key: **user_id** references table **tbl_users**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	recurring_id	Int	Primary Key	Primary Key
2	user_id	Int	Foreign Key	Foreign Key
3	sub_activity_id	Int	Foreign Key	Foreign Key
4	start_date	Date	Not Null	Starting Date
5	end_date	Date	Not Null	Ending Date
6	booking_time	Time	Not Null	Time of booking
7	selected_days	Time	Not Null	Selected Days
8	status	Enum(active,completed,cancelled)	Not Null	Date of viewing
9	bill	Time	Not Null	Time of viewing
10	created_at	Timestamp	Not Null	Time of recurring booking

12. tbl_notificationPrimary key: **notification_id**Foreign key: **user_id** references table **tbl_users**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	notification_id	Int	Primary Key	Primary Key
2	user_id	Int	Foreign Key	Foreign Key
3	title	Decimal(10,2)	Not Null	Notification title
4	message	Text	Not Null	Notification content
5	notification_viewed	Boolean	Default False	If viewed or not
6	created_at_date	Date	Not Null	Date of creation
7	created_at_time	Time	Not Null	Time of creation
8	viewed_at_date	Date	Not Null	Date of viewing
9	viewed_at_time	Time	Not Null	Time of viewing

13. tbl_recurring_slotsPrimary key: **slot_id**Foreign key: **recurring_id** references table **tbl_recurring_bookings**,**timeslot_id** reference table **tbl_timeslots**,**booking_id** reference table **tbl_booking**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	slot_id	Int	Primary Key	Primary Key
2	recurring_id	Int	Foreign Key	Foreign Key
3	booking_date	Date	Not Null	Date of booking
4	timeslot_id	Int	Foreign Key	Foreign Key
5	status	Enum(pending,booked,cancelled)	Not Null	Status
6	booking_id	Int	Foreign Key	Foreign Key

14. tbl_membership_reg

Primary key: **membership_reg_id**

Foreign key: **user_id** references table **tbl_users**,

membership_id references table **tbl_memberships**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	membership_reg_id	Int	Primary Key	Primary Key
2	user_id	Int	Foreign Key	Foreign Key
3	membership_id	Int	Foreign Key	Foreign Key
4	membership_reg_date	Date	Not Null	Date of registration
5	membership_reg_time	Time	Not Null	Time of registration
6	expiration_date	Date	Not Null	Date of expiration
7	bill	Varchar(255)	Not Null	Bill path

15. tbl_user_calendar_events

Primary key: **id**

Foreign key: **sub_activity_id** references table **tbl_sub_activity**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	id	Int	Primary Key	Primary Key
2	user_id	Int	Foreign Key	Foreign Key
3	title	Varchar(255)	Not Null	Title in calendar
4	date	Date	Not Null	Date in calendar
5	time	Time	Not Null	Time in calendar
6	description	Text	Not Null	Description in calendar
7	event_type	Enum(custom,booking,event)	Default 0	Type of event
8	reminder_enabled	Tinyint	Default False	Enabling reminder

16. tbl_payment

Primary key: **payment_id**

Foreign key: **user_id** references table **tbl_users**,

booking_id references table **tbl_booking**,

event_reg_id references table **tbl_event_registration**

No:	Field Name	Datatype (Size)	Key Constraints	Description of the field
1	payment_id	Int	Primary Key	Primary Key
2	user_id	Int	Foreign Key	Foreign Key
3	amount	Decimal(10,2)	Not Null	Price
4	payment_date	Date	Not Null	Date of payment
5	payment_time	Time	Not Null	Time of payment
6	booking_id	Int	Foreign Key	Foreign Key
7	event_reg_id	Int	Foreign Key	Foreign Key
8	membership_reg_id	Int	Foreign Key	Foreign Key

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

Software testing is the procedure of meticulously monitoring the way software is used to see if it works as expected. Software testing is usually used in conjunction with the words validation and verification. A product, including software, is validated by being examined or evaluated to see if it conforms to all pertinent requirements. Software testing, one sort of verification, also makes use of reviews, analyses, inspections, and walkthroughs. Validation is the process of ensuring that what has been specified corresponds to what the user actually wants. Other procedures that are typically connected to software testing include static analysis and dynamic analysis. Without actually running the code, static analysis examines the software's source code to look for errors and gather statistics. Dynamic analysis looks at the behaviour of software while it is in use to provide information like execution traces, timing profiles, and test coverage specifics. Running a programme with the intention of finding any faults is how a programme is tested. Testing is a group of tasks that can be organized in advance and completed in a systematic way. Individual modules are tested first, followed by the integration of the entire computer-based system. There are numerous regulations that can be utilized as testing objectives, and testing is essential for the achievement of system testing objectives. The following:

- A test case with a high likelihood of detecting an unknown fault qualifies as a good test case.
- A test that finds an error that has not yet been found is successful.

A test that effectively accomplishes the aforementioned goals will reveal software problems. Testing also reveals that the software functions appear to function in accordance with the specification and that the performance requirements appear to have been met. There are three ways to test programs.

- For accuracy
- For effective implementation
- Regarding computational complexity

Testing for correctness is meant to ensure that a program performs exactly as it was intended to. This is much harder than it might initially seem, especially for big programs.

5.2 TEST PLAN

A test plan suggests a number of required steps that need be taken in order to complete various testing methodologies. The activity that is to be taken is outlined in the test plan. A computer programme, its documentation, and associated data structures are all created by software developers. It is always the responsibility of the software developers to test each of the program's separate components to make sure it fulfils the purpose for which it was intended. In order to solve the inherent issues with allowing the builder evaluate what they have developed, there is an independent test group (ITG). Testing's precise goals should be laid forth in quantifiable language. So that the mean time to failure, the cost to find and fix the defects, remaining defect density or frequency of occurrence and test work-hours per regression test all should be stated within the test plan.

The levels of testing include:

- Unit testing
- Integration Testing
- Data validation Testing
- Output Testing

5.2.1 UNIT TESTING

Unit testing concentrates verification efforts on the software component or module, which is the smallest unit of software design. The component level design description is used as a guide when testing crucial control paths to find faults inside the module's perimeter. the level of test complexity and the untested area determined for unit testing. Unit testing is white-box focused, and numerous components may be tested simultaneously. To guarantee that data enters and exits the software unit under test properly, the modular interface is tested. To make sure that data temporarily stored retains its integrity during each step of an algorithm's execution, the local data structure is inspected. Boundary conditions are tested to ensure that all statements in a module have been executed at least once. Finally, all error handling paths are tested. Before starting any other test, tests of data flow over a module interface are necessary. All other tests are irrelevant if data cannot enter and depart the system properly. An important duty during the unit test is the selective examination of execution pathways. Error circumstances must be foreseen in good design, and error handling paths must be put up to cleanly reroute or halt work when an error does arise. The final step of unit testing is boundary testing. Software frequently fails at its limits.

5.2.2 INTEGRATION TESTING

Integration testing is a methodical approach for creating the program's structure while also carrying out tests to find interface issues. The goal is to construct a programme structure that has been determined by design using unit tested components. The programme as a whole is tested. Correction is challenging since the size of the overall programme makes it challenging to isolate the causes. As soon as these mistakes are fixed, new ones arise, and the process repeats itself in an apparently unending cycle. All of the modules were integrated after unit testing was completed in the system to check for any interface inconsistencies. A distinctive programme structure also developed when discrepancies in programme structures were eliminated.

5.2.3 VALIDATION TESTING OR SYSTEM TESTING

The testing process comes to an end here. This involved testing the entire system in its entirety, including all forms, code, modules, and class modules. Popular names for this type of testing include system tests and black box testing. The functional requirements of the software are the main emphasis of the black box testing approach. That example, using Black Box testing, a software engineer can create sets of input conditions that will fully test every programme requirement. The following sorts of problems are targeted by black box testing: erroneous or missing functions, interface errors, data structure or external data access errors, performance errors, initialization errors, and termination errors

5.2.4 USER ACCEPTANCE TESTING

The system considered is tested for user acceptance; here it should satisfy the firm's need. The software should keep in touch with perspective system; user at the time of developing and making changes whenever required. This done with respect to the following points:

- Input Screen Designs,
- Output Screen Designs

The above testing is done taking various kinds of test data. Preparation of test data plays a vital role in the system testing. After preparing the test data, the system under study is tested using that test data. While testing the system by which test data errors are again uncovered and corrected by using above testing steps and corrections are also noted.

5.2.5 AUTOMATION TESTING

Software and other computer goods are tested automatically to make sure they abide by tight guidelines. In essence, it's a test to ensure that the hardware or software performs exactly as intended. It checks for errors, flaws, and any other problems that might occur throughout the creation of the product. Any time of day can be used to do automation testing. It looks at the software using scripted sequences. It then summarizes what was discovered, and this data can be compared to results from earlier test runs.

5.2.6 SELENIUM TESTING

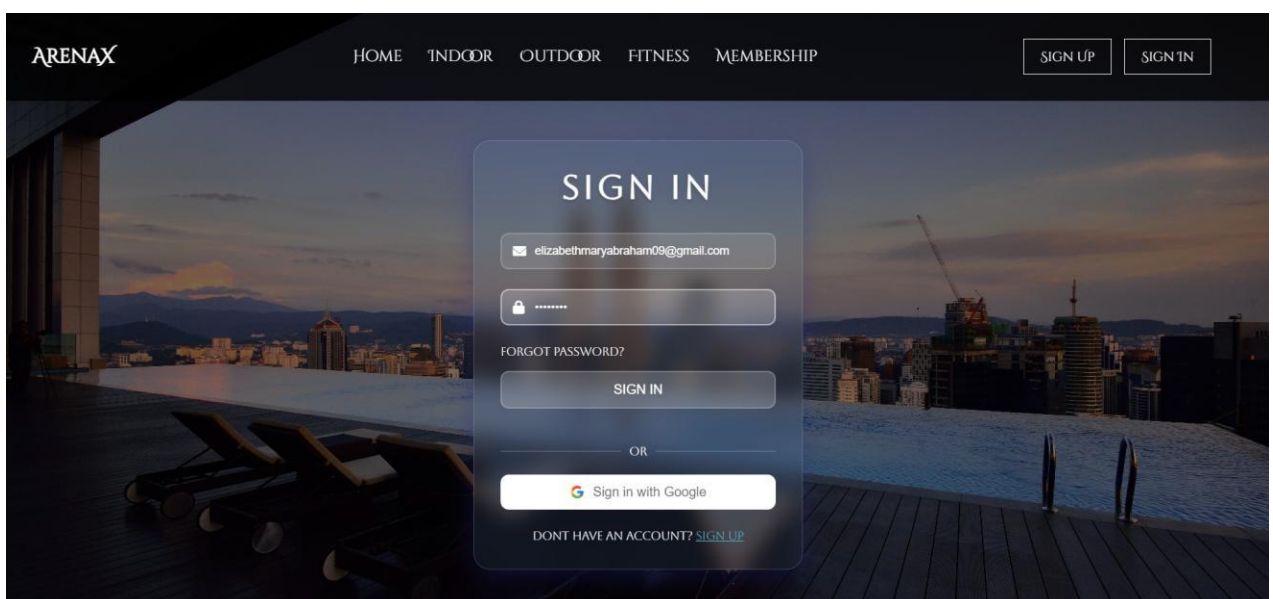
An open-source programme called Selenium automates web browsers. It offers a single interface that enables you to create test scripts in a number of different programming languages, including Ruby, Java, NodeJS, PHP, Perl, Python, and C #. Web application testing for cross browser compatibility is automated using the Selenium testing tool. Whether they are responsive, progressive, or standard, it is utilized to assure high-quality web apps. Selenium is a free software programme.

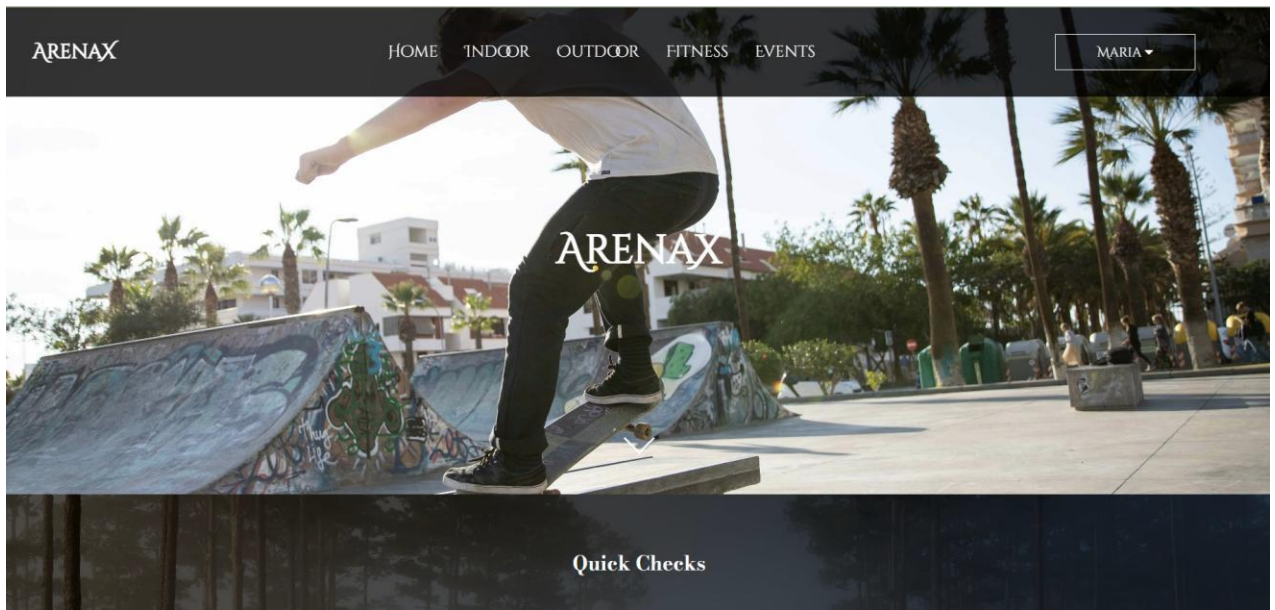
Test Case 1: Login Test

Code:

```
signup.py 8  signin.py 8 X
C: > Users > eliza > Desktop > Test > signin.py > ...
1  # Generated by Selenium IDE
2  import pytest
3  import time
4  import json
5  from selenium import webdriver
6  from selenium.webdriver.common.by import By
7  from selenium.webdriver.common.action_chains import ActionChains
8  from selenium.webdriver.support import expected_conditions
9  from selenium.webdriver.support.wait import WebDriverWait
10 from selenium.webdriver.common.keys import Keys
11 from selenium.webdriver.common.desired_capabilities import DesiredCapabilities
12
13 class TestSignin2():
14     def setup_method(self, method):
15         self.driver = webdriver.Chrome()
16         self.vars = {}
17
18     def teardown_method(self, method):
19         self.driver.quit()
20
21     def test_signin2(self):
22         self.driver.get("http://localhost/MiniProj/signin.php")
23         self.driver.set_window_size(1552, 832)
24         self.driver.find_element(By.NAME, "email").click()
25         self.driver.find_element(By.NAME, "email").send_keys("elizabethmaryabraham09@gmail.com")
26         self.driver.find_element(By.NAME, "password").click()
27         self.driver.find_element(By.NAME, "password").send_keys("12345678")
28         self.driver.find_element(By.CSS_SELECTOR, ".login-btn").click()
29
30
```

Screenshots:





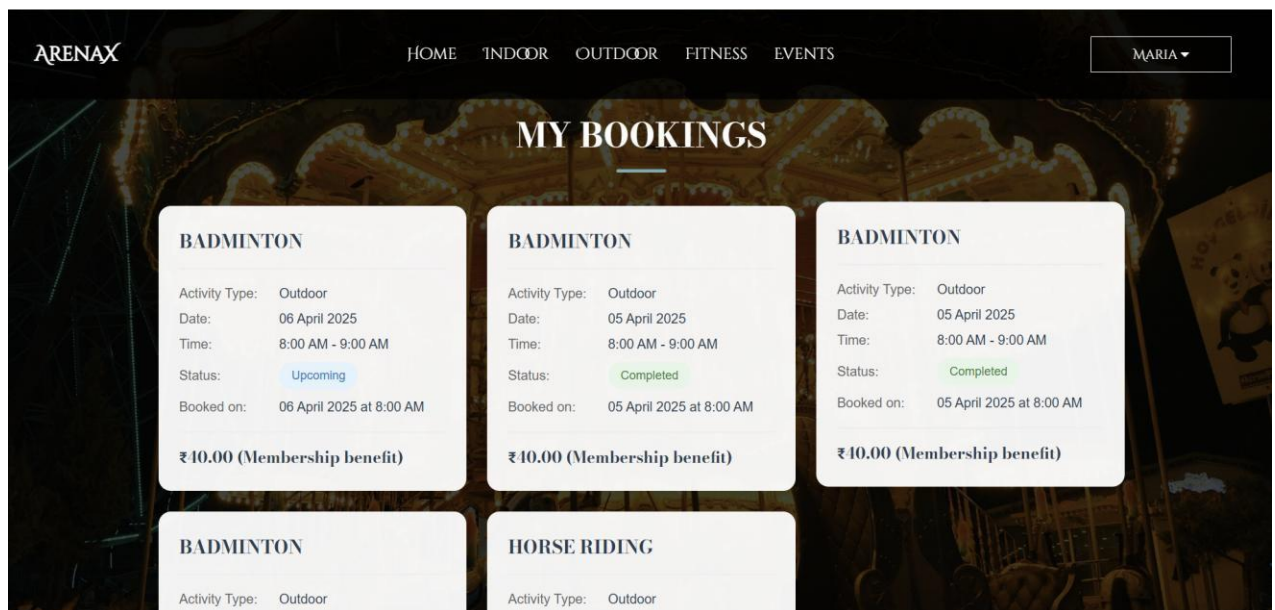
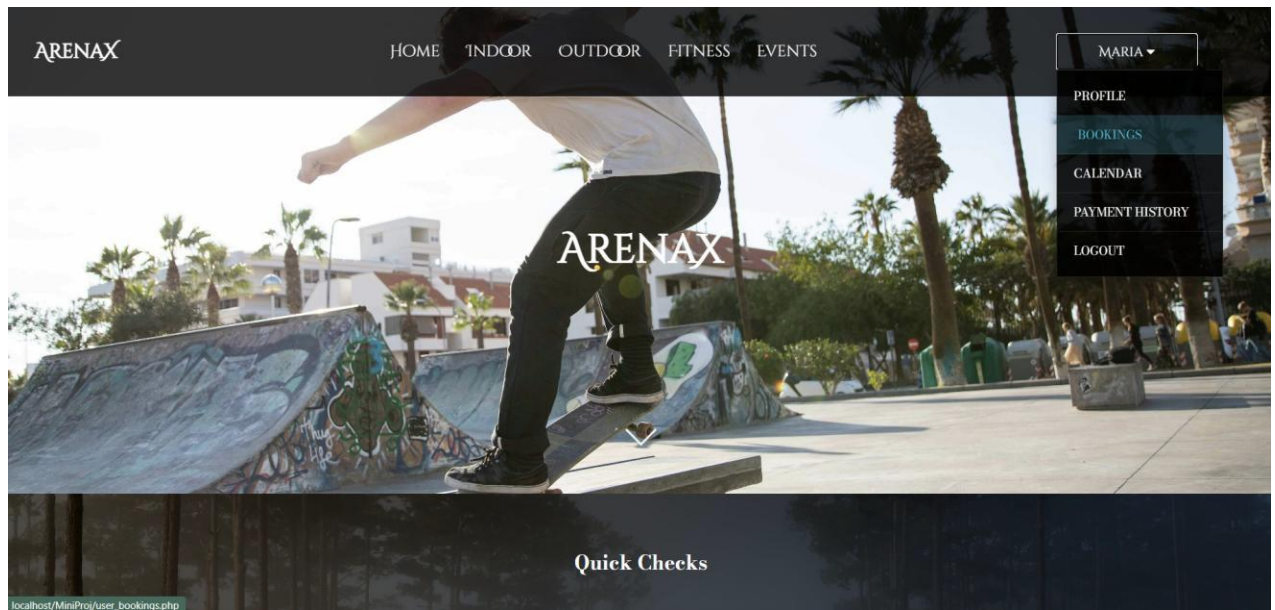
Test Report:

Test Case 1					
Project Name: ArenaX					
Login Test Case					
Test Case ID: Test_1			Test Designed By: Elizabeth Mary		
Test Priority (Low/Medium/High): High			Test Designed Date: 05/04/2025		
Module Name: Sign In Page			Test Executed By: Dr. Paulin Paul		
Test Title: Verify Login with valid email and password			Test Execution Date: 05/04/2025		
Description: Login Page Test					
Pre-Condition: User has valid username and password					
Step	Test Step	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1	Navigate to the login page	-	Display login page	Login page displayed	Pass
2	Valid Email entered	Email: eliza..gmail.com	User should enter the home page	User entered the home page	Pass
3	Valid Password entered	Password:12345678			
4	Click on Sign In Button	-			
Post-Condition: Username and password are checked with database values for successful login					

Test Case 2: Booked Schedules Functionality

Code:

```
test_schedules.py X
C: > Users > eliza > Desktop > Test > test_schedules.py
1  # Generated by Selenium IDE
2  import pytest
3  import time
4  import json
5  from selenium import webdriver
6  from selenium.webdriver.common.by import By
7  from selenium.webdriver.common.action_chains import ActionChains
8  from selenium.webdriver.support import expected_conditions
9  from selenium.webdriver.support.wait import WebDriverWait
10 from selenium.webdriver.common.keys import Keys
11 from selenium.webdriver.common.desired_capabilities import DesiredCapabilities
12
13 class TestSchedules():
14     def setup_method(self, method):
15         self.driver = webdriver.Chrome()
16         self.vars = {}
17
18     def teardown_method(self, method):
19         self.driver.quit()
20
21     def test_schedules(self):
22         self.driver.get("http://localhost/MiniProj/signin.php")
23         self.driver.set_window_size(1552, 832)
24         self.driver.find_element(By.NAME, "email").click()
25         self.driver.find_element(By.NAME, "email").send_keys("elizabethmaryabraham09@gmail.com")
26         self.driver.find_element(By.NAME, "password").click()
27         self.driver.find_element(By.NAME, "password").send_keys("12345678")
28         self.driver.find_element(By.CSS_SELECTOR, ".login-btn").click()
29         self.driver.find_element(By.LINK_TEXT, "Book Now").click()
30
31
```

Screenshots:

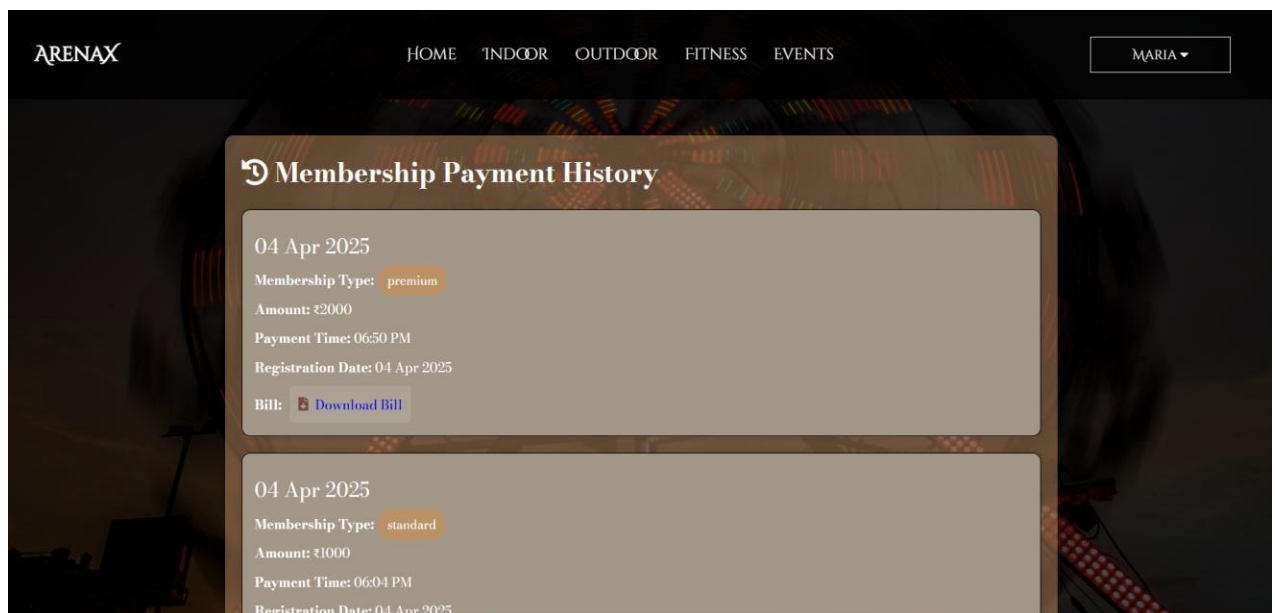
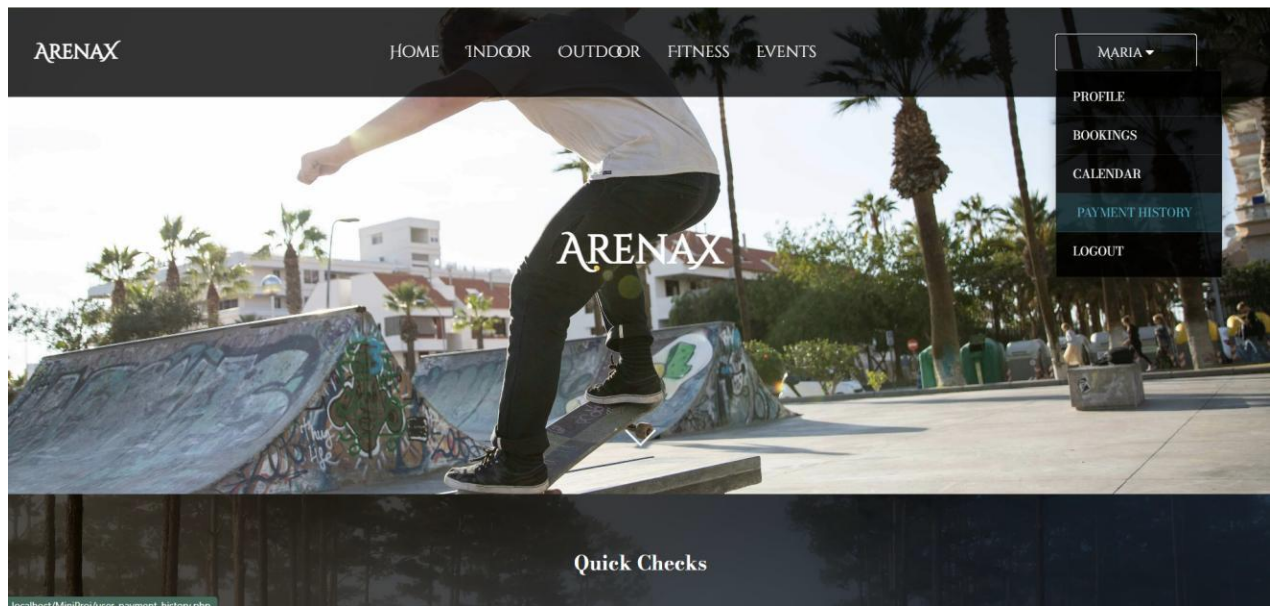
Test Report:

Test Case 2					
Project Name: ArenaX					
Booked Schedules Test Case					
Test Case ID: Test_2			Test Designed By: Elizabeth Mary		
Test Priority (Low/Medium/High): High			Test Designed Date: 05/04/2025		
Module Name: Booked Schedules			Test Executed By: Dr. Paulin Paul		
Test Title: Booked Schedules			Test Execution Date: 05/04/2025		
Description: Booked Schedules Test					
Pre-Condition: User should have made a booking					
Step	Test Step	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1	Navigate to the user dashboard	elizabethmar yabraham09@gmail.com	Display user dashboard	User dashboard displayed	Pass
2	Navigate to users bookings page	Table Tennis	User should enter the scheduled bookings page	User entered the scheduled bookings page and viewed all the upcoming bookings and the completed bookings	Pass
3	Display of the booked schedules	Booked Schedules			
Post-Condition: Upcoming bookings and the completed bookings are viewed by the user					

Test Case 3: Payment History

Code:

```
test_schedules.py 8 test_payhist.py 8 X
C: > Users > eliza > Desktop > Test > test_payhist.py > ...
1 # Generated by Selenium IDE
2 import pytest
3 import time
4 import json
5 from selenium import webdriver
6 from selenium.webdriver.common.by import By
7 from selenium.webdriver.common.action_chains import ActionChains
8 from selenium.webdriver.support import expected_conditions
9 from selenium.webdriver.support.wait import WebDriverWait
10 from selenium.webdriver.common.keys import Keys
11 from selenium.webdriver.common.desired_capabilities import DesiredCapabilities
12
13 class TestPayhist():
14     def setup_method(self, method):
15         self.driver = webdriver.Chrome()
16         self.vars = {}
17
18     def teardown_method(self, method):
19         self.driver.quit()
20
21     def wait_for_window(self, timeout = 2):
22         time.sleep(round(timeout / 1000))
23         wh_now = self.driver.window_handles
24         wh_then = self.vars["window_handles"]
25         if len(wh_now) > len(wh_then):
26             return set(wh_now).difference(set(wh_then)).pop()
27
28     def test_payhist(self):
29         self.driver.get("http://localhost/MiniProj/signin.php")
30         self.driver.set_window_size(1552, 832)
31         self.driver.find_element(By.NAME, "email").click()
32         self.driver.find_element(By.NAME, "email").send_keys("elizabethmaryabraham09@gmail.com")
33         self.driver.find_element(By.NAME, "password").click()
34         self.driver.find_element(By.NAME, "password").send_keys("12345678")
35         self.driver.find_element(By.CSS_SELECTOR, ".login-btn").click()
36         self.driver.find_element(By.LINK_TEXT, "Explore Activities").click()
37         self.vars["window_handles"] = self.driver.window_handles
38         self.driver.find_element(By.CSS_SELECTOR, ".payment-card:nth-child(2) .bill-box > span").click()
39         self.vars["win4380"] = self.wait_for_window(2000)
40         self.driver.switch_to.window(self.vars["win4380"])
41
```

Screenshots:

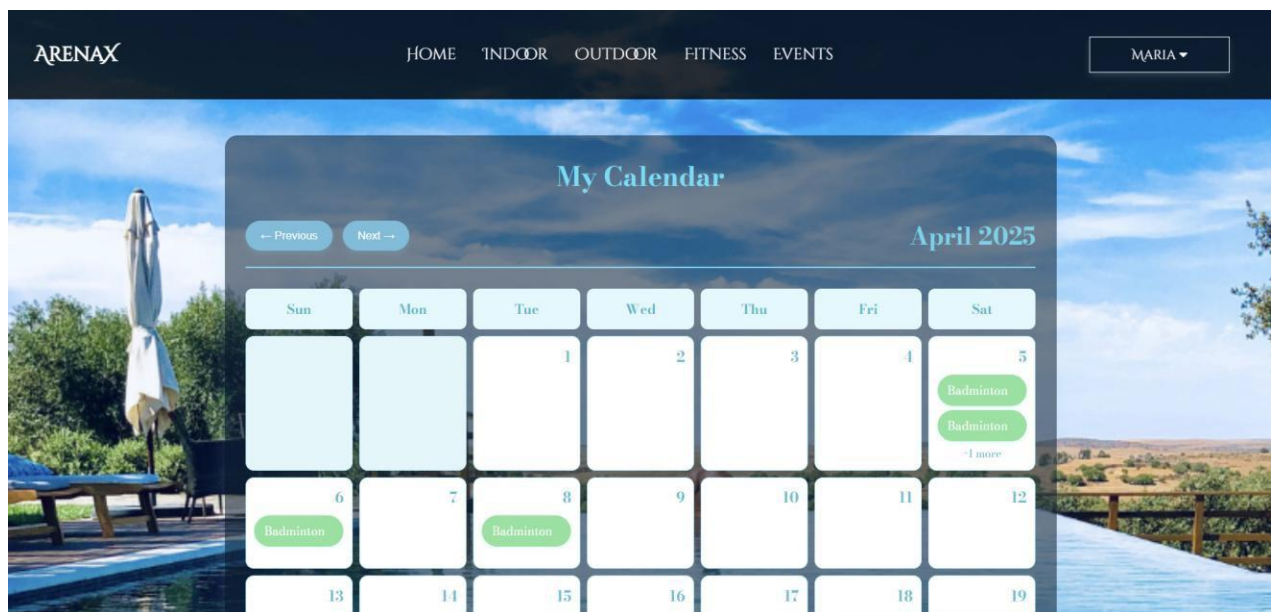
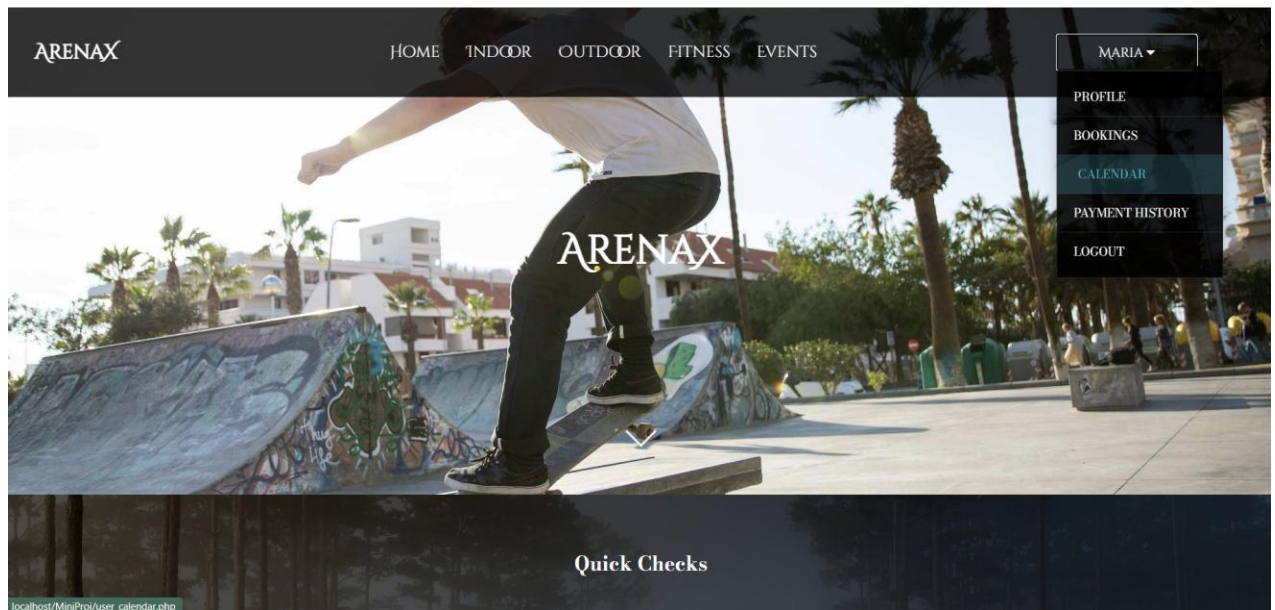
Test Report:

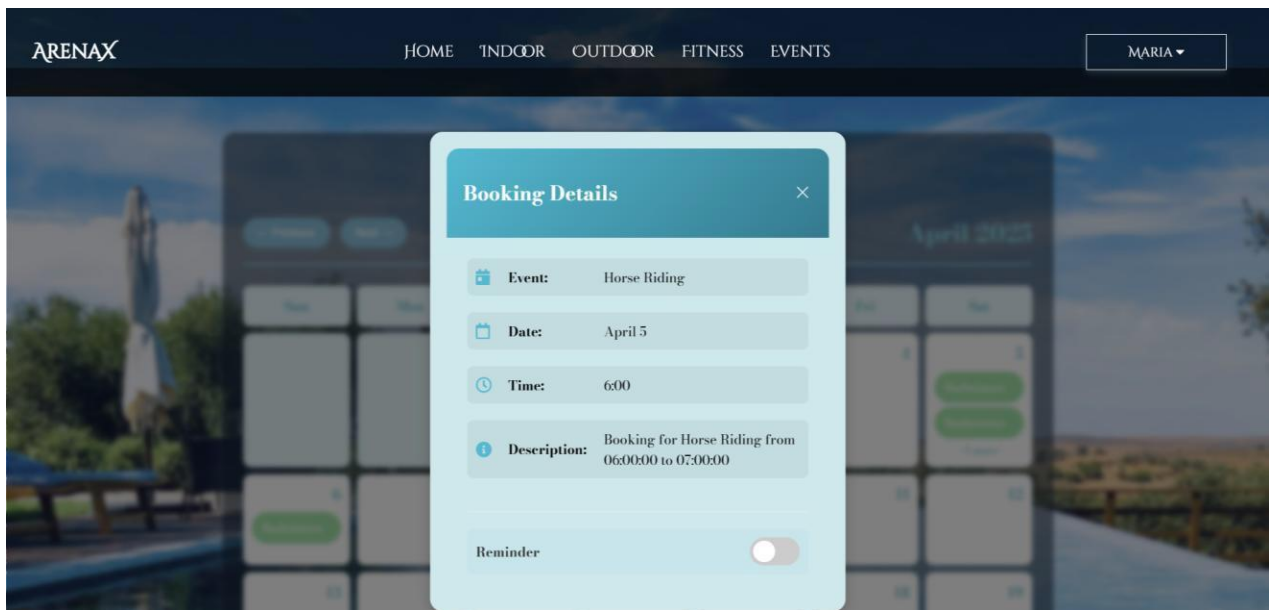
Test Case 3					
Project Name: ArenaX					
Payment History Test Case					
Test Case ID: Test_3			Test Designed By: Elizabeth Mary		
Test Priority (Low/Medium/High): High			Test Designed Date: 05/04/2025		
Module Name: Payment History			Test Executed By: Dr. Paulin Paul		
Test Title: Payment History			Test Execution Date: 05/04/2025		
Description: Payment History Test					
Pre-Condition: User should have made a booking					
Step	Test Step	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1	Navigate to the user dashboard	elizabethmary abraham09@gmail.com	Display user dashboard	User dashboard displayed	Pass
2	Navigate to users payment history page	Table Tennis	User should enter the payment history page	User entered the payment history page and viewed all the payments made	Pass
3	Display of the payment history	Payments made			
Post-Condition: All payments made for the bookings, memberships or events are viewed by the user					

Test Case 4: Calendar Functionality

Code:

```
test_calendar.py 8 X
C: > Users > eliza > Desktop > Test > test_calendar.py > ...
1  # Generated by Selenium IDE
2  import pytest
3  import time
4  import json
5  from selenium import webdriver
6  from selenium.webdriver.common.by import By
7  from selenium.webdriver.common.action_chains import ActionChains
8  from selenium.webdriver.support import expected_conditions
9  from selenium.webdriver.support.wait import WebDriverWait
10 from selenium.webdriver.common.keys import Keys
11 from selenium.webdriver.common.desired_capabilities import DesiredCapabilities
12
13 class TestCalendar():
14     def setup_method(self, method):
15         self.driver = webdriver.Chrome()
16         self.vars = {}
17
18     def teardown_method(self, method):
19         self.driver.quit()
20
21     def test_calendar(self):
22         self.driver.get("http://localhost/MiniProj/signin.php")
23         self.driver.set_window_size(1552, 832)
24         self.driver.find_element(By.NAME, "email").click()
25         self.driver.find_element(By.NAME, "email").send_keys("elizabethmaryabraham09@gmail.com")
26         self.driver.find_element(By.NAME, "password").click()
27         self.driver.find_element(By.NAME, "password").send_keys("12345678")
28         self.driver.find_element(By.CSS_SELECTOR, ".login-btn").click()
29         self.driver.find_element(By.LINK_TEXT, "Explore Activities").click()
30         self.driver.find_element(By.CSS_SELECTOR, ".day:nth-child(15) > .event").click()
31         self.driver.find_element(By.CSS_SELECTOR, ".modal-close-icon").click()
32
33
```

Screenshots:



Test Report:

Test Case 4

Project Name: ArenaX					
Calendar Test Case					
Test Case ID: Test_4			Test Designed By: Elizabeth Mary		
Test Priority (Low/Medium/High): High			Test Designed Date: 05/04/2025		
Module Name: Calendar			Test Executed By: Dr. Paulin Paul		
Test Title: Calendar			Test Execution Date: 05/04/2025		
Description: Calendar Test					
Pre-Condition: User should have made a booking					
Step	Test Step	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1	Navigate to the user dashboard	elizabethmar yabraham09 @gmail.com	Display user dashboard	User dashboard displayed	Pass
2	Navigate to calendar page	Table Tennis	View the upcoming schedules in the calendar	The schedules are viewed in the calendar	Pass
3	Display of the schedules in the calendar	Calendar schedules			
Post-Condition: All the schedules are displayed in the calendar for the respective dates fetched from the database					

CHAPTER 6

IMPLEMENTATION

6.1 INTRODUCTION

The project's implementation phase is where the conceptual design is transformed into a functional system. It can be regarded as the most important stage in creating a successful new system since it gives users assurance that the system will operate as intended and be reliable and accurate. User documentation and training are its main concerns. Usually, conversion happens either during or after the user's training. Implementation is the process of turning a newly revised system design into an operational one, and it simply refers to placing a new system design into operation. The user department now bears the most of the workload, faces the most disruption, and has the biggest influence on the current system. If the implementation is not carefully planned or controlled, it can create chaos and confusion. Implementation encompasses all of the steps used to switch from the old system to the new one. The new system could be entirely different, take the place of an existing manual or automated system, or it could be modified to work better. A reliable system that satisfies organizational needs must be implemented properly. System implementation refers to the process of actually using the built system. This comprises all the processes involved in switching from the old to the new system. Only after extensive testing and if it is determined that the system is operating in accordance with the standard can it be put into use. The system personnel check the feasibility of the system. The more complex the system being implemented, the more involved will be the system analysis and design effort required to implement the three main aspects: education and training, system testing and change over. The implementation state involves the following tasks:

- Careful planning.
- Investigation of system and constraints.
- Design of methods to achieve the changeover.

6.2 IMPLEMENTATION PROCEDURES

Software implementation refers to the complete installation of the package in its intended environment, as well as to the system's functionality and satisfaction of its intended applications. The software development project is frequently commissioned by someone who will not be using it. People have early reservations about the software, but we must watch out that they do not become more resistant by making sure that:

- The active user must be aware of the benefits of using the new system.

- Their confidence in the software is built up.
- Proper guidance is imparted to the user so that he is comfortable in using the application. Before going ahead and viewing the system, the user must know that for viewing the result, the server program should be running in the server. If the server object is not up running on the server, the actual process won't take place.

6.2.1 User Training

The purpose of user training is to get the user ready to test and modify the system. The people who will be involved must have faith in their ability to contribute to the goal and benefits anticipated from the computer-based system. Training is more necessary as systems get more complicated. The user learns how to enter data, handle error warnings, query the database, call up routines that will generate reports, and execute other important tasks through user training.

6.2.2 Training on the Application Software

The user will need to receive the essential basic training on computer awareness after which the new application software will need to be taught to them. This will explain the fundamental principles of how to use the new system, including how the screens work, what kind of help is displayed on them, what kinds of errors are made while entering data, how each entry is validated, and how to change the date that was entered. Then, while imparting the program's training on the application, it should cover the information required by the particular user or group to operate the system or a certain component of the system.

6.2.3 System Maintenance

Maintenance is the enigma of system development. The maintenance phase of the software cycle is the time in which a software product performs useful work. After a system is successfully implemented, it should be maintained in a proper manner. System maintenance is an important aspect in the software development life cycle. The need for system maintenance is for it to make adaptable to the changes in the system environment. Software maintenance is of course, far more than "Finding Mistakes".

6.2.4 Hosting

Hosting a website involves making the web pages and associated files of a website accessible to users over the internet. This process includes selecting a hosting provider and plan that meets the website's needs, uploading website files to the hosting server, configuring domain settings, and implementing security measures to protect the site and data. Regular monitoring, maintenance, and scaling ensure optimal performance and uptime, providing users with a seamless browsing experience. Sign up to make an account.

AWS (EC2)

AWS (Amazon Web Services) allows you to host your PHP applications on the cloud using various services that provide scalability, reliability, and security.

Amazon EC2 (Elastic Compute Cloud) is a foundational cloud service by AWS that provides scalable virtual servers to host web applications, including PHP-based websites. It is widely used for its flexibility and full control over the server environment, making it ideal for both developers and businesses with specific hosting requirements.

- **Hosting Services:** EC2 allows you to host dynamic web applications, APIs, and databases. You can set up a complete LAMP (Linux, Apache, MySQL, PHP) stack to run your PHP websites.
- **Flexibility:** EC2 provides root access to the server, enabling you to install and configure software based on your needs. This is ideal for advanced users requiring customized environments.

Procedure for hosting a website on AWS:

Step 1: Sign Up and Log In

Step 2: Launch an EC2 Instance

Step 3: Connect to the Instance

Step 4: Install LAMP Stack

Step 5: Upload Your PHP Files.

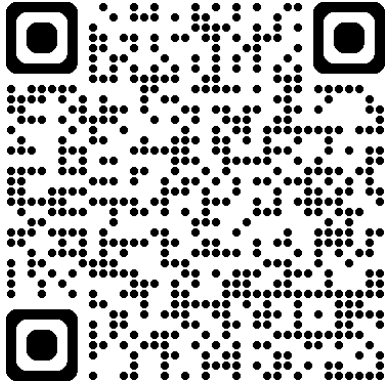
Step 6: Test Your Website

Step 7: Setup Domain and SSL

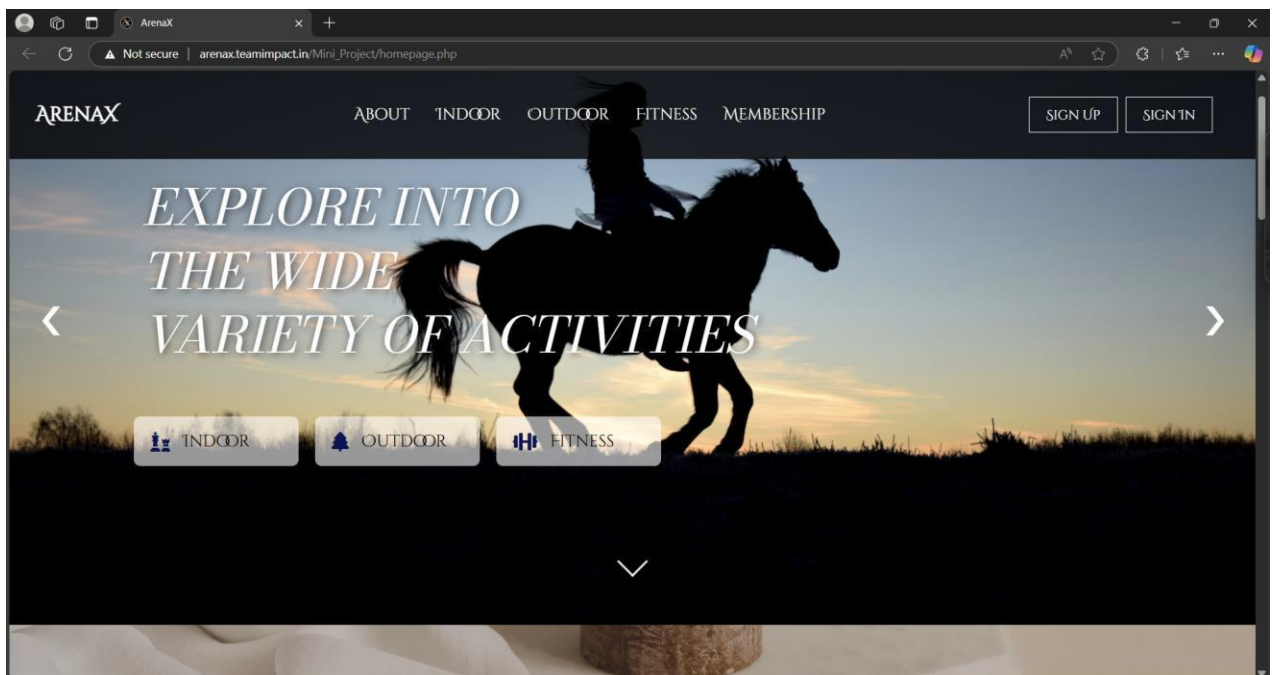
Hosted Website: <https://aws.amazon.com/>

Hosted Link: http://arenax.teamimpact.in/Mini_Project/homepage.php

Hosted Link QR Code:



Screenshot:



CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

ArenaX – Recreational Activity Booking System has been developed with precision to offer a seamless and efficient experience for both users and administrators. The goal of this platform is to simplify the process of booking indoor, outdoor, and fitness activities, as well as event participation. The system provides a user-friendly interface where users can easily browse activities, manage their bookings, and upgrade memberships according to their needs. With integrated payment systems and automated reminders, users enjoy a smooth and hassle-free booking experience. Administrators have the ability to manage activities, memberships, and events effectively. The system supports three types of memberships like Normal, Standard, and Premium with each offering various levels of access and benefits. Secure login, profile management, and activity tracking are all included to ensure privacy and ease of use. ArenaX has been thoroughly tested to ensure reliable performance and secure functionality, making it a robust solution for managing recreational services in a modern, digital environment.

7.2 FUTURE SCOPE

The ArenaX system holds great potential for future enhancements and expansion. As user demand grows, the platform can be extended to include mobile applications for both Android and iOS to improve accessibility and convenience. Advanced features like AI-powered activity recommendations based on user preferences and past bookings can be integrated to enhance personalization. A dynamic event creation module can allow organizers to post and manage their own events, expanding the platform's usability. Integration with wearable fitness devices and health tracking tools can also be considered to improve user engagement in fitness activities. Additionally, partnerships with local clubs and recreation centers can bring in a wider range of activities and offers for users. Enhanced analytics and reporting tools can help administrators gain deeper insights into user behaviour and improve service offerings. In the long run, ArenaX can evolve into a comprehensive ecosystem for all recreational and wellness-related services.

CHAPTER 8

BIBLIOGRAPHY

REFERENCES:

- JavaScript Enlightenment, Cody Lindley-First Edition, based on JavaScript 1.5, ECMA262, Edition
- Complete CSS Guide, Maxine Sherrin and John Allsopp-O'Reilly Media; September 2012
- IEEE Std 1016 Recommended Practice for Software Design Descriptions.

WEBSITES:

- <https://www.chittilappilysquare.com/>
- <https://play.decathlon.in/explore/sport-activities/?Offering=Events>
- <https://www.barrys.com/the-workout/>

CHAPTER 9

APPENDIX

9.1 Sample Code

User_bookings.php

```
<?php
session_start();
include('db_connect.php');

// Check if user is logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: signin.php');
    exit();
}

// Fetch user details with membership information
$user_id = $_SESSION['user_id'];
$user_email = $_SESSION['email']; // Ensure this is set from the session

// Get user's regular bookings (existing code)
$stmt = $conn->prepare("
    SELECT
        b.booking_id,
        sa.sub_activity_id,
        san.sub_act_name AS sub_activity_name,
        a.activity_type,
        DATE_FORMAT(ts.slot_date, '%d %M %Y') as formatted_date,
        TIME_FORMAT(ts.slot_start_time, '%l:%i %p') as start_time,
        TIME_FORMAT(ts.slot_end_time, '%l:%i %p') as end_time,
        p.amount,
        sa.sub_activity_price as original_price,
        DATE_FORMAT(b.booking_date, '%d %M %Y') as booking_date,
        TIME_FORMAT(b.booking_time, '%l:%i %p') as booking_time,
        'regular' as booking_type
    FROM booking b
    JOIN sub_activity sa ON b.sub_activity_id = sa.sub_activity_id
    JOIN sub_activity_name san ON sa.sub_act_id = san.sub_act_id
    JOIN activity a ON sa.activity_id = a.activity_id
    JOIN timeslots ts ON b.slot_id = ts.slot_id
    LEFT JOIN payment p ON b.booking_id = p.booking_id
    WHERE b.user_id = ?
");
$stmt->execute([$_SESSION['user_id']]);
$regular_bookings = $stmt->fetchAll(PDO::FETCH_ASSOC);

// Get user's recurring bookings
$stmt = $conn->prepare("
    SELECT
        rb.recurring_id,
        rb.sub_activity_id,
        san.sub_act_name AS sub_activity_name,
```

```

        a.activity_type,
        DATE_FORMAT(rb.start_date, '%d %M %Y') as start_date,
        DATE_FORMAT(rb.end_date, '%d %M %Y') as end_date,
        TIME_FORMAT(rb.booking_time, '%l:%i %p') as booking_time,
        TIME_FORMAT(ADDTIME(rb.booking_time, '1:00:00'), '%l:%i %p') as end_time,
        rb.selected_days,
        rb.status,
        DATE_FORMAT(rb.created_at, '%d %M %Y') as created_date,
        TIME_FORMAT(rb.created_at, '%l:%i %p') as created_time,
        'recurring' as booking_type,
        sa.sub_activity_price as price_per_session
    FROM recurring_bookings rb
    JOIN sub_activity sa ON rb.sub_activity_id = sa.sub_activity_id
    JOIN sub_activity_name san ON sa.sub_act_id = san.sub_act_id
    JOIN activity a ON sa.activity_id = a.activity_id
    WHERE rb.user_id = ?
    ORDER BY rb.start_date DESC
");
$stmt->execute([$SESSION['user_id']]);
$recurring_bookings = $stmt->fetchAll(PDO::FETCH_ASSOC);

// Combine all bookings
$all_bookings = array_merge($regular_bookings, $recurring_bookings);

// After the existing bookings query, update this query for events
$stmt = $conn->prepare("
    SELECT
        er.event_reg_id,
        e.event_title,
        e.event_description,
        DATE_FORMAT(e.event_date, '%d %M %Y') as formatted_date,
        TIME_FORMAT(e.event_time, '%l:%i %p') as event_time,
        e.event_location,
        e.event_price,
        a.activity_type,
        CASE
            WHEN u.membership_id = (SELECT membership_id FROM memberships WHERE
membership_type = 'premium')
            THEN 'Premium Registration'
            ELSE 'Standard Registration'
        END as registration_type
    FROM event_registration er
    JOIN events e ON er.event_id = e.event_id
    JOIN activity a ON e.activity_id = a.activity_id
    JOIN users u ON er.user_id = u.user_id
    WHERE er.user_id = ?
    ORDER BY e.event_date DESC, e.event_time DESC
");
$stmt->execute([$SESSION['user_id']]);
$event_registrations = $stmt->fetchAll(PDO::FETCH_ASSOC);

```

```

try {
    $query = "SELECT u.*, m.membership_type
              FROM users u
              LEFT JOIN memberships m ON u.membership_id = m.membership_id
              WHERE u.user_id = :user_id";
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':user_id', $user_id, PDO::PARAM_INT);
    $stmt->execute();

    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$user) {
        // Handle case where user is not found
        $_SESSION['error'] = "User not found";
        header('Location: logout.php');
        exit();
    }
} catch(PDOException $e) {
    // Log error and show user-friendly message
    error_log("Database error: " . $e->getMessage());
    $_SESSION['error'] = "An error occurred while fetching user data";
    header('Location: error.php');
    exit();
}

// Modify the existing SQL query to also fetch the user's name
$sql = "SELECT membership_id, name FROM users WHERE email = :email"; // Use a prepared
statement
$stmt = $conn->prepare($sql);
$stmt->bindParam(':email', $user_email, PDO::PARAM_STR);
$stmt->execute();

if ($stmt->rowCount() > 0) { // Use rowCount() instead of num_rows
    $row = $stmt->fetch(PDO::FETCH_ASSOC);
    $membership_id = $row['membership_id'];
    $user_name = $row['name'];
} else {
    $user_name = "Profile"; // Default value
}
error_reporting(E_ALL);
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_log("Starting profile update process");

// Function to send error response

?>

<!DOCTYPE html>
<html lang="en">

```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Bookings</title>
  <link rel="icon" href="img/logo3.png" type="image/png">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/5.15.4/css/all.min.css">
  <style>
    @import url('https://fonts.googleapis.com/css2?family=Aboreto&display=swap');
    @import
url('https://fonts.googleapis.com/css2?family=Cinzel+Decorative:wght@400&family=Goldman&
display=swap');
    @import
url('https://fonts.googleapis.com/css2?family=Cinzel+Decorative:wght@400&display=swap');
    @import
url('https://fonts.googleapis.com/css2?family=Bodoni+Moda:wght@400;700&display=swap');

    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Bodoni Moda', serif;
      background: linear-gradient(rgba(0, 0, 0, 0.7), rgba(0, 0, 0, 0.7)), url('img/event2.jpg') no-
repeat center center fixed;
      background-size: cover;
      color: white;
      overflow-x: hidden;
    }

    /* Header styles from user_home.php */
    .header {
      font-family: 'Cinzel Decorative', cursive;
      position: fixed;
      top: 0;
      left: 0;
      width: 100%;
      background: rgba(0, 0, 0, 0.8);
      padding: 2rem 2rem;
      display: flex;
      justify-content: space-between;
      align-items: center;
      z-index: 1000;
    }

    .header nav ul {
      list-style: none;
      display: flex;
      gap: 2rem;
```

```
        justify-content: center;
        margin: 0 auto;
        padding: 0;
    }

    .header div a button {
        transition: background-color 0.3s ease-in-out, transform 0.2s ease;
    }

    .header div a button:hover {
        color: #00bcd4;
        border-color: #00bcd4;
        transform: scale(1.1);
    }

    .log {
        padding: 10px 20px;
        font-size: 1rem;
        font-family: 'Cinzel Decorative', cursive;
        background-color: #007cd400;
        color: white;
        padding: 10px 50px;
        border-style: solid;
        border-width: 1px;
        border-color: white;
        border-radius: 0px;
        cursor: pointer;
        transition: background-color 0.3s ease-in-out;
    }

    .log:hover {
        color: #00bcd4;
        border-color: #00bcd4;
    }

    .header div {
        display: flex;
        gap: 15px;
        margin-right: 40px;
    }

    .header nav ul li {
        position: relative;
    }

    .header nav ul li a {
        text-decoration: none;
        color: white;
        font-size: 1.2rem;
        transition: color 0.2s ease-in-out;
    }
```

```
.header nav ul li a::after {
  content: "";
  position: absolute;
  bottom: -4px;
  left: 50%;
  width: 0;
  height: 2px;
  background: #00bcd4;
  transition: all 0.3s ease-in-out;
  transform: translateX(-50%);
}

.header nav ul li a:hover::after {
  width: 100%;
}

.header nav ul li a:hover {
  color: #00bcd4;
}

.are {
  text-decoration: none;
}

.dropdown {
  display: none;
  position: absolute;
  background-color: rgba(0, 0, 0, 0.9);
  min-width: 200px;
  border-radius: 0;
  padding: 8px 0;
  box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
  z-index: 1000;
  top: 100%;
  left: 0;
  border: 1px solid rgba(255, 255, 255, 0.1);
}

.dropdown a {
  display: block;
  padding: 12px 20px;
  color: #fff;
  text-decoration: none;
  font-family: 'Bodoni Moda', serif;
  font-size: 0.9rem;
  transition: all 0.3s ease;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

.dropdown a:last-child {
```

```
border-bottom: none;
}

.dropdown a:hover {
  background-color: rgba(0, 188, 212, 0.2);
  padding-left: 25px;
  color: #00bcd4;
}

.dropdown::before {
  content: "";
  position: absolute;
  top: -8px;
  left: 20px;
  border-left: 8px solid transparent;
  border-right: 8px solid transparent;
  border-bottom: 8px solid rgba(0, 0, 0, 0.9);
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
}

.page-title {
  text-align: center;
  color: white;
  margin-bottom: 40px;
  font-size: 2.5rem;
  position: relative;
  padding-top: 100px;
  text-transform: uppercase;
}

.page-title:after {
  content: "";
  display: block;
  width: 60px;
  height: 4px;
  background: #72aab0;
  margin: 10px auto;
  border-radius: 2px;
}

.bookings-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(350px, 1fr));
  gap: 25px;
```

```
    margin-top: 30px;
  }

.booking-card {
  background: rgba(255, 255, 255, 0.95);
  border-radius: 15px;
  padding: 25px;
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.1);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.booking-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 12px 25px rgba(0, 0, 0, 0.15);
}

.booking-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
  padding-bottom: 15px;
  border-bottom: 2px solid #eee;
}

.activity-name {
  font-size: 1.4rem;
  color: #2c3e50;
  font-weight: 600;
}

.booking-id {
  color: #72aab0;
  font-size: 0.9rem;
}

.booking-details {
  margin-bottom: 20px;
  font-family: 'Bodoni Moda', serif;
}

.booking-details .detail-row {
  font-family: Arial, sans-serif;
}

.detail-row {
  display: flex;
  margin-bottom: 12px;
  align-items: center;
}
```

```
.detail-label {
  width: 120px;
  color: #666;
  font-weight: 500;
}

.detail-value {
  color: #2c3e50;
  flex: 1;
}

.booking-footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 20px;
  padding-top: 15px;
  border-top: 2px solid #eee;
}

.price {
  font-size: 1.2rem;
  color: #2c3e50;
  font-weight: 600;
}

.booking-date {
  color: #666;
  font-size: 0.9rem;
}

.status-badge {
  padding: 8px 15px;
  border-radius: 20px;
  font-size: 0.9rem;
  font-weight: 500;
}

.status-upcoming {
  background: #e3f2fd;
  color: #1976d2;
}

.status-completed {
  background: #e8f5e9;
  color: #2e7d32;
}

.no-bookings {
  text-align: center;
  padding: 40px;
```

```
background: rgba(255, 255, 255, 0.95);
border-radius: 15px;
margin: 20px auto;
max-width: 600px;
font-family: 'Bodoni Moda', serif;
}

.no-bookings i {
  font-size: 50px;
  color: #72aab0;
  margin-bottom: 20px;
}

.no-bookings h3 {
  color: #2c3e50;
  margin-bottom: 10px;
}

.no-bookings p {
  color: #666;
  margin-bottom: 20px;
}

.book-now-btn {
  display: inline-block;
  padding: 12px 25px;
  background: #72aab0;
  color: white;
  text-decoration: none;
  border-radius: 25px;
  transition: background 0.3s ease;
}

.book-now-btn:hover {
  background: #5d8f94;
}

@media (max-width: 768px) {
  .bookings-grid {
    grid-template-columns: 1fr;
  }

  .booking-card {
    margin: 10px;
  }

  .page-title {
    font-size: 2rem;
  }
}
</style>
```

```

</head>
<body>
  <!-- Header Section -->
  <header class="header">
    <div class="logo">
      <a href="user_home.php" class="are"><h2 style="color: white;">ArenaX</h2></a>
    </div>
    <nav>
      <ul style="display: flex; justify-content: center; width: 100%;">
        <li><a href="user_home.php">Home</a></li>
        <li><a href="user_indoor.php">Indoor</a></li>
        <li><a href="user_outdoor.php">Outdoor</a></li>
        <li><a href="user_fitness.php">Fitness</a></li>
        <li><a href="user_events.php">Events</a></li>
      </ul>
    </nav>
    <div style="margin-right: 20px; position: relative;">
      <button class="log"><?php echo htmlspecialchars($user_name); ?> <i class="fas fa-caret-
down"></i></button>
      <div class="dropdown">
        <a href="user_profile.php">PROFILE</a>
        <a href="user_bookings.php">BOOKINGS</a>
        <a href="user_calendar.php">CALENDER</a>
        <a href="user_payment_history.php">PAYMENT HISTORY</a>
        <a href="logout.php">LOGOUT</a>
      </div>
    </div>
  </header>

  <!-- Profile Content -->
  <div class="container">
    <h1 class="page-title">My Bookings</h1>

    <?php if (isset($_SESSION['success_message'])): ?>
      <div class="success-message">
        <?php
          echo $_SESSION['success_message'];
          unset($_SESSION['success_message']);
        ?>
      </div>
    <?php endif; ?>

    <?php if (empty($all_bookings)): ?>
      <div class="no-bookings">
        <i class="fas fa-calendar-alt"></i>
        <h3>No Bookings Found</h3>
        <p>You haven't made any bookings yet. Start exploring our activities!</p>
        <a href="user_outdoor.php" class="book-now-btn">Book Now</a>
      </div>
    <?php else: ?>
      <div class="bookings-grid">

```

```

<?php foreach ($all_bookings as $booking):
    if ($booking['booking_type'] === 'regular') {
        $isUpcoming = strtotime($booking['formatted_date']) >= strtotime('today');
        // Regular booking display
        ?>
        <div class="booking-card">
            <div class="booking-header">
                <div class="activity-name"><?php echo
strtoupper(htmlspecialchars($booking['sub_activity_name'])); ?></div>
                <!--
                <div class="booking-id">#<?php echo $booking['booking_id']; ?></div>
                -->
            </div>

            <div class="booking-details">
                <div class="detail-row">
                    <span class="detail-label">Activity Type:</span>
                    <span class="detail-value"><?php echo
htmlspecialchars($booking['activity_type']); ?></span>
                </div>
                <div class="detail-row">
                    <span class="detail-label">Date:</span>
                    <span class="detail-value"><?php echo $booking['formatted_date'];
?></span>
                </div>
                <div class="detail-row">
                    <span class="detail-label">Time:</span>
                    <span class="detail-value">
                        <?php echo $booking['start_time'] . ' - ' . $booking['end_time']; ?>
                    </span>
                </div>
                <div class="detail-row">
                    <span class="detail-label">Status:</span>
                    <span class="status-badge <?php echo $isUpcoming ? 'status-upcoming' :
'status-completed'; ?>">
                        <?php echo $isUpcoming ? 'Upcoming' : 'Completed'; ?>
                    </span>
                </div>
                <div class="detail-row" style="padding-left: 0px;">
                    <span class="detail-label">Booked on:</span>
                    <span class="detail-value">
                        <?php echo $booking['booking_date'] . ' at ' . $booking['booking_time'];
?>
                    </span>
                </div>
            </div>

            <div class="booking-footer">
                <div class="price">
                    <?php
                    if (isset($booking['amount']) && $booking['amount'] !== null) {

```

```

        echo '₹' . number_format($booking['amount'], 2);
    } else {
        echo '₹' . number_format($booking['original_price'], 2) . ' (Membership
benefit)';
    }
?>
</div>
<!--
<div class="booking-date">
    Booked on <?php echo $booking['booking_date']; ?> at <?php echo
$booking['booking_time']; ?>
</div>
-->
</div>
</div>
<?php } else {
    // Recurring booking display
    $isActive = $booking['status'] === 'active';
    $dayNames = [
        '0' => 'Sunday', '1' => 'Monday', '2' => 'Tuesday',
        '3' => 'Wednesday', '4' => 'Thursday', '5' => 'Friday', '6' => 'Saturday'
    ];
    $selectedDays = json_decode($booking['selected_days']);
    $daysList = array_map(function($day) use ($dayNames) {
        return $dayNames[$day];
    }, $selectedDays);
    ?>
    <div class="booking-card">
        <div class="booking-header">
            <div class="activity-name"><?php echo
strtoupper(htmlspecialchars($booking['sub_activity_name'])); ?></div>
            <div class="booking-id">Recurring</div>
        </div>

        <div class="booking-details">
            <div class="detail-row">
                <span class="detail-label">Activity Type:</span>
                <span class="detail-value"><?php echo
htmlspecialchars($booking['activity_type']); ?></span>
            </div>
            <div class="detail-row">
                <span class="detail-label">Period:</span>
                <span class="detail-value">
                    <?php echo $booking['start_date'] . ' to ' . $booking['end_date']; ?>
                </span>
            </div>
            <div class="detail-row">
                <span class="detail-label">Time:</span>
                <span class="detail-value">
                    <?php echo $booking['booking_time'] . ' - ' . $booking['end_time']; ?>
                </span>
            </div>
        </div>
    </div>
}
?>

```

```

        </div>
        <div class="detail-row">
            <span class="detail-label">Days:</span>
            <span class="detail-value"><?php echo implode(' ', $daysList); ?></span>
        </div>
        <div class="detail-row">
            <span class="detail-label">Status:</span>
            <span class="status-badge <?php echo $isActive ? 'status-upcoming' :
'status-completed'; ?>">
                <?php echo ucfirst($booking['status']); ?>
            </span>
        </div>
        <div class="detail-row">
            <span class="detail-label">Booked on:</span>
            <span class="detail-value">
                <?php echo $booking['created_date'] . ' at ' . $booking['created_time']; ?>
            </span>
        </div>
    </div>

    <div class="booking-footer">
        <div class="price">
            <?php echo '₹' . number_format($booking['price_per_session'], 2) . ' per
session'; ?>
        </div>
    </div>
</div>
<?php } ?>
<?php endforeach; ?>
</div>
<?php endif; ?>

<h1 class="page-title">My Event Registrations</h1>

<?php if (empty($event_registrations)): ?>
    <div class="no-bookings">
        <i class="fas fa-calendar-check"></i>
        <h3>No Event Registrations Found</h3>
        <p>You haven't registered for any events yet. Check out our upcoming events!</p>
        <a href="user_events.php" class="book-now-btn">View Events</a>
    </div>
<?php else: ?>
    <div class="bookings-grid">
        <?php foreach ($event_registrations as $event):
            $isUpcoming = strtotime($event['formatted_date']) >= strtotime('today');
            ?>
            <div class="booking-card">
                <div class="booking-header">
                    <div class="activity-name"><?php echo htmlspecialchars($event['event_title']);
?></div>
                </div>
            </div>
        </div>
    </div>

```

```

<div class="booking-details">
  <div class="detail-row">
    <span class="detail-label">Activity Type:</span>
    <span class="detail-value"><?php echo
htmlspecialchars($event['activity_type']); ?></span>
  </div>
  <div class="detail-row">
    <span class="detail-label">Date:</span>
    <span class="detail-value"><?php echo $event['formatted_date']; ?></span>
  </div>
  <div class="detail-row">
    <span class="detail-label">Time:</span>
    <span class="detail-value"><?php echo $event['event_time']; ?></span>
  </div>
  <div class="detail-row">
    <span class="detail-label">Location:</span>
    <span class="detail-value"><?php echo
htmlspecialchars($event['event_location']); ?></span>
  </div>
  <div class="detail-row">
    <span class="detail-label">Status:</span>
    <span class="status-badge <?php echo $isUpcoming ? 'status-upcoming' :
'status-completed'; ?>">
      <?php echo $isUpcoming ? 'Upcoming' : 'Completed'; ?>
    </span>
  </div>
</div>

<div class="booking-footer">
  <div class="price">
    <?php
    if ($event['registration_type'] === 'Premium Registration') {
      echo '<span style="color: #72aab0;">Premium Member - Free
Entry</span>';
    } else {
      echo '₹' . number_format($event['event_price'], 2);
    }
    ?>
  </div>
</div>
</div>
<?php endforeach; ?>
</div>
<?php endif; ?>
</div>

<script>
const header = document.querySelector('.header');
window.addEventListener('scroll', () => {
  if (window.scrollY > 50) {

```

```

        header.classList.add('scrolled');
    } else {
        header.classList.remove('scrolled');
    }
});

const profileButton = document.querySelector('.log');
const dropdown = document.querySelector('.dropdown');
dropdown.style.display = 'none'; // Ensure dropdown is hidden initially

profileButton.addEventListener('click', () => {
    if (dropdown.style.display === 'none' || dropdown.style.display === "") {
        dropdown.style.display = 'block';
    } else {
        dropdown.style.display = 'none';
    }
});

document.addEventListener('click', function(event) {
    const dropdown = document.querySelector('.dropdown');
    const profileButton = document.querySelector('.log');

    if (!dropdown.contains(event.target) && !profileButton.contains(event.target)) {
        dropdown.style.display = 'none';
    }
});

dropdown.addEventListener('click', function(event) {
    event.stopPropagation();
});
</script>
</body>
</html>

```

User_calendar.php

```

<?php
session_start();
require_once 'db_connect.php'; // Update the path as necessary

// Redirect to login if user is not logged in
if (!isset($_SESSION['email'])) {
    header("Location: signin.php");
    exit();
}

// Fetch user's membership and name
$user_email = $_SESSION['email']; // Define $user_email from session
$sql = "SELECT u.membership_id, u.name, m.membership_type

```

```
        FROM users u
        JOIN memberships m ON u.membership_id = m.membership_id
        WHERE u.email = ?";
$stmt = $conn->prepare($sql);
$stmt->bindParam(1, $user_email);
$stmt->execute();
$userData = $stmt->fetch(PDO::FETCH_ASSOC);

$membership_type = isset($userData['membership_type']) ?
strtolower($userData['membership_type']) : "normal";
$isPremiumMembership = ($membership_type === 'premium');

// Check if user has normal membership
$isNormalMembership = ($membership_type === 'normal');

// Modify the existing SQL query to also fetch the user's name
$sql = "SELECT membership_id, name, user_id FROM users WHERE email = '$user_email'";
$stmt = $conn->query($sql); // Use query() to execute the SQL statement
$user_id = null;

if ($stmt && $stmt->rowCount() > 0) { // Use rowCount() to check for rows
    $row = $stmt->fetch(PDO::FETCH_ASSOC);
    $membership_id = $row['membership_id'];
    $user_id = $row['user_id'];
}

// Create a new table for calendar events if it doesn't exist
$createTableSQL = "CREATE TABLE IF NOT EXISTS user_calendar_events (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    date DATE NOT NULL,
    time TIME DEFAULT NULL,
    description TEXT,
    event_type ENUM('custom', 'booking', 'event') NOT NULL DEFAULT 'custom',
    related_id INT DEFAULT NULL,
    reminder_enabled BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
)";

$conn->exec($createTableSQL);

// Create a table for reminders if it doesn't exist
$createRemindersTableSQL = "CREATE TABLE IF NOT EXISTS user_reminders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    event_id INT NOT NULL,
    reminder_time DATETIME NOT NULL,
    is_sent BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (event_id) REFERENCES user_calendar_events(id) ON DELETE CASCADE
```

```
);

$conn->exec($createRemindersTableSQL);

// Fetch user's bookings
$bookingsSQL = "SELECT b.booking_id, s.sub_act_name, t.slot_date, t.slot_start_time,
t.slot_end_time
    FROM booking b
    JOIN timeslots t ON b.slot_id = t.slot_id
    JOIN sub_activity sa ON b.sub_activity_id = sa.sub_activity_id
    JOIN sub_activity_name s ON sa.sub_act_id = s.sub_act_id
    WHERE b.user_id = :user_id";

$bookingsStmt = $conn->prepare($bookingsSQL);
$bookingsStmt->execute(['user_id' => $user_id]);
$bookings = $bookingsStmt->fetchAll(PDO::FETCH_ASSOC);

// Fetch user's event registrations
$eventsSQL = "SELECT er.event_reg_id, e.event_title, e.event_date, e.event_time,
e.event_description
    FROM event_registration er
    JOIN events e ON er.event_id = e.event_id
    WHERE er.user_id = :user_id";

$eventsStmt = $conn->prepare($eventsSQL);
$eventsStmt->execute(['user_id' => $user_id]);
$events = $eventsStmt->fetchAll(PDO::FETCH_ASSOC);

// Sync bookings and events to the calendar_events table if not already there
foreach ($bookings as $booking) {
    $checkSQL = "SELECT id FROM user_calendar_events
        WHERE user_id = :user_id AND event_type = 'booking'
        AND related_id = :booking_id";
    $checkStmt = $conn->prepare($checkSQL);
    $checkStmt->execute([
        'user_id' => $user_id,
        'booking_id' => $booking['booking_id']
    ]);

    if ($checkStmt->rowCount() == 0) {
        // Create a new calendar event for this booking
        $insertSQL = "INSERT INTO user_calendar_events
            (user_id, title, date, time, description, event_type, related_id)
            VALUES (:user_id, :title, :date, :time, :description, 'booking', :related_id)";
        $insertStmt = $conn->prepare($insertSQL);
        $insertStmt->execute([
            'user_id' => $user_id,
            'title' => $booking['sub_act_name'],
            'date' => $booking['slot_date'],
            'time' => $booking['slot_start_time'],
            'description' => "Booking for " . $booking['sub_act_name'] . " from " .
```

```

        $booking['slot_start_time'] . " to " . $booking['slot_end_time'],
        'related_id' => $booking['booking_id']
    );
}
}

foreach ($events as $event) {
    $checkSQL = "SELECT id FROM user_calendar_events
        WHERE user_id = :user_id AND event_type = 'event'
        AND related_id = :event_reg_id";
    $checkStmt = $conn->prepare($checkSQL);
    $checkStmt->execute([
        'user_id' => $user_id,
        'event_reg_id' => $event['event_reg_id']
    ]);

    if ($checkStmt->rowCount() == 0) {
        // Create a new calendar event for this event registration
        $insertSQL = "INSERT INTO user_calendar_events
            (user_id, title, date, time, description, event_type, related_id)
            VALUES (:user_id, :title, :date, :time, :description, 'event', :related_id)";
        $insertStmt = $conn->prepare($insertSQL);
        $insertStmt->execute([
            'user_id' => $user_id,
            'title' => $event['event_title'],
            'date' => $event['event_date'],
            'time' => $event['event_time'],
            'description' => $event['event_description'],
            'related_id' => $event['event_reg_id']
        ]);
    }
}

// Process notification reminders for upcoming events/bookings
$today = date('Y-m-d');
$twoDaysLater = date('Y-m-d', strtotime('+2 days'));

// Check for events in the next 2 days that need reminders
$upcomingSQL = "SELECT uce.id, uce.title, uce.date, uce.time, uce.event_type
    FROM user_calendar_events uce
    LEFT JOIN user_reminders ur ON uce.id = ur.event_id AND ur.is_sent = 0
    WHERE uce.user_id = :user_id
    AND uce.date BETWEEN :today AND :two_days_later
    AND (ur.id IS NULL OR ur.is_sent = 0)
    AND uce.reminder_enabled = 1";

$upcomingStmt = $conn->prepare($upcomingSQL);
$upcomingStmt->execute([
    'user_id' => $user_id,
    'today' => $today,
    'two_days_later' => $twoDaysLater

```

```

]);

$upcomingEvents = $upcomingStmt->fetchAll(PDO::FETCH_ASSOC);

// Generate notifications for upcoming events
foreach ($upcomingEvents as $upcoming) {
    $daysUntil = (strtotime($upcoming['date']) - strtotime($today)) / (60 * 60 * 24);
    $daysText = $daysUntil <= 1 ? "tomorrow" : "in " . ceil($daysUntil) . " days";

    $eventType = $upcoming['event_type'] == 'booking' ? 'activity' : 'event';

    // Insert notification
    $notifySQL = "INSERT INTO notification
        (user_id, title, message, notification_viewed, created_at_date, created_at_time)
        VALUES (:user_id, :title, :message, 0, CURDATE(), CURTIME())";
    $notifyStmt = $conn->prepare($notifySQL);
    $notifyStmt->execute([
        'user_id' => $user_id,
        'title' => "Upcoming " . ucfirst($eventType) . " Reminder",
        'message' => "Reminder: You have " . $upcoming['title'] . " scheduled " . $daysText .
            " (" . $upcoming['date'] . ") at " . ($upcoming['time'] ? $upcoming['time'] : "All day")
    ]);
}

// Mark reminder as sent
$reminderSQL = "INSERT INTO user_reminders
    (user_id, event_id, reminder_time, is_sent)
    VALUES (:user_id, :event_id, NOW(), 1)";
$reminderStmt = $conn->prepare($reminderSQL);
$reminderStmt->execute([
    'user_id' => $user_id,
    'event_id' => $upcoming['id']
]);
}

// Get all user's calendar events for JavaScript
$userEventsSQL = "SELECT id, title, date, time, description, event_type, reminder_enabled
    FROM user_calendar_events
    WHERE user_id = :user_id";
$userEventsStmt = $conn->prepare($userEventsSQL);
$userEventsStmt->execute(['user_id' => $user_id]);
$userEvents = $userEventsStmt->fetchAll(PDO::FETCH_ASSOC);

// Prepare the calendar events data for JavaScript
$calendarEventsJson = json_encode($userEvents);
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Calendar</title>

<!-- Favicon link -->
<link rel="icon" href="img/logo3.png" type="image/png">

<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/5.15.4/css/all.min.css">

<style>
  @import url('https://fonts.googleapis.com/css2?family=Aboreto&display=swap');
  @import
url('https://fonts.googleapis.com/css2?family=Cinzel+Decorative:wght@400&family=Goldman&
display=swap');
  @import
url('https://fonts.googleapis.com/css2?family=Bodoni+Moda:wght@400;700&display=swap');

  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
  }

  body {
    font-family: 'Cinzel Decorative', cursive;
    background: url('img/out1.png') no-repeat center center;
    background-size: cover;
    overflow-x: hidden;
  }

  .header {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background: rgba(0, 0, 0, 0.8);
    padding: 2rem 2rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
    z-index: 1000;
  }

  .header.scrolled {
    background: rgba(0, 0, 0, 1);
  }

  .header nav ul {
    list-style: none;
    display: flex;
```

```
        gap: 2rem;
        justify-content: center;
        margin: 0 auto;
        padding: 0;
    }

    .header div a button {
        transition: background-color 0.3s ease-in-out, transform 0.2s ease;
    }

    .header div a button:hover {
        color: #00bcd4;
        border-color: #00bcd4;
        transform: scale(1.1);
    }

    .log {
        padding: 10px 20px;
        font-size: 1rem;
        font-family: 'Cinzel Decorative', cursive;
        background-color: #007cd400;
        color: white;
        padding: 10px 50px;
        border-style: solid;
        border-width: 1px;
        border-color: white;
        border-radius: 0px;
        cursor: pointer;
        transition: background-color 0.3s ease-in-out;
    }

    .log:hover {
        color: #00bcd4;
        background-color: rgba(0, 0, 0, 0);
        border-color: #00bcd4;
    }

    .header div {
        display: flex;
        gap: 15px;
        margin-right: 40px; /* Adjust spacing to push buttons further right */
    }

    .header nav ul li {
        position: relative;
    }

    .header nav ul li a {
        text-decoration: none;
        color: white;
        font-size: 1.2rem;
```

```
        transition: color 0.2s ease-in-out;
    }

    .header nav ul li a::after {
        content: "";
        position: absolute;
        bottom: -4px;
        left: 50%;
        width: 0;
        height: 2px;
        background: #00bcd4;
        transition: all 0.3s ease-in-out;
        transform: translateX(-50%);
    }

    .header nav ul li a:hover::after {
        width: 100%;
    }

    .header nav ul li a:hover {
        color: #00bcd4;
    }

    .are {
        text-decoration: none;
    }

    /* Second Section: About What We Offer for Indoor Games */
    .about-section {
        padding: 4rem 2rem;
        background: linear-gradient(rgba(0, 0, 0, 0.38), rgba(0, 0, 0, 0.64)), url('img/r13.jpg') no-
repeat center center;
        background-size: cover;
        display: flex;
        flex-direction: column;
        align-items: center;
        gap: 3rem;
        max-width: 100%;
        margin: 0 auto;
        color: white;
        text-align: center;
        scroll-margin-top: 100px;
    }

    .about-content {
        opacity: 0;
        transform: translateY(50px);
        transition: all 1.5s ease;
    }

    .about-content.visible {
```

```
    opacity: 1;
    transform: translateY(0);
}

.about-image {
    opacity: 0;
    transform: translateX(-50px);
    transition: all 1.5s ease;
}

.about-image.visible {
    opacity: 1;
    transform: translateX(0);
}

.about-image img {
    width: 100%;
    max-width: 2000px;
    height: auto;
    border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}

.about-content h2 {
    font-size: 2.8rem;
    margin-bottom: 3rem;
    text-align: center;
    color: white;
    font-family: 'Bodoni Moda', serif;
    position: relative;
}

.about-content h2::after {
    content: "";
    position: absolute;
    bottom: -8px;
    left: 50%;
    transform: translateX(-50%);
    width: 100px;
    height: 3px;
    background: linear-gradient(90deg, #00bcd4, #ff4081, #00bcd4);
    background-size: 200% 100%;
    animation: gradientMove 3s ease infinite;
}

.about-content p {
    font-size: 1.2rem;
    line-height: 1.6;
    font-family: 'Aboreto', cursive;
    color: white;
}
```

```
@media screen and (max-width: 768px) {
  .about-section {
    flex-direction: column;
    padding: 2rem 1rem;
  }

  .about-image {
    min-width: 100%;
  }
}

/* Footer Styles */
footer {
  background-color: #282c34;
  font-family: 'Goldman', cursive;
  color: white;
  padding: 40px 20px;
}

.footer-container {
  display: flex;
  justify-content: space-between;
  flex-wrap: wrap;
  max-width: 1200px;
  margin: 0 auto;
}

.footer-column {
  flex: 1;
  min-width: 250px;
  margin: 10px;
}

.footer-column h3 {
  margin-bottom: 15px;
  font-size: 18px;
  text-transform: uppercase;
  color: #9f799e; /* Highlighted color for headings */
}

.footer-column p,
.footer-column ul {
  font-size: 14px;
  line-height: 1.6;
}

.footer-column ul {
  list-style: none;
  padding: 0;
```

```
}

.footer-column ul li {
  margin-bottom: 10px;
}

.footer-column ul li a {
  color: white;
  text-decoration: none;
  transition: color 0.3s ease;
}

.footer-column ul li a:hover {
  color: #00eeff; /* Highlight color on hover */
}

.social-links {
  display: flex;
  gap: 10px;
}

.social-links a {
  color: white;
  font-size: 20px;
  transition: color 0.3s ease;
}

.social-links a:hover {
  color: #6ad3d8; /* Highlight color on hover */
}

.footer-bottom {
  text-align: center;
  padding-top: 20px;
  font-size: 14px;
  border-top: 1px solid #444;
  margin-top: 20px;
}

@media (max-width: 768px) {
  .footer-container {
    flex-direction: column;
    text-align: center;
  }

  .footer-column {
    margin: 20px 0;
  }
}
```

```
/* Dropdown styles */
.dropdown {
  display: none;
  position: absolute;
  background-color: rgba(0, 0, 0, 0.9);
  min-width: 200px;
  border-radius: 0;
  padding: 8px 0;
  box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
  z-index: 1000;
  top: 100%;
  left: 0;
  border: 1px solid rgba(255, 255, 255, 0.1);
}

.dropdown a {
  display: block;
  padding: 12px 20px;
  color: #fff;
  text-decoration: none;
  font-family: 'Bodoni Moda', serif;
  font-size: 0.9rem;
  transition: all 0.3s ease;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

.dropdown a:last-child {
  border-bottom: none;
}

.dropdown a:hover {
  background-color: rgba(0, 188, 212, 0.2);
  padding-left: 25px;
  color: #00bcd4;
}

/* Arrow at the top of dropdown */
.dropdown::before {
  content: "";
  position: absolute;
  top: -8px;
  left: 20px;
  border-left: 8px solid transparent;
  border-right: 8px solid transparent;
  border-bottom: 8px solid rgba(0, 0, 0, 0.9);
}

.container {
  max-width: 1000px;
  margin: 130px auto;
  background-color: rgba(9, 38, 58, 0.57);
```

```
border-radius: 15px;
box-shadow: 0 10px 25px rgba(139, 213, 250, 0.2);
padding: 25px;
font-family: 'Bodoni Moda', serif;
}

h1 {
  text-align: center;
  color:rgb(87, 222, 246); /* Deep purple */
  margin-bottom: 25px;
  font-weight: 600;
}

.calendar-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 25px;
  padding-bottom: 15px;
  border-bottom: 2px solid rgb(102, 224, 255); /* Light purple border */
}

.calendar-nav {
  display: flex;
  gap: 12px;
}

button {
  background-color:rgb(112, 190, 219); /* Medium purple */
  color: white;
  border: none;
  padding: 10px 18px;
  border-radius: 25px;
  cursor: pointer;
  transition: all 0.3s;
  font-weight: 500;
  box-shadow: 0 4px 8px rgba(112, 201, 219, 0.2);
}

button:hover {
  background-color:rgb(91, 176, 213); /* Darker purple on hover */
  transform: translateY(-2px);
}

button:active {
  transform: translateY(0);
}

#addEventBtn {
  background-color:rgb(133, 231, 255); /* Soft pink */
  box-shadow: 0 4px 8px rgba(255, 133, 162, 0.3);
```

```
}

#addEventBtn:hover {
  background-color:rgb(48, 210, 228); /* Deeper pink on hover */
}

.month-year {
  font-size: 1.8rem;
  font-weight: bold;
  color:rgb(96, 212, 235); /* Deep purple */
  text-shadow: 1px 1px 2px rgba(50, 153, 160, 0.1);
}

.calendar {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  gap: 8px;
  margin-bottom: 20px;
}

.day-header {
  text-align: center;
  font-weight: bold;
  padding: 12px;
  background-color:rgb(224, 249, 255); /* Very light purple */
  border-radius: 8px;
  color:rgb(47, 187, 202); /* Deep purple */
}

.day {
  min-height: 110px;
  border: 1px solid rgb(164, 240, 255); /* Light purple border */
  padding: 10px;
  background-color: white;
  position: relative;
  cursor: pointer;
  border-radius: 10px;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
  overflow: visible; /* Changed from hidden to visible */
}

.day:hover {
  transform: scale(1.02);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.1);
  z-index: 10; /* Ensure the expanded day appears above others */
}

.day-number {
  font-weight: bold;
  margin-bottom: 8px;
  color:rgb(75, 192, 212); /* Deep purple */
}
```

```
    font-size: 1.1rem;
    text-align: right;
}

.day.inactive {
    background-color:rgb(225, 247, 251); /* Even lighter purple for inactive days */
    color:rgb(182, 224, 226); /* Medium-light purple */
    box-shadow: none;
}

.day.inactive:hover {
    transform: none;
    box-shadow: none;
}

.event {
    background: linear-gradient(to right,rgb(133, 233, 255),rgb(167, 230, 255)); /* Pink
gradient */
    color: white;
    padding: 6px 10px;
    border-radius: 20px;
    margin-bottom: 5px;
    font-size: 0.85rem;
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
    box-shadow: 0 2px 4px rgba(133, 249, 255, 0.25);
    transition: all 0.3s ease;
    transform-origin: top;
}

.event:hover {
    transform: translateX(3px);
}

.modal {
    display: none;
    position: fixed;
    z-index: 1;
    left: 50%; /* Center horizontally */
    top: 60%; /* Adjusted to move the modal down */
    transform: translate(-50%, -50%); /* Adjust position to truly center */
    width: 100%;
    height: 100%;
    overflow: auto;
    background-color: rgba(13, 12, 12, 0.53); /* Purple tinted overlay */
    backdrop-filter: blur(3px);
}

.modal-content {
    background: rgb(200, 233, 236);
```

```
border-radius: 15px;
padding: 20px;
width: 90%;
max-width: 500px;
box-shadow: 0 5px 15px rgba(0, 0, 0, 0.2);
font-family: 'Bodoni Moda', serif; /* Ensure font is set */
position: absolute; /* Change to absolute positioning */
left: 50%; /* Center horizontally */
top: 50%; /* Center vertically */
transform: translate(-50%, -50%); /* Adjust position to truly center */
}

.modal-header {
background: linear-gradient(135deg, #00bcd4, #007c91);
color: white;
padding: 20px;
border-radius: 15px 15px 0 0;
display: flex;
justify-content: space-between;
align-items: center;
}

.modal-header h2 {
margin: 0;
font-size: 1.5rem;
color: white;
}

.modal-body {
padding: 25px;
}

.event-details {
display: flex;
flex-direction: column;
gap: 20px;
}

.detail-row {
display: flex;
align-items: center;
gap: 15px;
padding: 10px;
background: rgba(0, 0, 0, 0.05);
border-radius: 8px;
}

.detail-row i {
color: #00bcd4;
font-size: 1.2rem;
width: 24px;
```

```
}

.detail-label {
  font-weight: 600;
  color: black;
  min-width: 100px;
}

.detail-value {
  color: black;
  flex: 1;
}

.reminder-section {
  margin-top: 20px;
  padding-top: 20px;
  border-top: 1px solid #eee;
}

.toggle-container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px;
  background: rgba(0, 188, 212, 0.05);
  border-radius: 8px;
}

.toggle-label {
  font-weight: 600;
  color: #333;
}

.toggle-switch-wrapper {
  position: relative;
  width: 60px;
  height: 34px;
  user-select: none;
}

.toggle-switch {
  opacity: 0;
  width: 0;
  height: 0;
}

.toggle-slider {
  position: absolute;
  cursor: pointer;
  top: 0;
  left: 0;
```

```
    right: 0;
    bottom: 0;
    background-color: #ccc;
    transition: background-color 0.3s ease;
    border-radius: 34px;
}

.toggle-slider:before {
    position: absolute;
    content: "";
    height: 26px;
    width: 26px;
    left: 4px;
    bottom: 4px;
    background-color: white;
    transition: transform 0.3s ease;
    border-radius: 50%;
}

.toggle-switch:checked + .toggle-slider {
    background-color: #00bcd4;
}

.toggle-switch:checked + .toggle-slider:before {
    transform: translateX(26px);
}

.toggle-switch:disabled + .toggle-slider {
    opacity: 0.5;
    cursor: not-allowed;
}

.event {
    padding: 8px 12px;
    margin-bottom: 5px;
    border-radius: 50px;
    font-size: 0.9rem;
    cursor: pointer;
    transition: transform 0.2s;
}

.booking-event {
    background:rgb(136, 226, 158);
    color: white;
}

.special-event {
    background:rgb(209, 174, 231);
    color: white;
}
```

```
/* Allow header to be clickable */
.header {
  z-index: 10000; /* Ensure header is above modal */
}

/* Only prevent scrolling for content below header */
body {
  overflow: auto; /* Allow scrolling */
}

.header {
  position: fixed;
  width: 100%;
  top: 0;
  left: 0;
}

.container {
  margin-top: 150px; /* Adjust based on your header height */
  overflow: auto; /* Allow scrolling */
}

/* Add CSS styles for the close icon */
.modal-close-icon {
  font-size: 44px; /* Increase font size */
  color:rgb(255, 255, 255); /* Color of the icon */
  cursor: pointer; /* Pointer cursor */
  transition: color 0.3s; /* Transition effect */
}

.modal-close-icon:hover {
  color:rgb(26, 228, 255); /* Darker color on hover */
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(-20px); }
  to { opacity: 1; transform: translateY(0); }
}

@keyframes fadeOut {
  from { opacity: 1; transform: translateY(0); }
  to { opacity: 0; transform: translateY(-20px); }
}

.month-display {
  text-align: center; /* Center the text horizontally */
  margin: 20px 0; /* Optional: Add some vertical spacing */
}

.show-more {
  font-size: 0.8rem; /* Adjust size as needed */
  margin-top: 5px;
```

```

        transition: opacity 0.3s ease;
    }

    .show-more:hover {
        color: #ff4081; /* Change color on hover */
    }
</style>
</head>
<body>
    <header class="header">
        <div class="logo">
            <a href="user_home.php" class="are"><h2 style="color: white;">ArenaX</h2></a>
        </div>
        <nav>
            <ul style="display: flex; justify-content: center; width: 100%;">
                <li><a href="user_home.php">Home</a></li>
                <li><a href="user_indoor.php">Indoor</a></li>
                <li><a href="user_outdoor.php">Outdoor</a></li>
                <li><a href="user_fitness.php">Fitness</a></li>
                <li><a href="user_events.php">Events</a></li>
            </ul>
        </nav>
        <div style="margin-right: 20px; position: relative;">
            <button class="log"><?php echo htmlspecialchars($userData['name']); ?> <i class="fas fa-
            caret-down"></i></button>
            <div class="dropdown">
                <a href="user_profile.php">PROFILE</a>
                <a href="user_bookings.php">BOOKINGS</a>
                <a href="user_calendar.php">CALENDAR</a>
                <a href="user_payment_history.php">PAYMENT HISTORY</a>
                <a href="logout.php">LOGOUT</a>
            </div>
        </div>
    </header>
    <div class="container">
        <h1>My Calendar</h1>

        <?php if ($isPremiumMembership): ?>
            <div class="calendar-header">
                <div class="calendar-nav">
                    <button id="prevMonth">← Previous</button>
                    <button id="nextMonth">Next →</button>
                </div>
                <div class="month-year" id="monthYear"></div>
                <?php
                // <button id="addEventBtn">+ Add Event</button>
                ?>
            </div>

            <div class="calendar" id="calendar">
                <!-- Calendar will be generated with JavaScript -->

```

```

    </div>
    <?php else: ?>
    <!-- Upgrade Modal for Normal and Standard Membership Users -->
    <div id="upgradeModal" class="modal" style="display: block;">
        <div class="modal-content" style="font-family: 'Bodoni Moda', serif; padding: 20px;">
            <h2 style="text-align: center; margin-bottom: 20px; color:
#00bcd4;">MEMBERSHIP REQUIRED</h2>
            <hr style="border: 1px solid white; margin: 10px 0;">
            <div style="text-align: center; margin-bottom: 30px;">
                <i class="fas fa-lock" style="font-size: 60px; color: #00bcd4; margin-bottom:
20px;"></i>
                <p style="font-size: 1.2rem; margin-bottom: 20px;">Access to the calendar feature
requires a premium membership.</p>
                <p style="font-size: 1.1rem; margin-bottom: 30px;">Your current membership:
<strong><?php echo strtoupper($membership_type); ?></strong>. This feature requires
PREMIUM membership.</p>
            </div>
            <div style="text-align: center;">
                <button style="background-color: #00bcd4; color: white; border: none; padding:
12px 30px; font-size: 1.1rem; border-radius: 5px; cursor: pointer; transition: all 0.3s ease; font-
family: 'Bodoni Moda', serif; margin-top: 20px;"
onclick="window.location.href='user_home.php?showMembershipModal=true'">Upgrade
Membership</button>
            </div>
        </div>
    </div>
    <?php endif: ?>
</div>

<!-- Event Modal -->
<div id="eventModal" class="modal">
    <div class="modal-content">
        <span class="modal-close-icon" onclick="closeModal()">&times;</span>
        <h2 id="modalTitle">Add Event</h2>
        <form id="eventForm">
            <input type="hidden" id="eventId" value="">
            <input type="hidden" id="eventDate" value="">

            <div class="form-group">
                <label for="eventTitle">Event Title:</label>
                <input type="text" id="eventTitle" name="eventTitle" required placeholder="Enter
event title...">
            </div>

            <div class="form-group">
                <label for="eventTime">Time:</label>
                <input type="time" id="eventTime" name="eventTime">
            </div>

            <div class="form-group">
                <label for="eventDescription">Description:</label>

```

```

        <textarea id="eventDescription" name="eventDescription" placeholder="Add details
about your event..."></textarea>
    </div>

    <div class="form-actions">
        <button type="button" id="deleteEvent">Delete</button>
        <button type="submit" id="saveEvent">Save</button>
    </div>
</form>
</div>
</div>

<script>
    document.addEventListener('DOMContentLoaded', function() {
        // Calendar state
        let currentDate = new Date();
        let events = <?php echo $calendarEventsJson; ?> || [];

        // DOM elements
        const calendar = document.getElementById('calendar');
        const monthYear = document.getElementById('monthYear');
        const prevMonthBtn = document.getElementById('prevMonth');
        const nextMonthBtn = document.getElementById('nextMonth');
        const addEventBtn = document.getElementById('addEventBtn');
        const eventModal = document.getElementById('eventModal');
        const closeModal = document.querySelector('.modal-close-icon');
        const eventForm = document.getElementById('eventForm');
        const modalTitle = document.getElementById('modalTitle');
        const eventIdInput = document.getElementById('eventId');
        const eventDateInput = document.getElementById('eventDate');
        const eventTitleInput = document.getElementById('eventTitle');
        const eventTimeInput = document.getElementById('eventTime');
        const eventDescriptionInput = document.getElementById('eventDescription');
        const saveEventBtn = document.getElementById('saveEvent');
        const deleteEventBtn = document.getElementById('deleteEvent');

        // Add reminder checkbox to the form
        const reminderDiv = document.createElement('div');
        reminderDiv.className = 'form-group';
        reminderDiv.innerHTML = `
            <label for="eventReminder">
                <input type="checkbox" id="eventReminder" name="eventReminder">
                Enable reminder for this event
            </label>
        `;

        // Find the right position to insert the reminder checkbox
        const formActions = eventForm.querySelector('.form-actions');
        eventForm.insertBefore(reminderDiv, formActions);

        // Initialize calendar

```

```
renderCalendar();

// Event listeners
prevMonthBtn.addEventListener('click', () => {
  currentDate.setMonth(currentDate.getMonth() - 1);
  renderCalendar();
});

nextMonthBtn.addEventListener('click', () => {
  currentDate.setMonth(currentDate.getMonth() + 1);
  renderCalendar();
});

addEventBtn.addEventListener('click', () => {
  openModal();
});

closeModal.addEventListener('click', () => {
  eventModal.style.display = 'none';
});

window.addEventListener('click', (event) => {
  if (event.target === eventModal) {
    eventModal.style.display = 'none';
  }
});

eventForm.addEventListener('submit', (e) => {
  e.preventDefault();
  saveEvent();
});

deleteEventBtn.addEventListener('click', () => {
  deleteEvent();
});

// Functions
function renderCalendar() {
  // Clear calendar
  calendar.innerHTML = "";

  // Update month and year display
  const monthNames = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
'September', 'October', 'November', 'December'];
  monthYear.textContent = `${monthNames[currentDate.getMonth()]}
${currentDate.getFullYear()}`;

  // Add day headers
  const dayNames = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];
  dayNames.forEach(day => {
    const dayHeader = document.createElement('div');
```

```
        dayHeader.className = 'day-header';
        dayHeader.textContent = day;
        calendar.appendChild(dayHeader);
    });

    // Get first day of month and number of days
    const firstDay = new Date(currentDate.getFullYear(), currentDate.getMonth(), 1);
    const lastDay = new Date(currentDate.getFullYear(), currentDate.getMonth() + 1, 0);
    const daysInMonth = lastDay.getDate();
    const startingDayOfWeek = firstDay.getDay();

    // Add blank cells for days before start of month
    for (let i = 0; i < startingDayOfWeek; i++) {
        const blankDay = document.createElement('div');
        blankDay.className = 'day inactive';
        calendar.appendChild(blankDay);
    }

    // Get today's date for highlighting current day
    const today = new Date();
    const isCurrentMonth = today.getMonth() === currentDate.getMonth() &&
today.getFullYear() === currentDate.getFullYear();

    // Add days of the month
    for (let i = 1; i <= daysInMonth; i++) {
        const dayCell = document.createElement('div');

        // Add "today" class if this is the current day
        if (isCurrentMonth && i === today.getDate()) {
            dayCell.className = 'day today';
        } else {
            dayCell.className = 'day';
        }

        const dayNumber = document.createElement('div');
        dayNumber.className = 'day-number';
        dayNumber.textContent = i;
        dayCell.appendChild(dayNumber);

        // Format date string for comparison with events
        const dateStr = formatDate(new Date(currentDate.getFullYear(),
currentDate.getMonth(), i));
        dayCell.dataset.date = dateStr;

        // Add click event to open modal for this day
        dayCell.addEventListener('click', () => {
            openModal(null, dateStr);
        });

        calendar.appendChild(dayCell);
    }
}
```

```
// Add events to calendar
displayEvents();
}

function displayEvents() {
  // Clear existing events from calendar
  document.querySelectorAll('.event').forEach(el => el.remove());

  // Add events to corresponding days
  events.forEach(event => {
    const dayCell = document.querySelector(`.day[data-date="${event.date}"]`);
    if (dayCell) {
      const eventDiv = document.createElement('div');
      eventDiv.className = 'event';

      // Set different class based on event type
      if (event.event_type === 'booking') {
        eventDiv.classList.add('booking-event');
      } else if (event.event_type === 'event') {
        eventDiv.classList.add('special-event');
      }

      eventDiv.textContent = event.title;
      eventDiv.title = event.description || event.title;

      // Add smooth transition for events
      eventDiv.style.transition = 'all 0.3s ease';
      eventDiv.style.opacity = '1';
      eventDiv.style.transform = 'translateY(0)';

      eventDiv.addEventListener('click', (e) => {
        e.stopPropagation();
        openModal(event);
      });

      dayCell.appendChild(eventDiv);
    }
  });

  // Handle events display limit
  const dayCells = document.querySelectorAll('.day');
  dayCells.forEach(dayCell => {
    const eventsInCell = dayCell.querySelectorAll('.event');
    if (eventsInCell.length > 2) {
      // Initially hide all but the first two events
      for (let i = 2; i < eventsInCell.length; i++) {
        eventsInCell[i].style.opacity = '0';
        eventsInCell[i].style.transform = 'translateY(-10px)';
        eventsInCell[i].style.display = 'none';
      }
    }
  });
}
```

```

// Create a "Show more" element
const showMore = document.createElement('div');
showMore.className = 'show-more';
showMore.textContent = `+${eventsInCell.length - 2} more`;
showMore.style.color = '#00bcd4';
showMore.style.textAlign = 'center';
showMore.style.marginTop = '5px';
showMore.style.transition = 'opacity 0.3s ease';
dayCell.appendChild(showMore);

// Add hover effect to the entire day cell
dayCell.addEventListener('mouseenter', () => {
  showMore.style.opacity = '0';
  eventsInCell.forEach((event, index) => {
    if (index >= 2) {
      event.style.display = 'block';
      // Delay the appearance slightly for each event
      setTimeout(() => {
        event.style.opacity = '1';
        event.style.transform = 'translateY(0)';
      }, (index - 2) * 100);
    }
  });
});

dayCell.addEventListener('mouseleave', () => {
  showMore.style.opacity = '1';
  for (let i = 2; i < eventsInCell.length; i++) {
    eventsInCell[i].style.opacity = '0';
    eventsInCell[i].style.transform = 'translateY(-10px)';
    // Hide after transition completes
    setTimeout(() => {
      eventsInCell[i].style.display = 'none';
    }, 300);
  }
});
}
});
}

function toggleReminder(event) {
  // Ensure we're working with numbers for reminder_enabled
  const currentState = parseInt(event.reminder_enabled);
  const newReminderState = currentState === 1 ? 0 : 1;

  // Update through AJAX
  fetch('toggle_reminder.php', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    }
  })

```

```
    },
    body: JSON.stringify({
      id: event.id,
      reminder_enabled: newReminderState
    })
  })
).then(response => response.json())
.then(data => {
  if (data.success) {
    // Update local event state in the events array
    const eventIndex = events.findIndex(e => e.id === event.id);
    if (eventIndex !== -1) {
      events[eventIndex].reminder_enabled = newReminderState;
    }

    // Update the current event object
    event.reminder_enabled = newReminderState;

    // Re-render the modal content to reflect the new state
    openModal(event);

    // Show message
    const message = newReminderState === 1
      ? 'Reminder set successfully!'
      : 'Reminder disabled.';

    // Optional: show a less intrusive notification instead of alert
    const notification = document.createElement('div');
    notification.style.cssText = `
      position: fixed;
      top: 20px;
      right: 20px;
      background-color: #00bcd4;
      color: white;
      padding: 15px 25px;
      border-radius: 5px;
      z-index: 1000;
      animation: fadeIn 0.3s, fadeOut 0.3s 2.7s;
    `;
    notification.textContent = message;
    document.body.appendChild(notification);

    setTimeout(() => {
      notification.remove();
    }, 3000);
  } else {
    alert('Error updating reminder: ' + data.message);
  }
})
).catch(error => {
  console.error('Error:', error);
});
```

```

        alert('Error updating reminder. Please try again.');
```

```

    });
}
```

```

function openModal(event = null, dateStr = null) {
    if (event) {
        // Get the current state of the event from the events array
        const currentEvent = events.find(e => e.id == event.id) || event;

        const modalContent = `
            <div class="modal-header">
                <h2>${currentEvent.event_type === 'booking' ? 'Booking Details' : 'Event
Details'}</h2>
                <span class="modal-close-icon" onclick="closeModal()">&times;</span>
            </div>
            <div class="modal-body">
                ${generateEventDetails(currentEvent)}
            </div>
        `;

        document.querySelector('.modal-content').innerHTML = modalContent;
        eventModal.style.display = 'block';
    } else {
        // Handle new event creation...
    }
}
```

```

function generateEventDetails(event) {
    // Ensure reminder_enabled is properly parsed as a number
    const reminderEnabled = Number(event.reminder_enabled);

    return `
        <div class="event-details">
            <div class="detail-row">
                <i class="fas fa-calendar-day"></i>
                <span class="detail-label">Event:</span>
                <span class="detail-value">${event.title}</span>
            </div>
            <div class="detail-row">
                <i class="far fa-calendar"></i>
                <span class="detail-label">Date:</span>
                <span class="detail-value">${formatDateForDisplay(event.date)}</span>
            </div>
            <div class="detail-row">
                <i class="far fa-clock"></i>
                <span class="detail-label">Time:</span>
                <span class="detail-value">${formatTime(event.time)}</span>
            </div>
            <div class="detail-row">
                <i class="fas fa-info-circle"></i>
                <span class="detail-label">Description:</span>
            </div>
        `;
}
```

```

        <span class="detail-value">${event.description || 'No description
available'}</span>
    </div>
    <div class="reminder-section">
        <label class="toggle-container">
            <span class="toggle-label">Reminder</span>
            <div class="toggle-switch-wrapper">
                <input type="checkbox"
                    id="reminderToggle_${event.id}"
                    class="toggle-switch"
                    ${reminderEnabled === 1 ? 'checked' : ''}
                    onchange="handleReminderToggle(${event.id})"
                    data-event-id="${event.id}">
                <span class="toggle-slider"></span>
            </div>
        </label>
    </div>
</div>
`;
}

```

```

function handleReminderToggle(eventId) {
    const toggleElement = document.getElementById(`reminderToggle_${eventId}`);
    const newReminderState = toggleElement.checked ? 1 : 0;

    // Disable the toggle while the request is being processed
    toggleElement.disabled = true;

    fetch('toggle_reminder.php', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({
            id: eventId,
            reminder_enabled: newReminderState
        })
    })
    .then(response => response.json())
    .then(data => {
        if (data.success) {
            // Update the events array
            const eventIndex = events.findIndex(e => e.id == eventId);
            if (eventIndex !== -1) {
                events[eventIndex].reminder_enabled = newReminderState;
            }

            // Ensure the toggle reflects the correct state
            toggleElement.checked = newReminderState === 1;

            showNotification(

```

```

        newReminderState === 1
        ? 'Email reminder set for 1 day before the event!'
        : 'Reminder disabled'
    );
} else {
    // Revert the checkbox if there was an error
    toggleElement.checked = !toggleElement.checked;
    showNotification('Error updating reminder', 'error');
}
})
.catch(error => {
    console.error('Error:', error);
    toggleElement.checked = !toggleElement.checked;
    showNotification('Error updating reminder', 'error');
})
.finally(() => {
    // Re-enable the toggle after the request completes
    toggleElement.disabled = false;
});
}

function showNotification(message, type = 'success') {
    const notification = document.createElement('div');
    notification.style.cssText = `
        position: fixed;
        top: 20px;
        right: 20px;
        padding: 15px 25px;
        border-radius: 5px;
        z-index: 1000;
        animation: fadeIn 0.3s, fadeOut 0.3s 2.7s;
        color: white;
        background-color: ${type === 'success' ? '#00bcd4' : '#ff4444'};
    `;
    notification.textContent = message;
    document.body.appendChild(notification);

    setTimeout(() => {
        notification.remove();
    }, 3000);
}

function saveEvent() {
    const reminderCheckbox = document.getElementById('eventReminder');

    const eventData = {
        id: eventIdInput.value || null,
        date: eventDateInput.value,
        title: eventTitleInput.value,
        time: eventTimeInput.value,
        description: eventDescriptionInput.value,
    };

```

```
        reminder_enabled: reminderCheckbox.checked ? 1 : 0
    };

    // Use AJAX to save event to PHP backend
    fetch('save_event.php', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(eventData)
    })
    .then(response => response.json())
    .then(data => {
        if (data.success) {
            // If successful, update local events
            if (eventData.id) {
                // Update existing event
                const index = events.findIndex(e => e.id == eventData.id);
                if (index !== -1) {
                    events[index] = { ...events[index], ...eventData };
                }
            } else {
                // Add new event
                events.push({
                    ...eventData,
                    id: data.id,
                    event_type: 'custom'
                });
            }

            // Close modal and redisplay events
            eventModal.style.display = 'none';
            displayEvents();
        } else {
            alert('Error saving event: ' + data.message);
        }
    })
    .catch(error => {
        console.error('Error:', error);
        alert('Error saving event. Please try again.');
```

```
    });
}

function deleteEvent() {
    const eventId = eventIdInput.value;

    if (confirm('Are you sure you want to delete this event?')) {
        fetch('delete_event.php', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
```

```

        },
        body: JSON.stringify({ id: eventId })
    })
    .then(response => response.json())
    .then(data => {
        if (data.success) {
            // If successful, remove from local events
            events = events.filter(e => e.id !== eventId);

            // Close modal and redisplay events
            eventModal.style.display = 'none';
            displayEvents();
        } else {
            alert('Error deleting event: ' + data.message);
        }
    })
    .catch(error => {
        console.error('Error:', error);
        alert('Error deleting event. Please try again.');
```

```

    });
}
}

function formatDate(date) {
    const year = date.getFullYear();
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const day = String(date.getDate()).padStart(2, '0');
    return `${year}-${month}-${day}`;
}

function formatDateForDisplay(dateStr) {
    const date = new Date(dateStr);
    const monthNames = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
'September', 'October', 'November', 'December'];
    const month = monthNames[date.getMonth()];
    const day = date.getDate();
    return `${month} ${day}`;
}

function formatTime(timeStr) {
    if (timeStr) {
        const time = new Date('1970-01-01T' + timeStr);
        const hours = time.getHours();
        const minutes = time.getMinutes();
        return `${hours}:${minutes.toString().padStart(2, '0')}`;
    }
    return 'All day';
}
});
```

// Add this after your existing script

```

function closeModal() {
    document.getElementById('eventModal').style.display = 'none';
}
</script>
<script>
    const header = document.querySelector('.header');
    window.addEventListener('scroll', () => {
        if (window.scrollY > 50) {
            header.classList.add('scrolled');
        } else {
            header.classList.remove('scrolled');
        }
    });
</script>
<footer>
    <div class="footer-container">
        <div class="footer-column">
            <h3>ArenaX</h3>
            <p>Your premier destination for sports and fitness. Explore a variety of activities and
join our vibrant community.</p>
        </div>
        <div class="footer-column">
            <h3>Quick Links</h3>
            <ul>
                <li><a href="homepage.php">Home</a></li>
                <li><a href="#about">About Us</a></li>
                <li><a href="indoor.php">Indoor Activities</a></li>
                <li><a href="outdoor.php">Outdoor Activities</a></li>
                <li><a href="homepage.php#membership">Membership</a></li>
            </ul>
        </div>
        <div class="footer-column">
            <h3>Contact Us</h3>
            <p>Email: arenax@gmail.com</p>
            <p>Phone: 9544147855</p>
            <p>Address: 123 ArenaX Avenue, Sportstown</p>
        </div>
        <div class="footer-column">
            <h3>Follow Us</h3>
            <div class="social-links">
                <a href="#"><i class="fab fa-facebook-f"></i></a>
                <a href="#"><i class="fab fa-twitter"></i></a>
                <a href="#"><i class="fab fa-instagram"></i></a>
                <a href="#"><i class="fab fa-linkedin"></i></a>
            </div>
        </div>
    </div>
    <div class="footer-bottom">
        <p>&copy; 2025 ArenaX. All rights reserved.</p>
    </div>
</footer>

```

```
<script>
function scrollToAbout() {
  const aboutSection = document.querySelector('.about-section');
  const headerHeight = document.querySelector('.header').offsetHeight;
  const elementPosition = aboutSection.getBoundingClientRect().top;
  const offsetPosition = elementPosition + window.pageYOffset - headerHeight;

  window.scrollTo({
    top: offsetPosition,
    behavior: "smooth"
  });
}

const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.classList.add('visible');
    }
  });
}, {
  threshold: 0.2 // Trigger when 20% of the element is visible
});

// Observe both the content and image
document.querySelectorAll('.about-content, .about-image').forEach(element => {
  observer.observe(element);
});

// Profile dropdown functionality
const profileButton = document.querySelector('.log');
const dropdown = document.querySelector('.dropdown');
dropdown.style.display = 'none'; // Ensure dropdown is hidden initially

profileButton.addEventListener('click', () => {
  if (dropdown.style.display === 'none' || dropdown.style.display === "") {
    dropdown.style.display = 'block';
  } else {
    dropdown.style.display = 'none';
  }
});

// Handle clicking outside dropdown
document.addEventListener('click', function(event) {
  const dropdown = document.querySelector('.dropdown');
  const profileButton = document.querySelector('.log');

  if (!dropdown.contains(event.target) && !profileButton.contains(event.target)) {
    dropdown.style.display = 'none';
  }
});
});
```

```
// Prevent the dropdown from closing when clicking inside it
dropdown.addEventListener('click', function(event) {
    event.stopPropagation();
});

</script>
</body>
</html>
```

User_payment_history.php

```
<?php
session_start();
include('db_connect.php');

// Check if user is logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: signin.php');
    exit();
}

// Fetch user details with membership information
$user_id = $_SESSION['user_id'];
$user_email = $_SESSION['email']; // Ensure this is set from the session

try {
    $query = "SELECT u.*, m.membership_type
              FROM users u
              LEFT JOIN memberships m ON u.membership_id = m.membership_id
              WHERE u.user_id = :user_id";
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':user_id', $user_id, PDO::PARAM_INT);
    $stmt->execute();

    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$user) {
        // Handle case where user is not found
        $_SESSION['error'] = "User not found";
        header('Location: logout.php');
        exit();
    }
} catch(PDOException $e) {
    // Log error and show user-friendly message
    error_log("Database error: " . $e->getMessage());
    $_SESSION['error'] = "An error occurred while fetching user data";
    header('Location: error.php');
```

```
        exit();
    }

    // Modify the existing SQL query to also fetch the user's name
    $sql = "SELECT membership_id, name FROM users WHERE email = :email"; // Use a prepared
    statement
    $stmt = $conn->prepare($sql);
    $stmt->bindParam(':email', $user_email, PDO::PARAM_STR);
    $stmt->execute();

    if ($stmt->rowCount() > 0) { // Use rowCount() instead of num_rows
        $row = $stmt->fetch(PDO::FETCH_ASSOC);
        $membership_id = $row['membership_id'];
        $user_name = $row['name'];
    } else {
        $user_name = "Profile"; // Default value
    }
    error_reporting(E_ALL);
    ini_set('display_errors', 1);
    ini_set('display_startup_errors', 1);
    error_log("Starting profile update process");

    // Function to send error response
    if (!function_exists('sendError')) {
        function sendError($message, $details = null) {
            $response = ['success' => false, 'message' => $message];
            if ($details && ini_get('display_errors')) {
                $response['debug'] = $details;
            }
            echo json_encode($response);
            exit;
        }
    }

    $user_id = $_SESSION['user_id'];

    // Fetch membership payment history
    $sql = "SELECT p.payment_id, p.amount, p.payment_date, p.payment_time,
        m.membership_type, mr.membership_reg_date, mr.bill
        FROM payment p
        INNER JOIN membership_reg mr ON p.membership_reg_id = mr.membership_reg_id
        INNER JOIN memberships m ON mr.membership_id = m.membership_id
        WHERE p.user_id = ? AND p.membership_reg_id IS NOT NULL
        ORDER BY p.payment_date DESC, p.payment_time DESC";

    $stmt = $conn->prepare($sql);
    $stmt->bindValue(1, $user_id, PDO::PARAM_INT);
    $stmt->execute();
    $result = $stmt->fetchAll(PDO::FETCH_ASSOC);

    // Add this near the top of your file, after session_start()
```

```

$billsDir = __DIR__ . '/bills';
if (!file_exists($billsDir)) {
    mkdir($billsDir, 0777, true);
}

// Add this to allow PDF file downloads
if (isset($_GET['bill']) && !empty($_GET['bill'])) {
    $billFile = $billsDir . '/' . basename($_GET['bill']);
    if (file_exists($billFile) && pathinfo($billFile, PATHINFO_EXTENSION) === 'pdf') {
        header('Content-Type: application/pdf');
        header('Content-Disposition: attachment; filename="' . basename($billFile) . '"');
        readfile($billFile);
        exit;
    } else {
        // File not found error handling
        $_SESSION['error'] = "The requested bill file was not found.";
        header('Location: error.php'); // Redirect to an error page
        exit;
    }
}

?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Membership Payment History</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">

    <!-- Favicon link -->
    <link rel="icon" href="img/logo3.png" type="image/png">

    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/5.15.4/css/all.min.css">

    <style>
        @import url('https://fonts.googleapis.com/css2?family=Aboreto&display=swap');
        @import
url('https://fonts.googleapis.com/css2?family=Cinzel+Decorative:wght@400&family=Goldman&
display=swap');
        @import
url('https://fonts.googleapis.com/css2?family=Cinzel+Decorative:wght@400&display=swap');
        @import
url('https://fonts.googleapis.com/css2?family=Bodoni+Moda:wght@400;700&display=swap');

        * {
            margin: 0;
            padding: 0;

```

```
        box-sizing: border-box;
    }

    body {
        font-family: 'Cinzel Decorative', cursive;
        background: linear-gradient(rgba(0, 0, 0, 0.7), rgba(0, 0, 0, 0.7)), url('img/event1.jpg') no-
repeat center center fixed;
        background-size: cover;
        color: white;
        overflow-x: hidden;
    }

    /* Header styles from user_home.php */
    .header {
        position: fixed;
        top: 0;
        left: 0;
        width: 100%;
        background: rgba(0, 0, 0, 0.8);
        padding: 2rem 2rem;
        display: flex;
        justify-content: space-between;
        align-items: center;
        z-index: 1000;
    }

    .header nav ul {
        list-style: none;
        display: flex;
        gap: 2rem;
        justify-content: center;
        margin: 0 auto;
        padding: 0;
    }

    .header div a button {
        transition: background-color 0.3s ease-in-out, transform 0.2s ease;
    }

    .header div a button:hover {
        color: #00bcd4;
        border-color: #00bcd4;
        transform: scale(1.1);
    }

    .log {
        padding: 10px 20px;
        font-size: 1rem;
        font-family: 'Cinzel Decorative', cursive;
        background-color: #007cd400;
        color: white;
    }
```

```
padding: 10px 50px;
border-style: solid;
border-width: 1px;
border-color: white;
border-radius: 0px;
cursor: pointer;
transition: background-color 0.3s ease-in-out;
}

.log:hover {
  color: #00bcd4;
  border-color: #00bcd4;
}

.header div {
  display: flex;
  gap: 15px;
  margin-right: 40px;
}

.header nav ul li {
  position: relative;
}

.header nav ul li a {
  text-decoration: none;
  color: white;
  font-size: 1.2rem;
  transition: color 0.2s ease-in-out;
}

.header nav ul li a::after {
  content: "";
  position: absolute;
  bottom: -4px;
  left: 50%;
  width: 0;
  height: 2px;
  background: #00bcd4;
  transition: all 0.3s ease-in-out;
  transform: translateX(-50%);
}

.header nav ul li a:hover::after {
  width: 100%;
}

.header nav ul li a:hover {
  color: #00bcd4;
}
```

```
.are {
  text-decoration: none;
}

.dropdown {
  display: none;
  position: absolute;
  background-color: rgba(0, 0, 0, 0.9);
  min-width: 200px;
  border-radius: 0;
  padding: 8px 0;
  box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
  z-index: 1000;
  top: 100%;
  left: 0;
  border: 1px solid rgba(255, 255, 255, 0.1);
}

.dropdown a {
  display: block;
  padding: 12px 20px;
  color: #fff;
  text-decoration: none;
  font-family: 'Bodoni Moda', serif;
  font-size: 0.9rem;
  transition: all 0.3s ease;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

.dropdown a:last-child {
  border-bottom: none;
}

.dropdown a:hover {
  background-color: rgba(0, 188, 212, 0.2);
  padding-left: 25px;
  color: #00bcd4;
}

.dropdown::before {
  content: "";
  position: absolute;
  top: -8px;
  left: 20px;
  border-left: 8px solid transparent;
  border-right: 8px solid transparent;
  border-bottom: 8px solid rgba(0, 0, 0, 0.9);
}

.payment-history-container {
  max-width: 1000px;
  margin: 40px auto;
}
```

```
padding: 20px;
background-color: rgba(199, 143, 94, 0.5); /* Semi-transparent background for readability
*/
border-radius: 10px; /* Rounded corners */
box-shadow: 0 0 15px rgba(0,0,0,0.2); /* Shadow effect */
margin-top: 150px; /* Added margin to create space below the header */
font-family: 'Bodoni Moda', serif; /* Set font to Bodoni for all content in this container */
}
.payment-card {
margin-bottom: 20px;
border: 1px solid rgb(0, 0, 0);
border-radius: 10px;
padding: 15px;
background-color: rgb(165, 150, 136);
box-shadow: 0 0 10px rgba(0,0,0,0.1);
}

.payment-card p {
margin: 10px 0; /* Added margin for spacing between details */
}

.status-badge {
padding: 8px 12px;
border-radius: 20px;
font-size: 0.9em;
background-color: rgb(195, 144, 96); /* Changed background color to yellow */
}

.no-payments {
text-align: center;
padding: 40px;
color: #6c757d;
}

.bill-box {
display: flex;
align-items: center;
background-color: rgba(255, 255, 255, 0.1); /* Light background for visibility */
padding: 10px;
border-radius: 5px; /* Rounded corners */
cursor: pointer; /* Change cursor to pointer to indicate it's clickable */
transition: background-color 0.3s; /* Smooth background change on hover */
text-decoration: none; /* Remove underline from link */
}

.bill-box:hover {
background-color: rgba(97, 149, 185, 0.2); /* Darker background on hover */
}

.bill-box i {
margin-right: 8px; /* Space between icon and text */
color: rgb(124, 72, 71); /* Icon color */
}
```

```

</style>
</head>
<body>
  <!-- Header Section -->
  <header class="header">
    <div class="logo">
      <a href="user_home.php" class="are"><h2 style="color: white;">ArenaX</h2></a>
    </div>
    <nav>
      <ul style="display: flex; justify-content: center; width: 100%;">
        <li><a href="user_home.php">Home</a></li>
        <li><a href="user_indoor.php">Indoor</a></li>
        <li><a href="user_outdoor.php">Outdoor</a></li>
        <li><a href="user_fitness.php">Fitness</a></li>
        <li><a href="user_events.php">Events</a></li>
      </ul>
    </nav>
    <div style="margin-right: 20px; position: relative;">
      <button class="log"><?php echo htmlspecialchars($user_name); ?> <i class="fas fa-caret-
down"></i></button>
      <div class="dropdown">
        <a href="user_profile.php">PROFILE</a>
        <a href="user_bookings.php">BOOKINGS</a>
        <a href="user_calendar.php">CALENDER</a>
        <a href="user_payment_history.php">PAYMENT HISTORY</a>
        <a href="logout.php">LOGOUT</a>
      </div>
    </div>
  </div>
  </header>

  <div class="payment-history-container">
    <h2 class="mb-4 text-white" style="font-size: 2em; margin-bottom: 20px;">
      <i class="fas fa-history"></i> Membership Payment History
    </h2>

    <?php if (count($result) > 0): ?>
      <?php foreach ($result as $row): ?>
        <div class="payment-card">
          <p>
            <span style="font-size: 1.5em; color: white;">
              <?php echo date('d M Y', strtotime($row['payment_date'])); ?>
            </span>
          </p>
          <p><strong>Membership Type:</strong>
            <span class="badge status-badge">
              <?php echo htmlspecialchars($row['membership_type']); ?>
            </span>
          </p>
          <p><strong>Amount:</strong> ₹<?php echo intval($row['amount']); ?></p>
          <p><strong>Payment Time:</strong> <?php echo date('h:i A',

```

```

strtotime($row['payment_time'])); ?></p>
    <p><strong>Registration Date:</strong> <?php echo date('d M Y',
strtotime($row['membership_reg_date'])); ?></p>
    <div style="display: flex; align-items: center;">
        <p style="margin: 0; margin-right: 10px;"><strong>Bill:</strong></p>
        <?php if (!empty($row['bill'])): ?>
            <a class="bill-box" href="bills/<?php echo htmlspecialchars($row['bill']); ?>"
target="_blank">
                <i class="fas fa-file-download"></i>
                <span>Download Bill</span>
            </a>
        <?php else: ?>
            <span>No Bill Available</span>
        <?php endif; ?>
    </div>
</div>
<?php endforeach; ?>
<?php else: ?>
    <div class="no-payments">
        <i class="fas fa-info-circle fa-2x mb-3"></i>
        <p>No membership payment history found.</p>
    </div>
<?php endif; ?>

<!-- Booking Payment History Section -->
<h2 class="mb-4 text-white" style="font-size: 2em; margin-bottom: 20px;">
    <i class="fas fa-history"></i> Booking Payment History
</h2>

<?php
// Fetch regular booking payments
$sql_regular = "SELECT
    p.payment_id,
    p.amount,
    p.payment_date,
    p.payment_time,
    b.booking_date,
    b.bill,
    'Regular' as booking_type
FROM payment p
INNER JOIN booking b ON p.booking_id = b.booking_id
WHERE p.user_id = ? AND p.booking_id IS NOT NULL
ORDER BY p.payment_date DESC, p.payment_time DESC";

$stmt_regular = $conn->prepare($sql_regular);
$stmt_regular->bindValue(1, $user_id, PDO::PARAM_INT);
$stmt_regular->execute();
$regular_result = $stmt_regular->fetchAll(PDO::FETCH_ASSOC);

// Display regular booking payments
if (count($regular_result) > 0): ?>

```

```

<?php foreach ($regular_result as $booking_row): ?>
    <div class="payment-card">
        <p>
            <span style="font-size: 1.5em; color: white;">
                <?php echo date('d M Y', strtotime($booking_row['payment_date'])); ?>
            </span>
            <span class="badge status-badge" style="margin-left: 10px;">
                Regular Booking
            </span>
        </p>
        <p><strong>Amount:</strong> ₹<?php echo
number_format($booking_row['amount'], 2); ?></p>
        <p><strong>Payment Time:</strong> <?php echo date('h:i A',
strtotime($booking_row['payment_time'])); ?></p>
        <p><strong>Booking Date:</strong> <?php echo date('d M Y',
strtotime($booking_row['booking_date'])); ?></p>

        <?php if (!empty($booking_row['bill'])): ?>
            <div style="display: flex; align-items: center;">
                <p style="margin: 0; margin-right: 10px;"><strong>Bill:</strong></p>
                <a class="bill-box" href="<?php echo htmlspecialchars($booking_row['bill']); ?>"
target="_blank">
                    <i class="fas fa-file-download"></i>
                    <span>Download Bill</span>
                </a>
            </div>
        <?php endif; ?>
    </div>
<?php endforeach; ?>
<?php endif; ?>

<?php if (count($regular_result) === 0): ?>
    <div class="no-payments">
        <i class="fas fa-info-circle fa-2x mb-3"></i>
        <p>No booking payment history found.</p>
    </div>
<?php endif; ?>

<!-- Events Payment History Section -->
<h2 class="mb-4 text-white" style="font-size: 2em; margin-bottom: 20px;">
    <i class="fas fa-history"></i> Events Payment History
</h2>

<?php
// Fetch events payment history
$sql_events = "SELECT p.payment_id, p.amount, p.payment_date, p.payment_time,
                    e.event_title, er.bill
                FROM payment p
                INNER JOIN event_registration er ON p.event_reg_id = er.event_reg_id
                INNER JOIN events e ON er.event_id = e.event_id
                WHERE p.user_id = ? AND p.event_reg_id IS NOT NULL

```

```

ORDER BY p.payment_date DESC, p.payment_time DESC";

$stmt_events = $conn->prepare($sql_events);
$stmt_events->bindValue(1, $user_id, PDO::PARAM_INT);
$stmt_events->execute();
$events_result = $stmt_events->fetchAll(PDO::FETCH_ASSOC);

if (count($events_result) > 0): ?>
    <?php foreach ($events_result as $event_row): ?>
        <div class="payment-card">
            <p>
                <span style="font-size: 1.5em; color: white;">
                    <?php echo date('d M Y', strtotime($event_row['payment_date'])); ?>
                </span>
            </p>
            <p><strong>Event:</strong>
                <span class="badge status-badge">
                    <?php echo htmlspecialchars($event_row['event_title']); ?>
                </span>
            </p>
            <p><strong>Amount:</strong> ₹<?php echo intval($event_row['amount']); ?></p>
            <p><strong>Payment Time:</strong> <?php echo date('h:i A',
                strtotime($event_row['payment_time'])); ?></p>
            <?php if (!empty($event_row['bill'])): ?>
                <div style="display: flex; align-items: center;">
                    <p style="margin: 0; margin-right: 10px;"><strong>Bill:</strong></p>
                    <a class="bill-box" href="bills/<?php echo htmlspecialchars($event_row['bill']);
?>" target="_blank">
                        <i class="fas fa-file-download"></i>
                        <span>Download Bill</span>
                    </a>
                </div>
            <?php endif; ?>
        </div>
    <?php endforeach; ?>
<?php else: ?>
    <div class="no-payments">
        <i class="fas fa-info-circle fa-2x mb-3"></i>
        <p>No events payment history found.</p>
    </div>
<?php endif; ?>
</div>

<script>
const header = document.querySelector('.header');
window.addEventListener('scroll', () => {
    if (window.scrollY > 50) {
        header.classList.add('scrolled');
    } else {
        header.classList.remove('scrolled');
    }
}

```

```
});

const profileButton = document.querySelector('.log');
const dropdown = document.querySelector('.dropdown');
dropdown.style.display = 'none'; // Ensure dropdown is hidden initially

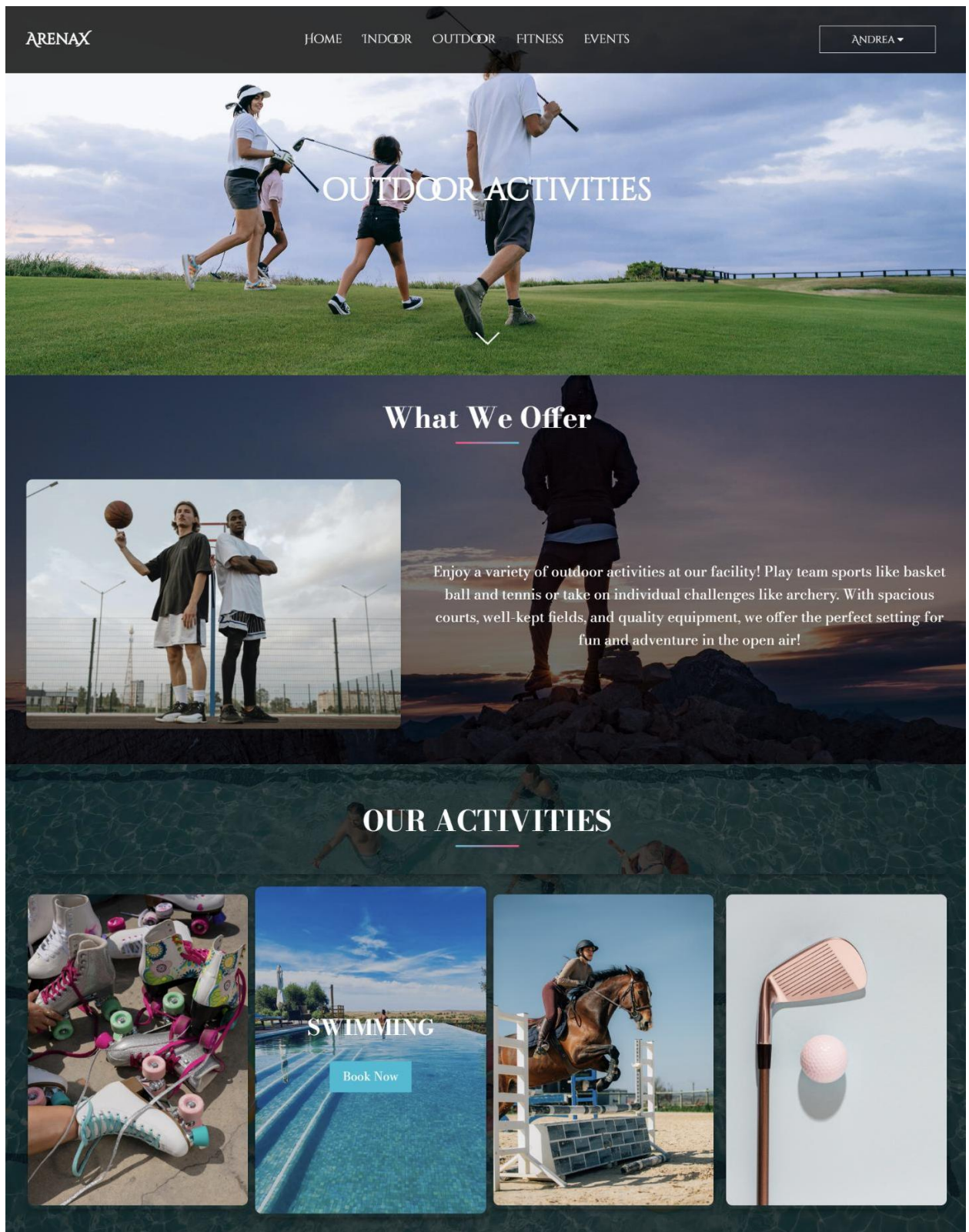
profileButton.addEventListener('click', () => {
  if (dropdown.style.display === 'none' || dropdown.style.display === '') {
    dropdown.style.display = 'block';
  } else {
    dropdown.style.display = 'none';
  }
});

document.addEventListener('click', function(event) {
  const dropdown = document.querySelector('.dropdown');
  const profileButton = document.querySelector('.log');

  if (!dropdown.contains(event.target) && !profileButton.contains(event.target)) {
    dropdown.style.display = 'none';
  }
});
</script>
</body>
</html>
```

9.2 Screen Shots

Outdoor Page



Booking activity:

ARENAX

HOMEINDOOROUTDOORFITNESSEVENTS

ANDREA

DETAILS



Swimming

ACTIVITY:

Swimming

BOOKING TYPE:

Single Booking

Recurring Booking

START DATE:

10-04-2025

NUMBER OF WEEKS:

1 WEEK

SELECT DAYS:

Mon

Tue

Wed

Thu

Fri

Sat

Sun

PREFERRED TIME:

9:00 AM - 10:00 AM

Booking Summary:

Start Date: 4/10/2025

End Date: 4/16/2025

Selected Days: Wednesday, Thursday

Total Sessions: 2

Total Price: ₹80.00

PRICE:

₹40.00

Proceed to Payment

Bookings Page:

ARENAX HOME INDOOR OUTDOOR FITNESS EVENTS ANDREA ▾

MY BOOKINGS

TABLE TENNIS

Activity Type: Indoor
Date: 10 April 2025
Time: 8:00 AM - 9:00 AM
Status: Upcoming
Booked on: 09 April 2025 at 10:06 AM
₹10.00

TABLE TENNIS

Activity Type: Indoor
Date: 10 April 2025
Time: 8:00 AM - 9:00 AM
Status: Upcoming
Booked on: 09 April 2025 at 10:06 AM
₹10.00

CHESS

Activity Type: Indoor
Date: 10 April 2025
Time: 9:00 AM - 10:00 AM
Status: Upcoming
Booked on: 09 April 2025 at 10:06 AM
₹10.00

FOOTBALL

Activity Type: Outdoor
Date: 10 April 2025
Time: 6:00 AM - 7:00 AM
Status: Upcoming
Booked on: 09 April 2025 at 10:08 AM
₹20.00

YOGA

Activity Type: Fitness
Date: 10 April 2025
Time: 9:00 AM - 10:00 AM
Status: Upcoming
Booked on: 09 April 2025 at 10:10 AM
₹20.00

CARDS

Activity Type: Indoor
Date: 10 April 2025
Time: 9:00 AM - 10:00 AM
Status: Upcoming
Booked on: 10 April 2025 at 9:00 AM
₹10.00 (Membership benefit)

BADMINTON

 Recurring

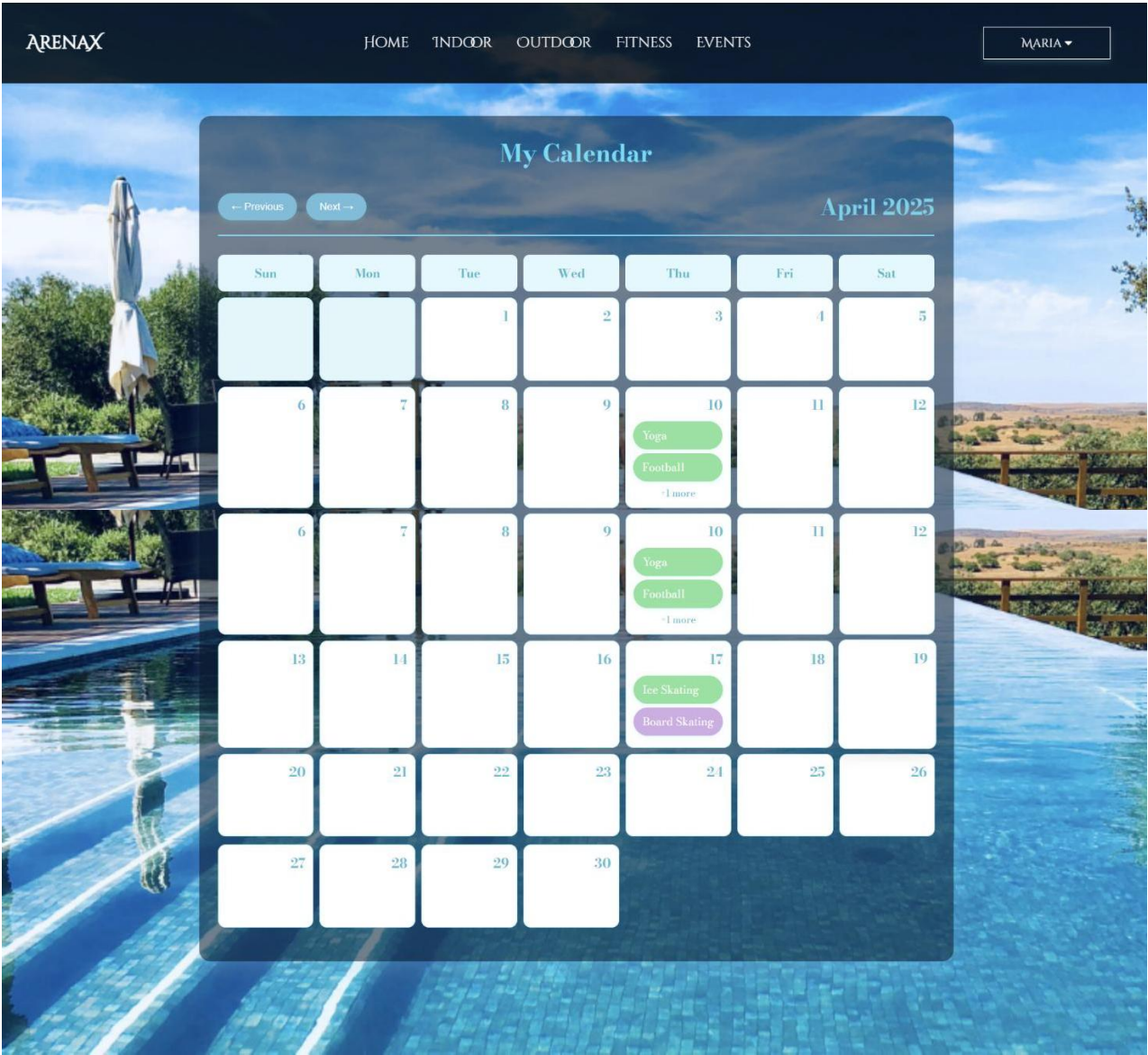
Activity Type: Outdoor
Period: 09 April 2025 to 15 April 2025
Time: 6:00 AM - 7:00 AM
Days: Thursday, Friday
Status: Active
Booked on: 09 April 2025 at 10:16 AM
₹20.00 per session

MY EVENT REGISTRATIONS

Rifle Shooting

Activity Type: Outdoor
Date: 12 April 2025
Time: 10:00 AM
Location: Rifle Court
Status: Upcoming
₹200.00

Calendar Page:



Payment History Page:

The screenshot displays the 'Payment History' page on the ArenaX website. The page features a dark header with the 'ARENAX' logo and navigation links: HOME, INDOOR, OUTDOOR, FITNESS, and EVENTS. A user profile dropdown for 'MARIA' is visible in the top right corner. The main content area is divided into three sections: 'Membership Payment History', 'Booking Payment History', and 'Events Payment History'. Each section contains a list of payment records for '09 Apr 2025'. Each record includes the membership type, amount, payment time, registration date, and a 'Download Bill' button.

Section	Date	Type	Amount	Time	Date	Action
Membership Payment History	09 Apr 2025	premium	₹2000	07:05 AM	09 Apr 2025	Download Bill
	09 Apr 2025	standard	₹1000	06:58 AM	09 Apr 2025	Download Bill
Booking Payment History	09 Apr 2025	Regular Booking	₹10.00	10:26 AM	09 Apr 2025	Download Bill
	09 Apr 2025	Regular Booking	₹20.00	10:23 AM	09 Apr 2025	Download Bill
Events Payment History	09 Apr 2025	Board Skating	₹100	07:00 AM		Download Bill

